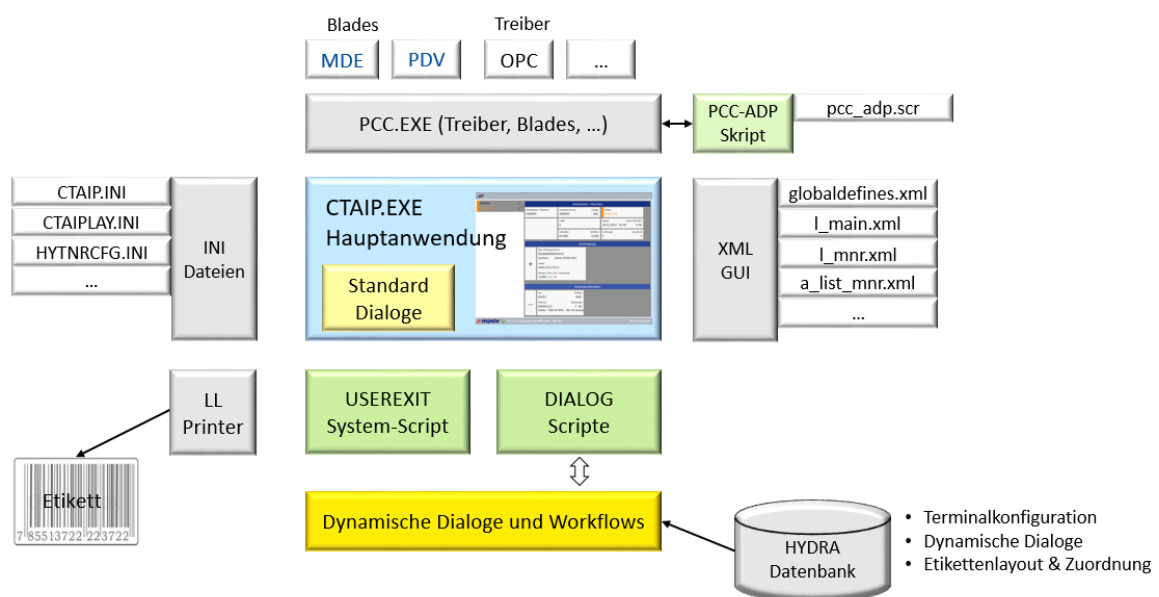


1 AIP2 - Scripting - Reference

1.1 Features

You can use the MES Development Suite (MDS) to change and extend the data collection functions of the Acquisition Information Panel (AIP2). For specific sections, the MDS provides user exits to implement changes of the standard processing.

This section describes the AIP2 processing of terminal scripts. The diagram below provides an overview of the structure and the logic components of the AIP2.



On the AIP2, the following types of terminal scripts are available:

- **USEREXIT (system script):**
USEREXITs are used to extend standard and customized functions on the AIP2.
- **DIALOG – script:**
DIALOG scripts are used to extend existing standard dialog functions or to implement new customer-specific dialog functions on the AIP2.
DIALOG scripts are used to control the configured dynamic dialogs.
- **PCC scripts:**
Further script functions are available to control or connect machines via the PCC interface (e.g. pcc_adp.scr). The documentation of the PCC scripts is included in the document "CUT-PCC_81_PCC-ADP_Kurzreferenz".

You can use the terminal scripts to implement the following changes and extensions, for example:

- Changing and extending the existing displays and dialog functions
- New dialog functions
- Interfacing of external interfaces
 - Blades
 - Drivers (e.g. OPC-UA)
 - Scanner
- etc.

1.2 Programming aids

1.2.1 Visual Basic

The script language used is based on VBScript. There are also so-called callback functions that are used as interfaces to the main application. The script functions available are described in the sections that follow.

1.2.2 Naming conventions

1.2.2.1 Script files



The file names of script files on the AIP2 can be in lower or upper case letters. The file name of the ZIP container may only have lower case letters because with Linux operating systems the ZIP files having upper case letters in their file names are not loaded!

ZIP container: The download of the terminal scripts from the server to the terminal is performed using a so-called ZIP container (ZIP file with extension .zip).

Note the following for script file names:

- Script files for the AIP2 always start with "aip_".
- Customer-specific DIALOG scripts start with "aip_U_" unless the name is otherwise specified by its intended use. This ensures that customer-specific scripts are overwritten by MPDV updates.
- Customer-specific USEREXIT scripts must start with "aip_system".
- Optionally, you can extend script files via project abbreviation/customer numbers and scopes.

1.2.2.2 Includes in script files

Note: Include files are supported as of AIP2 version V# 8.2.1.11.

You use include files to integrate other files into existing terminal scripts. A better structure of the scripts is then possible.

You can use the directive "`$<include-<Name>.scr>`" to integrate files into any row of the terminal script.

For reasons of downward compatibility, the include file is integrated as comment.

Include files are only loaded if

- the directive is at the beginning of the row.
- the include file starts with "`include-`" and has the extension "`.scr`".
- the include file is stored in the local directory (`.letc`, `.letc\var` or `.letc\local`) of the terminal.

Recursive loading of include files is not supported.

In a terminal script, an include file is loaded only once with the terminal script.

Example: File "include-custom-utils.scr":

```
'-----
' $Id: include-custom-utils.scr $
'-----
Sub doCustomValidation
' define Custom Validation for using function ...
  If VVar("UE:PAR","BTN.FKT") = "A_TR" Then
    ...
  End If
End Sub 'doCustomValidation
'-----
```

Include into "aip_system_<project>.scr":

```
<'-----
' $<include-custom-utils.scr>
'-----
Sub UserExitButtonClick '
  doCustomValidation
End Sub 'UserExitButtonClick
...
'-----
```

Only use the include files, if an organization of the scripts has advantages.

In case of a script error, a terminal script extract and the script or include file is displayed with the row of the error. (see section "Exception - Script - Dialog")

1.2.3 Scope Concept

The names of the terminal script files are based on the so-called scope concept. When names are assigned, the well-known scopes standard, custom, partner and local are supported.

MPDV standard	MPDV custom	Partner (@var)	Local (@local)
			aip2_<project>@local.zip aip2@local.zip local\aip_system@local.scr local\aip_DIALOG@local.scr
		aip2_<project>@var.zip aip2@var.zip var\aip_system@var.scr var\aip_DIALOG@var.scr	
	aip2_<project>.zip aip_system_<project>.scr aip_DIALOG_<project>.scr		
mpdv-aip.zip aip_mpdv-system.scr aip_mpdv-DIALOG.scr			

The more a scope is "special", the higher its priority. The special scope always takes priority over the general/standard scope. A file in the local scope takes priority over a file included in the standard scope.

New functions are only valid in the scope where they were implemented. They can be overridden by a scope of a higher priority.

The terminal script files are read and used in the following order/priority:

Scope	Prior ity	AIP 2	Description
MPDV	1	.laip_mpdv-system.scr .laip_mpdv-<dialog>.scr	MPDV standard
CUSTOM	2	.laip_system_<customer no>.scr .laip_<dialog>_<customer no>.scr	MPDV customization with customer number

Scope	Prior ity	AIP 2	Description
CUSTOM	3	<code>.laip_system_<project>.scr</code> <code>.laip_<dialog>_<project>.scr</code>	MPDV customization with project abbreviation
VAR	4	<code>.lvarlaip_system_<customer no>@var.scr</code> <code>.lvarlaip_<dialog>_<customer no>@var.scr</code>	Partner scripts (partner scope) for a customer project with customer number
VAR	5	<code>.lvarlaip_system@var.scr</code> <code>.lvarlaip_<dialog>@var.scr</code>	Partner scripts (partner scope) for standard partner software
LOCAL	6	<code>.llocalaip_system_<customer no>@local.scr</code> <code>.llocalaip_<dialog>_<customer no>@local.scr</code>	Customer scripts (local scope) of customer with project abbreviation and customer number
LOCAL	7	<code>.llocalaip_system@local.scr</code> <code>.llocalaip_<dialog>@local.scr</code>	Customer scripts (local scope) of customer

1.2.4 Storage structure of the scripts

ZIP files:

The terminal scripts are compressed and stored in a ZIP container on the server. The AIP2 downloads the ZIP files from the server and unpacks the files locally on the client under the directory `.etc`. If the download was successful, the ZIP file is locally unpacked.

Locally on the AIP2:

- `.etc` ; directory of extensions by MPDV (standard / custom)
- `.etc\var` ; directory of partner extensions (partner)
- `.etc\local` ; directory of extensions by the customer (local)

On the server:

Name and store the ZIP files for the relevant scope in the specified directory on the server as follows:

Scope	Prior ity	Storage structure of ZIP container on server	Description
MPDV	1	<code>.lctnet\win\aip2\etc\mpdv-aip.zip</code>	MPDV scripts (standard scope)
CUSTOM	2	<code>.lcustom\userexit\aip2_<customer number>.zip</code>	MPDV customization with customer number

Scope	Prior ity	Storage structure of ZIP container on server	Description
CUSTOM	3	.lcustom\userexit\aip2_<project>.zip	MPDV scripts (custom scope) for customer project
VAR	4	.lcustom\userexit\aip2_<project>@var.zip	Partner scripts (partner scope) for customer project
VAR	5	.lcustom\userexit\aip2@var.zip	Partner scripts (partner scope) for standard partner software
LOCAL	6	.lcustom\userexit\aip2_<project>@local.zip	Customer scripts (local scope) of customer with project abbreviation
LOCAL	7	.lcustom\userexit\aip2@local.zip	Customer scripts (local scope) of customer

Note: customer number and project are specified in the basic settings for each system.

1.2.5 Program parameters for developer mode

The following useful program parameters are available on the AIP2 in developer mode:

Parameters for development (ctaip.ini: INI section [**system**])

parameters= ... -AskForOverwriteScriptFiles -AlwaysReloadScript ...

„-AskForOverwriteScriptFiles“

Prevents overwriting of locally changed scripts on restart. Before unpacking the ZIP files, the AIP asks whether locally changed scripts should be overwritten.

„-NeverOverwriteScriptFiles“

Prevents overwriting of locally changed scripts on restart (without confirmation).

„-AlwaysReloadScript“

Reloads terminal scripts of the file each time its called. Use the button to process changes during runtime.

„-SkipAipStartUpUpdate“

Prevents overwriting of locally changed INI, CFG, XML files and DLLs on restart (without confirmation).

„-AskForRemoveDirectory“

Prevents deleting of local partner/customer extensions when the relevant directory is deleted on restart.

The AIP asks before deleting the following directories:

.letc\var\ ; directory for *partner extensions*

.letc\local\ ; directory for *customer extensions*

„-NeverRemoveDirectory“

Similar to „-AskForRemoveDirectory“. The directories are not deleted, there is no query.

„DEMO mode“

When you start AIP2 in DEMO mode, the ZIP containers are not unpacked.

1.2.6 Communication interfaces

The AIP2 provides different communication interfaces that are used to exchange data. The most important interfaces are the following:

Interface	Description
PDM commands	The PDM interface is used to send PDM commands (e.g. DLG=A_AN) to the server. You use PDM commands to send and book postings to the server.
PDM list requests	You use list requests to the server to request data as list file (e.g. mnr.lst – list of assigned machines).
File transfer	You can use the file interface to transfer files directly to or from the server.
Port (gateway)	You can use the input port to send PDM messages to the terminal. The external PDM client requires a relevant communication interface. e.g. update of lists
PCC interface	Blades, OPC, PCC-DIF (file interface) All of these interfaces are connected via the additional program PCC.EXE. It is possible to exchange messages between the PCC and the main application CTAIP.EXE and to process them specifically.
Scanner via COM interface	The scanners are logistically assigned to a COM port. If data is read via scanner, this data is provided to the AIP2 via interrupt handler. Via user exit, you can process this data in a specified manner. You assign the COM port in the CTAIP.INI.



The results of the lists are written in files on the server. The client passes the file name with a relative path to the server. The server creates the file. The client then loads the file and can process it.

The file should be created in the spool directory on the server.

Note: The file name must be unique per client. Only then, the server will not overwrite files of another request. If unique file names are not guaranteed, processes can be blocked on the server because these processes must access the same file.

You can use the following methods to assign unique file names:

- Integrate a unique number per client in the file name (e.g. with AIP use the user number = terminal number + 2000) On AIP2, the user number is included in the script variable SYS_USR.
- Integrate the current time stamp in the file name.

Examples:

- With user number: FILE=./spool/myfile**2043**.dat|
- With time stamp (format: MonDDhmmssMMM with milliseconds):
FILE=./spool/myfile**Dec31235959999**.dat|

1.2.7 Differences of the graphical user interface with and without XML GUI

With terminal scripting, there are 3 different GUI of the AIP2. The differences are as follows:

GUI	Description of the differences / notes
CTWIN	Similar to AIP8.1. But the buttons are situated at the bottom (via ctaipbut.ini). The graphic interface is displayed without skin. For the use of terminal scripts, there are no differences to the AIP8.1.
AIP 8.1	You configure the buttons in the main view via the file ctaipbut.ini.
XML GUI	The graphic interface is configured in XML. The main view displays the data as tiles. A selection is always performed when you open a detail dialog. The main view does not show any selected entries in the lists.

1.2.8 Static and temporary lists on the AIP

The AIP2 directory .\spool includes static and temporary list files, which are usually loaded from the server. The following section describes the most important files.

For more information on the content and properties, refer to the standard PDM documentation.



See the note on unique file names for lists on the server in section "1.2.6 Communication interfaces".

1.2.8.1 Static lists

aart.lst (order types)

Server command: DLG=LIST;87|..

Includes all order types configured in the system.

agrd.lst (scrap reason list)

Server command: DLG=LIST;84|MOD=T|TNR=706|..

The scrap reason list includes reasons for scrap/yield/rework/open quantity of the machines assigned to the terminal. (ART=G,A,N,P,...). It also includes SYSTEM reasons.

anr.lst (order list)

Server command: LIST;11|MOD=L|USR=2706|..

The order list includes all running orders of all machines, which are assigned to the terminal or logged on to the terminal.

bmkl.lst (list of RPA accounts)

Server command: DLG=BMK.LIST|..

The list of the resource performance accounts (RPA) includes all RPAs configured in the system.

bpos.lst (machine operator positions)

Server command: DLG=LIST;14|USR=2706|..

The list includes all operator positions of the machines assigned to the terminal.

hztyp.lst (material types)

Server command: DLG=LIST;21|..

The list includes all configured material types in the system (is only loaded with a machine in batch mode)

lizenz.lst (licenses)

Server command: DLG=LIST;48|..

The list includes all licenses and function keys.

mnr.lst (machine list)

Server command: LIST;10|USR=2706|..

The list includes all machines assigned to the terminal or dynamically assigned machines (via logon).

mstat.lst (machine status list)

Server command: DLG=LIST;16|MOD=T|USR=2706|..

The list includes all machine statuses of the machines that are assigned to the terminal in a fixed form (=configuration) or dynamically (logon, possibly only after server update).

paths.lst (directory list)

Server command: DLG=LIST;81|..

The list includes all paths of the modules configured in the system (DNC, DOK,...)

pnr.lst (list of persons)

Server command: DLG=LIST;12|USR=2706|MOD=V|..

The list includes all persons logged on to the machines of the terminal.

qrdfcg.lst (label printing – configuration (only with active license))

Server command: DLG=SYSTEM.CALL|PROG=hyettlst.scr|USR=2222|..

Includes all labels assigned that are active on the terminal and assigned to a dialog.

Note: the label definition is included in the sub folder llprinter of the AIP2

schicht.lst (list of shifts)

Server command: DLG=LIST;38|MOD=T|TNR=706|..

The list includes the shift configurations of the MDE machines assigned to the terminal

tkenn.lst (terminal label)

Server command: DLG=LIST;45|TNR=706|..

The list includes the configured terminal label with settings from the basic settings (e.g. batch number length).

tnrmat.lst (terminal – list of input material (only loaded with machine in batch mode))

Server command: DLG=LIST;13|MOD=T|USR=2706|..

Terminal – list of input material (only loaded with machine in batch mode)). The list includes all input batches/materials logged on to the machines of the terminal.

tnrres.lst (terminal – resource list (WRM))

Server command: DLG=LIST;129|MOD=T|USR=2706|..

Terminal resource list (WRM). The list includes all active resources of all machines of the terminal

tpe.lst (transport units (MPL))

Server command: DLG=LIST;52|..

The list includes all transport units configured in the system

vlpkz.lst (list of premium indicators)

Server command: DLG=LIST;24|..

The list includes all premium indicators created for all machines of the terminal.

zloueb.lst (material buffers / target locations (MPL))

Server command: DLG=LIST;49|TNR=706|..

List of all material buffers/target locations of the system.

vlist.<MNR>.lst (order sequencing list)

Server command: DLG=LIST;11|MOD=V|MNR=DBC1010|..

The order/sequencing list includes all operations for this machine if the status of the machine is configured to be displayed in the sequencing list (usually prepared and interrupted operations).

<MNR>_amat.lst (output batches)

Server command: --- / this list is only locally maintained by the terminal!

List of the output batch created per machine.

1.2.8.2 Temporary lists

mat.lst (list of input material)

Server command:
DLG=LIST;13|MOD=M|MNR=DBC1010|ANR=010001010010|DLG.DLGCFG=A_AN_MPL|..

This file includes the component list last loaded of an order at a machine (with logged on input batches).

NOTE: this file is only read in ONLINE mode (e.g. with OP logon or input batch change)

nanr.lst (order info)

Server command: DLG=LIST;11|MOD=A|ANR=010001010010|..

This file includes the order info last loaded of an operation.

amat.lst (list of produced output batches for an operation)

Server command: LIST;13|MOD=A|DBC1010|ANR=010001010010

List of produced output batches for an operation

1.3 Script – functions and variables

User exits are exit point that are called when the terminal main application is running. You can use the user exits to interfere in the terminal program and implement extensions.

The script functions are callback functions that the terminal main application provides for a script to read or change information or perform functions.

An AIP terminal script includes global variables valid in the execution context and different script functions to access these variables.

1.3.1 Script variables

1.3.1.1 UE_RET (general data exchange)

When used in read access, the complete content of the script variable **[n#]UE:RET** is returned.
[n#] If functions are called recursively, a reference index is added as prefix

Read access is performed with `VVar("UE:RET", "<DIgID>")`
 Or all values with `VVar("UE:RET", "#GET#ALL#VALUES#")`

If used in write access, the complete content of the script variable **[n#]UE:RET** is deleted if an empty string is assigned. If you assign a DIgID with value (e.g. "TEST=1"), an already existing entry is replaced. If the entry does not exist, it is added to the existing values.

e.g. `VVar("UE:RET", "U_ERRCODE")`

1.3.1.2 UE_SND (general data exchange)

If used in read access, the complete content of the script variable **[n#]UE:SND** is returned.
[n#] If functions are called recursively, a reference index is added as prefix

Read access is performed with `VVar("UE:SND", "<DIgID>")`
 Or all values with `VVar("UE: SND", "#GET#ALL#VALUES#")`

When used in write access, the complete content of the script variable **[n#]UE:SND** is deleted if an empty string is assigned. If you assign a DIgID with value (e.g. "TEST=1"), an already existing entry is replaced. If the entry does not exist, it is added to the existing values.

1.3.1.3 UE_RCV (general data exchange)

If used in read access, the complete content of the script variable **[n#]UE:RCV** is returned.
[n#] If functions are called recursively, a reference index is added as prefix

Read access is performed with `VVar("UE:RCV", "<DIgID>")`
 Or all values with `VVar("UE: RCV", "#GET#ALL#VALUES#")`

When used in write access, the complete content of the script variable **[n#]UE:RCV** is deleted if an empty string is assigned. If you assign a DIgID with value (e.g. "TEST=1"), an already existing entry is replaced. If the entry does not exist, it is added to the existing values.

1.3.1.4 DLGVAR (general data exchange)

If used in read access, the complete content of the script variable **[n#]DLG.DLG** is returned.
[n#] If functions are called recursively, a reference index is added as prefix

Read access of a value is performed with **VDlg ("#GET#ALL#VALUES#")** or **VVar("DLG.DLG", "#GET#ALL#VALUES#")**

See also **DLGOUT**

1.3.1.5 DLGSND (general data exchange)

A direct read access to this variable is not possible.

Read access of a value is performed with **VOut("<DlgID>")** or **VVar("DLG.OUT", "<DlgID>")**
 Read access of all values is performed with **VOut("#GET#ALL#VALUES#")** or
VVar("DLG.OUT", "#GET#ALL#VALUES#")
 Everything else is identical to **DLGOUT**

Special feature: deleting is performed with **'#DELETE#ALL#VALUES#'**
DLGSND="#DELETE#ALL#VALUES#"
sDlg=scrDeleteItems(sDlg,"EGT:GUT|EGT:AUS|EGT:GES")
DLGSND=sDlg

See also **DLGOUT**

1.3.1.6 DLGOUT (general data exchange)

If used in read access, the complete content of the script variable **[n#]DLG.OUT** is returned.
[n#] If functions are called recursively, a reference index is added as prefix

Read access of a value is performed with **VOut("<DlgID>")** or **VVar("DLG.OUT", "<DlgID>")**
 Read access of all values is performed with **VOut("#GET#ALL#VALUES#")** or
VVar("DLG.OUT", "#GET#ALL#VALUES#")

When used in write access, the complete content of the script variable **<[n#]DLG.OUT>** is **not** deleted if an empty string is assigned.
 If you assign a DlgID with value (e.g. "TEST=1"), an already existing entry is replaced. If the entry does not exist, it is added to the existing values.

Deleting **DlgID** with the function **EraseDlgOut("<DlgID>")**
 To delete all **DlgID** using the call **EraseDlgOut("#ERASE#ALL#DLG.OUT#")**

1.3.1.7 LSTVARS (general data exchange)

Here, a direct read access is not possible (e.g. MsgBox " LSTVARS " + LSTVARS)

[n#] If functions are called recursively, a reference index is added as prefix

For example: **LSTVARS = "LST.FILTER=" + "MNR=100 & ZUMAN=J"**
LSTVARS = "LST.MODE="+ "COLNUMSORT=TRUE|DYNAMICFILTER= MNR,MST"

Read access of a value is performed with **VVar("LST.MODE", "< COLNUMSORT >") == "TRUE"**
 Read access of all values is performed with **VVar("LST.FILTER", "#GET#ALL#VALUES#") == "MNR=100 & ZUMAN=J"**

When used in write access, the complete content of the script variable **[n#]LST...** is deleted if an empty string is assigned. During assignment (e.g. **LSTVARS = "LST.MODE=" + "COLNUMSORT=TRUE|**) the previously set value is completely replaced, i.e. there is no DlgID update.

To delete a "single entry", you use the assignment **LSTVARS = "LST.MODE="**
 To delete all values **LST.xyz** you use **LSTVARS = ""** or **EraseDlgVars("LST.")**

Used with - scrFktList - scrFieldChange (DynDlgFieldChange_XYZ) - scrFieldList
(DynDlgFieldListe_XYZ)

1.3.1.8 DD_SND (general data exchange)

If used in read access, the complete content of the script variable **[n#]UE:SND** is returned.
[n#] If functions are called recursively, a reference index is added as prefix

Read access is performed with **VVar("UE:SND", "<DlgID>")**
Or all values with **VVar("UE: SND", "#GET#ALL#VALUES#")**

When used in write access, the complete content of the script variable **[n#]UE:SND** is deleted if an empty string is assigned. If you assign a DlgID with value (e.g. "TEST=1"), an already existing entry is replaced. If the entry does not exist, it is added to the existing values.

1.3.1.9 DD_RCV (general data exchange)

If used in read access, the complete content of the script variable **[n#]UE:RCV** is returned.
[n#] If functions are called recursively, a reference index is added as prefix

Read access to a value is performed with **VRcv("<DlgID>")** or **VVar("DD.RCV", "<DlgID>")**
Read access to all values is performed with **VRcv("#GET#ALL#VALUES#")** or
VVar("DD.RCV", "#GET#ALL#VALUES#")

If used in write access, the complete content of the script variable **[n#]DD.RCV** is deleted if an empty string is assigned. If you assign a DlgID with value (e.g. "TEST=1"), an already existing entry is replaced. If the entry does not exist, it is added to the existing values.

1.3.1.10 SCRVARs (general data exchange)

Is identical to **LSTVARs** implementation
Here, a direct read access is not possible.
[n#] If functions are called recursively, a reference index is added as prefix

For example: **SCRVARs = "XXX.FILTER=" + "MNR=100 & ZUMAN=J"**
SCRVARs = "XXX.MODE=" + "COLNUMSORT=TRUE|DYNAMICFILTER= MNR,MST"

Read access to a value is performed with **VVar("XXX.MODE", "< COLNUMSORT >") == "TRUE"**
Read access to all values with **VVar("XXX.FILTER", "#GET#ALL#VALUES#") == "MNR=100 & ZUMAN=J"**

If used in write access, no script variable **[n#]xyz...** is deleted if an empty string is assigned. During assignment (e.g. **SCRVARs = "XXX.MODE=" + "COLNUMSORT=TRUE"**) the previously set value is completely replaced, i.e. there is no DlgID update.
To delete a "single entry", you use the assignment **SCRVARs = "XXX.MODE="**
To delete all values **<XXX.xyz>** you use **EraseDlgVars(" XXX.")**

1.3.1.11 GLOBALVARs (general data exchange)

When used in read access, the complete content of the global variable is returned. (if necessary several rows)
For example: **GLOBALVARs = "#X#=" + Item("1","1")+ Item("2","2")**
GLOBALVARs = "#Z#=" + Item("A","A")+ Item("B","B")

Read access to a value of a row **GVars("#X#", "1") == "1"**
Read access to all values with **GVars ("#Z#", "", "") == "A=A|B=B|"**

If used in write access, no script variable **[n#]xyz...** is deleted if an empty string is assigned.
Setting / saving DD items **GLOBALVARs = "#XXX#=" + Item("1","1")**
GLOBALVARs = "#XXX#=" + Item("2","2")

Is equal to
Item("2","2")
Update
To delete a DD item in a row, you use the assignment
To delete a "row", you use the assignment
!!! IMPORTANT !!! To delete the global memory, you use

GLOBALVARS = "#XXX#=" + Item("1","1")+
GLOBALVARS = "#XXX#=" + Item("2","333")
GLOBALVARS = "#XXX#=" + Item("2","")
GLOBALVARS = "#XXX#=" + ""
GLOBALVARS = "#DELETE#ALL#GLOBALVARS#"

1.3.1.12 SYS_IP (IP address of the terminal)

Only read access: IP address of the terminal (according to TNR status = variable otherwise via API function)

1.3.1.13 SYS_DDHEADER (dialog data header)

- Only read access: Dialog data header („DAT=09/17/2017|ZEI=48637|USR=2706|SWZ=S|USR=2706|ID=4|“)

1.3.1.14 SYS_USR (user number)

- Only read access: User number = terminal number (TNR) + 2000 → 2001 .. 2999

1.3.1.15 SYS_TNR (terminal number)

- Only read access: Terminal number (TNR) → 1 .. 999

1.3.1.16 SYS_DAT (terminal system date)

- Only read access: Terminal system date in format ("MM/DD/YYYY") → "09/17/2017"

1.3.1.17 SYS_ZEI (terminal system time)

- Only read access: Terminal system time in format ("NNNNN" = seconds per day) → "43200"

1.3.1.18 SYS_DT (terminal system date/time string)

- Only read access: Terminal system date/time string (current Windows setting) → "17.09.2017 12:00:00"

1.3.1.19 SYS_SCRIPT_DEBUG (terminal script debug window)

- only read access: Terminal script debug window (see section "Script - Debug - Dialog")

1.3.1.20 SYS_NEW_CNR_FR (standard production batch)

- Read only: Generate a lot number for a standard production batch (not suitable for customer-specific batch number assignment).

1.3.1.21 SYS_NEW_CNR_WE (standard goods receipt batch)

- Read only: Generate a lot number for a standard goods-receipt batch (not suitable for customer-specific batch number assignment).

1.3.1.22 SYS_QUEUE_ITEMS (number of QUEUE entries)

- Only read access: Number of QUEUE entries (in spool\ddqueue.dta)

1.3.1.23 SYS_OFFLINE (check OFFLINE / ONLINE)

- Only read access: Check OFFLINE / ONLINE (using hypdm32.dll function)

1.3.1.24 SYS_DEMO (terminal demo mode)

- Only read access: Terminal demo mode active

1.3.1.25 SYS_SCRFCT (terminal script function)

- Only read access: Outputs the current terminal script function

1.3.1.26 SYS_TNRGRP (terminal group)

- Only read access: Outputs the terminal group of the terminal (0 = "" otherwise "xxx")

1.3.1.27 SYS_PING (online PDM command)

- Only read access: an online PDM command is performed "DLG=SCMD;47|" to check if the services run on the server (MIP1 MW-LANT-Server <N>)

1.3.1.28 cFF* (field attributes)

For the dialog control of the dynamic dialog fields **DLGVAR = AddIt("ANR", "", cFFEnable)**

Note: several field attributes are added as follows **DLGVAR = AddIt("ANR", "", cFFEnable+"#F")**

(write access is not possible)

cFFReadOnly	; #RO	= readonly (field → set attribute READONLY)
cFFEnable	; #E	= Enable (enable field)
cFFDisable	; #D	= Disable (disable field)
cFFHide	; #H	= Hide (hide field)
cFFVisible	; #V	= Visible (show field)
cFFRequired	; #R	= Required (mandatory field)
cFFFocus	; #F	= Focused (focus field)
cFFBarcode	; #B	= Barcode (field → set attribute BARCODE)
cFFHideListBtn	; #HL	= Hide-List-Btn (hide list List-Btn of an input field)
other	; #N	= Nullable (field → set attribute NULL = without input)
	; #C	= Change field caption/text

1.3.1.29 DIR_APP (application directory)

- Only read access: Application directory (e.g. C:\MPDV\AIP2\)

1.3.1.30 DIR_SPOOL (spool directory)

- Only read access: spool directory (e.g. C:\MPDV\AIP2\SPOOL\)

1.3.1.31 DIR_ETC (etc directory)

- Only read access: etc directory (e.g. C:\MPDV\AIP2\ETC\)

1.3.2 Script functions

1.3.2.1 Availability of script functions

The sections in the following show for each function where the function can be used:

- (UE) Used in user exit in the main application
- (DLG) Used in dynamic dialog

1.3.2.2 VTnr (read value from terminal label)

(UE) + (DLG) VTnr ("AKRONYM")

Available: (UE) + (DLG)

Read info from the static list of the terminal label (TKENN.LST)

1.3.2.3 VVar(variable , acronym):string

Read transfer parameters

(UE) VVar ("UE:PAR", "XYZ")

Info on current machine from list of assigned machines (MNR.LST)

(UE) VVar ("UE:MNR", "XYZ")

Info on current operation from list of running operations (ANR.LST)

(UE) VVar ("UE:ANR", "XYZ")

1.3.2.4 rsIni(inidatei , sektion, key, default):string

(UE) + (DLG) rsIni ("ctaipay.ini", "main", "CLASSIC_ONLINE_LAMP", "")

Read INI file (with automatic writing of <default> if entry is not available)

To delete an entry, you use `scrExecute("DeleteIniKey"...`(see below).

1.3.2.5 `wsIni( inidatei , sektion, key, value ):string`

(UE) + (DLG) `wsIni("ctaisplay.ini","main","CLASSIC_ONLINE_LAMP","...")`

Write INI file.

1.3.2.6 `scrUECmd( ... ):string`

(UE) + (DLG) `scrUECmd(...)`

Execution of a PDM command with a file as result



See the note on unique file names for lists on the server in section "1.2.6 Communication interfaces".

```

*** load cost centers
UE_SND = ""
UE_SND = Item("DLG", "SYSTEM.CALL" )
UE_SND = Item("PROG", "custom_list.scr" )
UE_SND = Item("AKTION", "kostenst" )
UE_SND = Item("DATEI", ".\spool\kostenst."+SYS_USR )
UE_SND = Item("FILE", "kostenst.lst" )
' ----- UE_SND = Item("CMD:CPY", "BINARY" ) ' load binary if required
scrUECmd( UE_SND )

```

1.3.2.7 `SYS_SCRIPT_DEBUG`

(UE) + (DLG) `SYS_SCRIPT_DEBUG`

Open script debug window dialog. Shows all available variables.

1.3.2.8 `SYS_DT`

(UE) + (DLG) `SYS_DT`

Date/time stamp. (Example: 01/31/2020)

1.3.2.9 `SYS_NEW_CNR_FR`

(DLG) `SYS_NEW_CNR_FR`

Generate a new production batch number (no user exit batch number() support).

1.3.2.10 `SYS_NEW_CNR_WE`

(DLG) `SYS_NEW_CNR_WE`

Generate a new goods receipt batch number (no user exit batch number() support).

1.3.2.11 SYS_NEW_CNR_HU

(DLG) SYS_NEW_CNR_HU

Generate a new packaging (handling unit) batch number (no UserExitLosnummer() support).

1.3.2.12 AddIt(id,value,attribut)

(UE) + (DLG) AddIt(id,value,attribut)

Script function (aip_mpdv-system.scr)

A dialog item in format "ID=VALUE;ATTR" is generated (the attribute is only attached if the third parameter does not equal "").

List of the attributes (see also section on system variables, section "cFF* (field attributes)":

cFFReadOnly, cFFEnable, cFFDisable, cFFHide, cFFRequired, cFFFocus,
cFFBarcode, cFFHideListBtn

1.3.2.13 Item(id,value)

(UE) + (DLG) Item(id,value)

Script function (from aip_mpdv-system.scr)

A dialog item in format "id=value" is generated

1.3.2.14 IncStrDec (int)

(UE) + (DLG) IncStrDec (int)

Script function (from aip_mpdv-system.scr)

Decimal incrementing of an integer string (note: up to 15 digits maximum) e.g. IncStrDec("100") becomes "101"

1.3.2.15 StrFmtRight(Value,Len,char)

(UE) + (DLG) StrFmtRight (Value,Len,char)

Script function (aip_mpdv-system.scr)

Right-aligned formatting of a string with fill characters

For example:StrFmtRight("101", 5, "0") becomes "00101"

StrFmtRight("101", 2, "0") becomes "01"

1.3.2.16 MsgPopUp(msg,sec)

(UE) + (DLG) MsgPopUp (msg, sec)

Script function (from aip_mpdv-system.scr)

MsgPopUp "Ticket [XYZ] is printed." , "3"

If parameter sec = "" the info dialog must be closed with OK

1.3.2.17 VVar(item,id)

(UE) + (DLG) VVar (item, id)

Function to read from script VARS – Items

1.3.2.18 VTnr(id)

(UE) + (DLG) VTnr (id)

Function to read from 'TKENN.LST' - items

1.3.2.19 VPar(id)

(DLG) VPar (id)

Function to read 'DLG.PAR' – Items

VPar(id) can only be used in user exit **DynDlgInit**

If you open a dialog with script initialization,

REOPEN = TRUE/FALSE is set in **DLG.PAR**

FALSE = first request

TRUE = repeated opening after e.g. DB error

1.3.2.20 VMnr(id)

(DLG) VMnr (id)

Function to read 'DLG.MNR' - Items ('DLG.PAR' takes priority)

VMnr(id) is only used in the user exit **DynDlgInit**

1.3.2.21 VAnr(id)

(DLG) VAnr(id)

Function to read 'DLG.ANR' - Items ('DLG.PAR' takes priority)

VAnr(id) is only used in the user exit **DynDlgnit**

1.3.2.22 VVAR(„*ANR“,id); VVAR(„*MNR“,id)

(DLG) VVAR(„*ANR“,id); VVAR(„*MNR“,id)

Direct access to „DLG.ANR“ or „DLG.MNR“ → „DLG.PAR“ is bypassed!!

1.3.2.23 VDlg(id)

(DLG) VDlg(id)

Function to read 'DLG.DLG' items

1.3.2.24 VDat(offset)

(UE) + (DLG) VDat(offset)

Function to read to current date

in format „MM/DD/YYYY“

with <offset> = "0" = today

with <offset> = "-1" = yesterday

1.3.2.25 VZei(offset)

(UE) + (DLG) VZei(offset)

Function to read the current time in format "NNNNN" = seconds since midnight

with <offset> = "0" = now

with <offset> = "-30" = now – 30 seconds

1.3.2.26 GStore(func,filter)

(DLG) GStore(func,filter)

Access function to a grid (with FIRST,NEXT,ACTIVE) in 'DLG.GRD'

Note on the selection of grid rows:

Using the instruction `DLGVAR = Item("GRD.ROW", "<value>")` in a "DynDlg" user exit, you can make a selection in the current dialog grid. Possible values are:

- "FIRST" first grid row is selected
- "LAST" last grid row is selected
- "PREV" Current display position – 1
(if display position > 1)
- "NEXT" Current display position + 1
(if display position < "LAST")

With the return value of function `GStore(..)` you can select as follows:

- "0".."X" Position index – without taking into account any sorting
- "#" + "0".."X" Display position – taking into account a possible sorting

1.3.2.27 VStore(id)

(DLG) `VStore(id)`

Function for Store **GStore(..)** to read in 'DLG.GRD'

1.3.2.28 SStore(id, value)

(DLG) `SStore(id, value)`

Function for Store **GStore(..)** to write in 'DLG.GRD'

1.3.2.29 AStore(id)

(DLG) `AStore(id)`

Function for Store **GStore(..)** to add (write) in 'DLG.GRD'

1.3.2.30 VOut(id)

(DLG) `VOut(id)`

Function to read the dialog data transferred in user exit **DynDlgInit**

1.3.2.31 VSnd(id)

(UE) + (DLG) `VSnd(id)`

Function to read the dialog data sent in the user exits **DynDlgAfterSend** and **UserExitDynDlgAfterSend**

1.3.2.32 VRcv(id)

(UE) + (DLG) VRcv(id)

Function to read the reply returned from the server in the user exits **DynDlgAfterSend** and **UserExitDynDlgAfterSend**

1.3.2.33 EraseDlgOut(id)

(UE) + (DLG) EraseDlgOut(id)

Function to delete individual IDs from the dialog string ('DLG.OUT'). Usually in the user exits **DynDlgBeforeSend** and **UserExitDynDlgBeforeSend**

1.3.2.34 EraseDlgVars(id)

(DLG) EraseDlgVars(id)

Function to delete SCR-VARS in "LST.", "FKT.", "DD.":...

1.3.2.35 scrMsgBox(msg)

(DLG) scrMsgBox(msg)

The simple message box (only single string in parameter msg) is modal. This means that the script processing stops at this place and waits until OK is pressed. In case of a message box that is automatically closed after x seconds, the script processing is continued. If the parameter "vModal" is additionally set, the script processing waits also in case of automatically closed messages until the message is confirmed or is automatically closed.

scrMsgBox(msg)

Display of an info window with text (msg)

scrMsgBox("3^Hallo")

3^: Display for 3 seconds; message closes automatically

scrMsgBox("3|vModal|Caption^Hallo")

Caption: text displayed in the title bar of the message

vModal: the dialog is displayed as a modal dialog window

1.3.2.36 DlgJaNein(caption,msg)

(DLG) DlgJaNein(caption,msg)

Display of a query with the options Ja/Nein (Yes/No)

Example:

```
sRes=DlgJaNein("delete advance logon","really delete batch logged on in
advance?")
If sRes="#JA#" Then
    DeleteVLos(sMNR)
End If
```

If the user chooses "No" in the query, the function returns "#2#".

1.3.2.37 DlgJaNeinAbbruch(caption,msg)

(UE) + (DLG) DlgJaNeinAbbruch(caption,msg)

Message box from script for <Ja/Nein/Abbruch> query (Yes/No/Cancel).

The following values are returned: #JA#, #NEIN#, #CANCEL#

1.3.2.38 VDlg(id)

(DLG) VDlg(id)

Function to read 'DLG.DLG' items

1.3.2.39 scrFieldChange

(DLG) scrFieldChange

You use this function to link data and text fields.

For example: If you enter a status, the status text can be updated additionally.

For an example, refer to the description of the user exit **DynDlgFieldChange**.

1.3.2.40 scrFieldList

(DLG) scrFieldList

You use this function to implement a list selection in the user exit **DynDlgFieldList**.

LSTVARS LST.xxx are used

„LST.MODE=..|FORCEAREOPEN=TRUE|..“ (only with function *scrFieldList*)

Effect: Data is loaded from the hard disk. This way, it is possible to display a file that is already in memory after a server comparison.

For examples, refer to the user exit description.

1.3.2.41 scrFieldVAGList

(DLG) scrFieldVAGList

deprecated / backward compatibility / use scrFieldList

LSTVARS LST.xxx are used

(optional with loading from server)

1.3.2.42 scrFktList

(DLG) scrFktList

LSTVARS LST.xxx are used

Function like list selection without dialog

(optional with loading from server)

Transfer of data in [n#]DLG.OUT if **LST.FILTER=xyz** + **LST.RET=xyz** are set.

1.3.2.43 scrDDSnRcv(oSnd:AnsiString;var pSnd:AnsiString;var pRcv:AnsiString):integer

(UE) + (DLG) scrDDSnRcv(oSnd:AnsiString;var pSnd:AnsiString;var pRcv:AnsiString):integer

Function to send dialog data (see < scrDDSnRcv[WOErr] >)

for application DLL interface!

Parameters:

	oSnd	"original data sent"
var	pSnd	"actual data sent" (MST without ";x")
var	pRcv	"actual data received" (server result, DDQueue result, ..)

1.3.2.44 scrDDSn

(DLG) scrDDSn

*deprecated / backward compatibility / use **scrDDSndRcv***

The script variable **DD_SND** is used here.

Note: with **EraseDlgVars("DD.")** the old data memory of the variable **DD_SND** can be deleted.

new send parameter **PROCESSDLGEVENT=TRUE**

- Execution via **ProcessDlgEventSend** with local booking of standard events (A_AN, A_TR, A_UN, A_AB, M_MST, P_AN, P_AB, ..) and with active PCC/MDE interfacing with list notification.

New send parameter **\$TNR.KEEP_DATETIME=ON**

- Transferred time stamp (DAT/ZEI) is kept

New send parameter **LST_RELOAD=OFF**

- Reload request of the server is ignored.

1.3.2.45 scrDDSndWOErr

(DLG) scrDDSndWOErr

sends the BAPI string from DD_SND like scrDDSnd, but does not automatically issue an error that might be returned.

New send parameter <PROCESSDLGEVENT=TRUE>

- Execution via < ProcessDlgEventSend > with ScriptlokalUpdate

1.3.2.46 scrPCCValues(value)

(DLG) scrPCCValues (value)

This function sends data to the "pccdll.dll" and therefore to the machine via the respective driver. In combination with the UserExitPccDllToTerminal the implementation of control tasks is possible.

Return of the requested values in (with "DLG=GETVAL|..")

(1) customer system script **UserExitPccDllToTerminal**

(2) In addition, all requested "V:..." values are sent in an opened dynamic dialog as bar code.

The values transferred can be processed in the **DynDlgFieldExit_ XYZ** (XYZ=DynDlgKennung). (Can be identified via request

"FLD.MOD" = "BARCODE")

1.3.2.47 vbsGetCentralPccID(sFilter)

(UE) + (DLG) vbsGetCentralPccID(sFilter)

This function is required if the terminal does not start the PCC.EXE and thus the MDE itself locally, but the PCC.EXE runs at a different location than central MDE.

ctaip.exe V# 8.2.1.35 / pcc.exe V# 7.2.4.3 / mpdv-aip.zip 03.12.2018 / MQTT

Configuration: ctaip.ini → [DLL] → BusDLL=CENTRAL or PCC.EXE

The function searches a PCC ID in the file "**central.tnr.lst**" that matches the filter criterion. By default, the list only includes the machines that are assigned to a PCC/TNR as MDE machine.

Syntax: "TYP=<.>&ID=<.>"

Examples: "TYP=M&ID=MDE100" = PCC ID of the entry found or ""

 "FIRST", "LAST" = PCC ID of the first, last entry

1.3.2.48 vbsCentralPCCValues(sCMD,ByVal sPCCID)

(UE) + (DLG) vbsCentralPCCValues(sCMD,ByVal sPCCID)

This function is required if the terminal does not start the PCC.EXE and thus the MDE itself locally, but the PCC.EXE runs at a different location than central MDE. In this case, the vbsCentralPCCValues function must be used instead of the scrPCValues function.

ctaip.exe V# 8.2.1.35 / pcc.exe V# 7.2.4.3 / mpdv-aip.zip 03.12.2018 / MQTT

Configuration: ctaip.ini → [DLL] → BusDLL=CENTRAL or PCC.EXE

The function dispatches the command transferred <sCMD> (GETVAL,SETVAL) to the stand-alone-PCC/MDE terminal in combined operation with the PCC/terminal number <sPCCID>.

You can identify the <sPCCID> of an MDE machine using the function

"vbsGetCentralPccID("TYP=M&ID=<MNR>")".

1.3.2.49 scrExecDynDlg(dlg,ret,values)

(DLG) scrExecDynDlg(dlg,ret,values)

Function for DynDlg Aufrufe (without dialog script)

Parameter **ret** = RETURN is set, if DLG is not available.

1.3.2.50 rsCfg(Sektion,Key,Value)

(UE) + (DLG) rsCfg(Section,Key,Value)

Function to read an entry from the file **HyTnrCfg.ini** as **string**.

The configuration file HyTnrCfg.ini contains an additional 0 for all terminals or 2000+terminal number in the section if the section is to apply to only one terminal.

Example:

```
[Konfiguration 0]
Value = 10

[Konfiguration 2100]
Value = 20
```

The query rsCfg("Configuration", "Value", "") at terminal 100 returns the result 20. The result is 10 for all other terminals. A terminal group specific configuration is possible by storing the HyTnrCfg.ini in the terminal group specific subdirectory (e.g. .\hydra\<1>\custom\aip2\tgrp_901\). The 0 must then be used in the section so that all terminals in the group are addressed.

A default value can be transferred in the function parameter "Value", which is returned if the configuration does not exist in the file.

1.3.2.51 scrFileExists(file)

(UE) + (DLG) scrFileExists(file)

Checks if a file is available. If the file exists, the function returns 0.

Example:

```
If scrFileExists(DIR_SPOOL+"test.txt")="0" Then
    'File exists
End If
```

1.3.2.52 scrFileDelete(file)

(UE) + (DLG) scrFileDelete(file)

Delete file (OK → „0“)

Example:

```
scrFileDelete(DIR_SPOOL+"test.txt")
```

1.3.2.53 scrFileCopy(file,newfile)

(UE) + (DLG) scrFileCopy(file,newfile)

Copy file (OK → "0")

Example:

```
rc=scrFileCopy(DIR_SPOOL+"mat.lst",DIR_SPOOL+"mat.tmp")
```

1.3.2.54 scrFileRename(file,newfile)

(UE) + (DLG) scrFileRename(file,newfile)

Rename file (OK → "0")

Example:

```
rc=scrFileRename(DIR_SPOOL+"mat.lst",DIR_SPOOL+"mat.tmp")
```

1.3.2.55 GSrce(sFct,sParam)

(DLG) GSrce(sFct,sParam)

Access to static DD lists

Example:

```
Dim rc
rc=GSrce("LOAD","FILE="+DIR_SPOOL+"mstat.lst")
rc=GSrce("FIRST","MNR=110")
While rc<>"#EOF#STORE#"
    If bActive Then
        rc=SSrce("HARC:ID","1")
    Else
        rc=SSrce("HARC:ID","0")
    End If
    'Read access: sMST=VSrce("MST")
    rc=GSrce("NEXT","MNR=110")
Wend
rc=GSrce("CLOSE","SAVE=TRUE")
```

You will find further information in chapter „1.7.3 How to use the functions GSrce, VSrce“.

1.3.2.56 VSrce(sID)

(DLG) VSrce(sID)

Read access to DD list

1.3.2.57 SSrce(sID,sValue)

(DLG) SSrce(sID,sValue)

Write access (update)

1.3.2.58 ASrce(sID,sValue)

(DLG) ASrce(sID, sValue)

Write access (add)

Example:

```
rc=ASrce("EGR:GUTP", "5")
```

1.3.2.59 scrStatusBarMsg(sMsg,sMode,sSec)

(UE+DLG) scrStatusBarMsg(sMsg, sMode, sSec)

Output of messages via status bar

1.3.2.60 scrLog(sLine)

(DLG) scrLog(sLine)

Write to the log file (spool\script.txt) An exact time stamp is automatically set at the beginning of each line. The call stack of the user exit is at the end of the row.

1.3.2.61 scrReadRemoteFile(remote,local,params)

(DLG) scrReadRemoteFile(remote, local, params)

Function to read a file from the server

Example:

```
r1 = scrReadRemoteFile("./spool/tnr"+SYS_USR+".rld", DIR_SPOOL+"tnr"+SYS_USR+".rld",  
"CMD:LST=DELETE|CMD:CPY=BINARY|")
```



See the note on unique file names for lists on the server in section "1.2.6 Communication interfaces".

1.3.2.62 scrReadCfgFile(ssLstCmd,ssLstFile)

(DLG) scrReadCfgFile(ssLstCmd, ssLstFile)

Function to create (DLG=LIST;..) and read a file from the server

Example:

```
rc=scrReadCfgFile("DLG=LIST;11|MOD=A|MNR=M100|ANR=123450010", DIR_SPOOL+"nanr.lst")
```



See the note on unique file names for lists on the server in section "1.2.6 Communication interfaces".

1.3.2.63 **scrDeleteItemsInDlgLstFileWithFilter** (sFileName,sFilter,sParam)

(UE+DLG) `scrDeleteItemsInDlgLstFileWithFilter (sFileName,sFilter,sParam)`

Deletes entries in a DD list file that match the filtering

Example:

```
scrDeleteItemsInDlgLstFileWithFilter DIR_SPOOL+"lokvlst.lst", "ANR="+sAnr, ""
```

1.3.2.64 **scrMergeDlgLstFileIntoFile** (NewItemFile,SourceFile)

(UE+DLG) `scrMergeDlgLstFileIntoFile (NewItemFile,SourceFile)`

Merging of a DD list file to a target file.

New items are created in the target file.

Example:

```
rc=scrMergeDlgLstFileIntoFile(DIR_SPOOL+"u_l_seqlist.lst",DIR_SPOOL+"u_scrap_op.lst")
```

1.3.2.65 **scrQuickSearch(sFilename,sFilter)**

(UE+DLG) `scrQuickSearch(sFilename,sFilter)`

Searches in a DD list file for the first entry matching the filter.

If the value FIRST is passed as filter, then the first row is returned.

Example:

```
asMnr=scrQuickSearch(DIR_SPOOL+"mnr.lst", "MNR="+sMnr)
```

1.3.2.66 **scrClearDlgLstFile(sFilename)**

(UE+DLG) `scrClearDlgLstFile(sFilename)`

Deletes all rows in a DD list file except the header

Example:

```
scrClearDlgLstFile(DIR_SPOOL+"anr_x.lst")
```


1.3.2.67 scrCreateEmptyFile(sFilename)

(UE+DLG) `scrCreateEmptyFile(sFilename)`

Creates an empty file with size 0 byte

Example:

```
rc=scrCreateEmptyFile(DIR_SPOOL+"data.lst")
```

1.3.2.68 scrMergeDataIntoDlgLstFile(asData, asDlgLstFile,"TRUE")

(UE+DLG) `scrMergeDataIntoDlgLstFile(asData, asDlgLstFile,"TRUE")`

Generates a DD List file or adds a data line with all IDs contained in the DD List header. TRUE forces a reload of the file (e.g. ANR.LST) otherwise only the file itself is extended.

Example:

```
rc=scrMergeDataIntoDlgLstFile("CNR=1234|ATK=100|SLP=5|",DIR_SPOOL+"xmat.lst","FALSE")
```

1.3.2.69 scrGetDlgLstLine(sFilename,sLine)

(UE+DLG) `scrGetDlgLstLine(sFilename,sLine)`

Reads any row in a DD list file (sLine=„1“/ „2“ .. or „FIRST“/„LAST“)

Example:

```
asCnr=scrGetDlgLstLine(DIR_SPOOL+"mat.lst","3")
```

1.3.2.70 scrStr2Real(value):real

(DLG) `scrStr2Real(value):real`

Converts a string 123.256 into a real value.

Background: The values in a list created by the HYDRA server use a decimal point. In a list created by the HYDRA server, floating point values are always formatted with the period as decimal separator. The VBA functions for type conversion use the decimal separator set in the operating system, for example, in Germany, the comma. Therefore, use the function `scrStr2Real()` to convert the floating point numbers from a list works independently of the operating system settings. Likewise, you should use the function `scrReal2Str()` described below when writing information in lists or dialog strings. Therefore the function `scrStr2Real()` should be used when reading values from a list. Conversely, the function `scrReal2Str()` described below should be used when writing.

Example:

```
rValue=scrStr2Real (VStore ("EGR:GUTP"))
```

1.3.2.71 scrReal2Str(value:real):string

```
(DLG) scrReal2Str (value:real) :string
```

Converts a real value 123.256 into a string

Example:

```
rc=SSrce ("EGG:GUTS", scrReal2Str (rValue))
```

1.3.2.72 scrDDItem(sID,Values):string

```
(DLG) scrDDItem (sID,Values) :string
```

Identifies an item from a DD string

Example:

```
sCnr=scrDDItem ("CNR", asCnr)
```

1.3.2.73 scrStrReplace(Value,OldPattern,NewPattern):string

```
(DLG) scrStrReplace (Value,OldPattern,NewPattern) :string
```

Replaces <old strings> with <new strings> in a string

Example:

```
sNewString=scrStrReplace("Auftrag <ANR> nicht gefunden","<ANR>",sAnr)
```

1.3.2.74 scrEraseDDItem(sID,Values) :string

```
(DLG) scrEraseDDItem (sID,Values) :string
```

Deletes a DD item from a DD string

Example:

```
sNewString=scrEraseDDItem (asCnr, "DLL")
```

1.3.2.75 scrReplaceDDItem(sID, sItem,Values):string

```
(DLG) scrReplaceDDItem (sID, sItem,Values) :string
```

Replaces a DD item <slD> by <sltem> in a DD string <values>

Example:

```
sNewString=scrReplaceDDItem(asDat,"CNR",SYS_NEW_CNR_FR)
```

1.3.2.76 scrReplaceAllDDKennung(sVor,sNach,sValues,sNoCnvIDs)

```
(DLG) scrReplaceAllDDKennung(sVor,sNach,sValues,sNoCnvIDs)
```

Replaces all DD items of <sValues> mit Präfix <sVor> and Suffix <sNach> and leaves <sNoCnvIds>

Example:

```
asDat=scrReplaceAllDDKennung("V.", ".N", "DLG=XX|MNR=1|X=3|ID=5|", "DLG|ID")
result string = "DLG=XX|V.MNR.N=1|V.X.N=3|ID=5|"
```

1.3.2.77 scrGetPart(sString,sSeparator,sIndex)

```
(DLG) scrGetPart(sString,sSeparator,sIndex)
```

Returns a substring of <sString> with separator <sSeparator> with index <sIndex>

Examples:

```
scrGetPart("R|W|Q","|","1")    →  "R"
scrGetPart("R|W|Q","|","3")    →  "Q"
scrGetPart("R|W=T|Q=Z","=", "2") →  " T|Q"
```

1.3.2.78 scrPosStr(ssSubString,ssString)

```
(DLG) scrPosStr(ssSubString,ssString)
```

Checks if a substring **ssSubString** is contained in **ssString** .

Example:

```
sItem=scrPosStr("DLG=", "XXX=100|DLG=12|..") → "DLG=12|.."
```

1.3.2.79 scrLosnummer(sParam,sMnr,sAnr)

```
(DLG) scrLosnummer(sParam,sMnr,sAnr)
```

Function for the customer-specific generation of batch numbers
Internally calls the user exit **UserExitLosnummer**.

Param:

CNR->TYP= **cmFertigung**,cmWarenEingang,cmVerpackung

Example:

```
sCnr=scrLosnummer ("CNR->TYP=cmVerpackung", "", "")
```

1.3.2.80 scrStoreUpdate(sMode,sID,sValue)

(DLG) `scrStoreUpdate(sMode,sID,sValue)`

Fuction for a local update of the list files ANR.LST and MNR.LST in user exit
UserExitLocalMnrAnrUpdate

Explanation <sMode> = "READ" <sID> = "XYZ"

→ reads value from DD list

<sMode> = "ADD" <sID> = "XYZ" <sValue> = "10"

→ adds <10> to value in DD list

<sMode> = "UPDATE" <sID> = "XYZ" <sValue> = "ABC"

→ updates value in DD list to <ABC>

Examples and further information, see user exit **UserExitLocalMnrAnrUpdate**

1.3.2.81 scrTranslate(Text,Data)

(DLG) `scrTranslate(Text,Data)`

Function to translate texts to other languages.

Example:

Text: "The password of the person [<PNR>]<n>runs on<PWD:VALIDTG>. day(s)."

Data: "RET=0|KT=|LT=|INFO=3645|PNR=44444444|PWD:VALIDTG=1|ID=152|"

Notes: The placeholders <XYZ> are replaced from "Data".

<n> = line feed + <t> = tabulator

1.3.2.82 scrWriteRemoteFile(local,remote)

(DLG) `scrWriteRemoteFile(local,remote)`

Function to write a local file to the server



See the note on unique file names for lists on the server in section "1.2.6 Communication interfaces".

Example:

```
Dim sDBFileName,sLocalFileName,asRet
sDBFileName=scrGetInfo("HydraPath","")+ "spool\barcodes."+SYS_TNR
sLocalFileName=DIR_SPOOL+"barcodes.lst"
asRet=scrWriteRemoteFile(sLocalFileName,sDBFileName)
If scrDDItem("RET.OK",asRet)="TRUE" Then
    ' File was successfully transferred
End If
```

1.3.2.83 scrProcessQuickReportPrinterForDialog (dlg,data)

(DLG) scrProcessQuickReportPrinterForDialog (dlg,data)

Enables printing a label without sending the assigned dialog

Example:

```
rc=scrProcessQuickReportPrinterForDialog("U_ETK",asPrint)
```

1.3.2.84 scrProcessQuickReportPrinter (dlg,data,ret)

(DLG) scrProcessQuickReportPrinter (dlg,data,ret)

Function to print a configured label with transfer of the return value of the PDM command <RET>

1.3.2.85 scrExecuteQuickReportPrinter(params,file)

(DLG) scrExecuteQuickReportPrinter(params,file)

Function to print a file in RPB format. This function is used by default in the function "Label reprint (EV_NDRUCK)".

1.3.2.86 vbsFolderExists(sFolder)

(DLG) vbsFolderExists(sFolder)

Example see <aip_mpdv-system.scr>

Checks if the directory exists -> OK = "0"

1.3.2.87 vbsFolderCreate(sFolder)

(DLG) vbsFolderCreate(sFolder)

Creates directory -> OK = "0"(exists) or "1"(has been created)

1.3.2.88 vbsFileExists(sFile)

(DLG) vbsFileExists(sFile)

Checks if file exists -> OK = "0"

1.3.2.89 vbsCreateFolderTree(sFolder)

(DLG) vbsCreateFolderTree(sFolder)

Creates a directory tree

1.3.2.90 vbsValidateFolder(sFolder)

(DLG) vbsValidateFolder(sFolder)

Identify directory string with closing "\" and placeholders <DIR_APP> + <DIR_SPOOL>.

1.3.2.91 scrWriteDataIntoFile(asData,asFile)

(DLG) scrWriteDataIntoFile(asData,asFile)

Attaches a data string to a file or creates it if it does not exist.

Example:

```
Dim sHeader,rc
sHeader="MNR=Maschine|ANR=Auftrag|KNR=Kartenummer|"
rc=scrFileDelete(DIR_SPOOL+"u_pnr.lst")
rc=scrWriteDataIntoFile(sHeader,DIR_SPOOL+"u_pnr.lst")
```

1.3.2.92 scrAddAction(sAction,Param,Data)

(UE) + (DLG) scrAddAction(sAction,Param,Data)

Saves an action that is processed in the main loop of the application (CTAIP).

Example:

```
rc=scrAddAction("mtaDIALOG","DLG=AUTO:CA_WL_RS|..","MNR=100|..")
```

➔ saves an action to automatically open a dialog

Further notes:

- with Data = Item("ANR+MNR","RELOADED.WITH.VALUES")

the start dates ANR/MNR row are identified using the parameters

In the example above, MNR=100 is used irrespective of the currently selected machine row to start the script dialog.

Note: this function is only possible when you use script dialogs!

- scrAddAction("#STATE#", "#BASIC#", "") returns the number of actions and the execution status

- „0|0“ = no action / no action / dialog open

- „1|1“ = 1 action / Action/Dialog open

IMPORTANT: You can only transfer data to the dialog if the user exit **USEREXITButtonClick** is implemented in the script.

1.3.2.93 GVars(id,item)

(UE) + (DLG) GVars(id,item)

Saving data in the script with GLOBALVARS = „ABC=XYZ=1|...“

Global buffer of variables

Example:

```
GLOBALVARS="DATA=DLG=A_AN|MNR=M100|ANR=123450010|KNR=9999| "
sDlg=GVars("DATA","DLG")
sAnr=GVars("DATA","ANR")
```

Every time a dialog is opened via script, the call parameters "#{DLG-ID}#PAR#" are saved.

Additional "#{DLG-ID}#ANRR#" , "#{DLG-ID}#MNR#" , ..

e.g. xRowID = GVars("#CE_WL_RF#PAR#", "ID#ROW")

1.3.2.94 scrEvaluateDuration(sDatB,sZeiB)

(UE) scrEvaluateDuration(sDatB,sZeiB)

Creation of a continuous string in the configured format to be displayed in the OP info.

The parameters sDatB and sZeiB are passed in MPDV format (MM/DD/YYYY, Sec. since midnight). The duration since this point in time is calculated.

Example:

```
s=scrEvaluateDuration(scrDDItem("DAT",asDat),scrDDItem("ZEI",asDat))
```

1.3.2.95 scrFormatDuration(sSeconds)

```
(UE) scrFormatDuration(sSeconds)
```

Formats a duration in seconds and uses the specified format (industrial time unit, if required) for the display in the OP info

1.3.2.96 scrFormatTimeStamp(sDat,sZei)

```
(UE) scrFormatTimeStamp(sDat,sZei)
```

Formats a time stamp in MPDV format for the display in the OP info (hh:mm dd.mm.yyyy)

1.3.2.97 scrUrlDownload

(scheme,user,password,host,port,url_path,loc_path,prot_path:AnsiString):integer;

```
(UE) + (DLG) scrUrlDownload
(scheme,user,password,host,port,url_path,loc_path,prot_path:AnsiString):integer;
```

Load files via URL download

Example:

```
iRes=scrUrlDownload("hydra","hydadm","hydadm","win2003-3","10403",
"\hydra724\dncfiles\H10007410750.pdf",".\spool\Temp.pdf",".\spool\prot_ev.txt")
```

1.3.2.98 Ret=scrUrlDownload2(Path,FileName)

```
(UE) + (DLG) Ret=scrUrlDownload2(Path,FileName)
```

Simplified call of URL download:

Path: using this string, the download parameters are read from „Paths.lst“

1.3.2.99 scrUrlUpload

(scheme,user,password,host,port,url_path,loc_path,prot_path:AnsiString):integer;

```
(UE) + (DLG) scrUrlUpload
(scheme,user,password,host,port,url_path,loc_path,prot_path:AnsiString):integer;
```


Copy files via URL upload (same syntax as scrURLDownload)

1.3.2.100 scrSplitOrder(sAuftrag)

(UE) + (DLG) scrSplitOrder(sAuftrag)

Returns all separate IDs for an order number (ANR, AUNR, AFOLG, AGNR, UAGNR, SPLNR)

Example:

```
asDat=scrSplitOrder("123450010")
```

```
ANR=123450010|AUNR=12345|AFOLG=|AGNR=0010|UAGNR=|SPLNR=
```

1.3.2.101 scrDateTime(Mode:Ansistring):double

(UE) + (DLG) scrDateTime(Mode:Ansistring):double

Function to read the TickCount/Now → Result = DOUBLE

- "TC" = provides the current time in seconds since the program has been started.
- "TCMS" = provides the current time in milliseconds since the program has been started.
- "TCSYS" = provides the current time in seconds since the computer has been started.
- "TCSYSMS" = provides the current time in milliseconds since the computer has been started.
- "DTMS" = Time in milliseconds since 30 December 1899
- "", "DT" = Time in seconds since 30 December 1899

1.3.2.102 scrGetInfo(Fkt,Param:string):string;

```
scrGetInfo(Fkt,Param:string):string;
```

(UE) + (DLG) scrGetInfo(Fkt,Param:string):string;

This function requests data from the terminal.

GetPLock

```
scrGetInfo("GetPLock","MASCH100")= "J"/"N"
```

Prod. lock active?

HasShift

```
scrGetInfo("HasShift", "MASCH100") = "J"/"N"
```

Machine has shift?

GetAllOrdersOfMachine

```
sAnrLst=scrGetInfo("GetAllOrdersOfMachine", <Maschine>)
```

Returns orders logged on to machine separated by commas.

GetParallelOrders

```
sAnrLst=scrGetInfo("GetParallelOrders", <order>)
```

Returns all orders logged on in parallel to the same machine including the order transferred (separated by comma).

GetDefaultPerson

```
scrGetInfo("GetDefaultPerson", "MNR=4711")
```

Specified person to get dialogs (only if HoldPersonInfo=on is set)

GetDlgBufferValue

```
scrGetInfo("GetDlgBufferValue", "DLG=@ACTIVE | AKRO=ANR")
```

Read value from dialog (set values with scrSetData("SetField"..)

GetPathData

```
scrGetInfo("GetPathData", "PATH="+sPath+" | FILE="+sLoadDateiName+" | EXT=")
```

Query of path data → SCHEME=FTP|USR=TK|PWD=123...

GetSelected

```
scrGetInfo("GetSelected", "")
```

Returns selected machine and order → MNR=..|ANR=..

GetProductionLock

```
scrGetInfo("GetProductionLock","4711")
```

Query production lock of a machine → „J“/„N“

GetGridLineWithFilter

```
scrGetInfo("GetGridLineWithFilter","ATK=12345")
```

Only (DLG)

Read a line of the dialog grid specified by the filter.

GetGridData

```
scrGetInfo("GetGridData","-1")
```

Only (DLG)

Read active row of the dialog grid

```
asGrid=scrGetInfo("GetGridData","DLG=@FIL=DLG=A_AN|LINE=-1")
```

Extension by filter dialog

GetBatchMode

```
scrGetInfo("GetBatchMode","MNR=4711")
```

Read batch mode of a machine →

ImNormal,ImLos,ImDurchlauflos,ImChargenVerw,ImUser1,ImUnknown

IsDlgFieldVisible

```
scrGetInfo("IsDlgFieldVisible","DLG=@ACTIVE|AKRO=MNR")
```

Only (DLG)

Query if a field is visible in the dialog → Y/N

GetDDlgEntries

```
scrGetInfo("GetDDlgEntries", "TNR=100|DLG=GEB_DRU|TGRP=42")
```

Read all configured dialog fields of a dialog

GetOrderData

```
scrGetInfo("GetOrderData", "ANR=TK0000000010")
```

Reads order row from file anr.lst, vlist.lst or from server (nanr.lst)

GetButtonCaption

```
scrGetInfo("GetButtonCaption", "DLG=@ACTIVE|FKT=FKT=WG1")
```

Read button text from a dynamic dialog (reference via FKT)

GetTimeDiff

```
scrGetInfo("GetTimeDiff", "D1=11/04/2018|T1=48744|D2=11/04/2018|T2=52000")
```

Specification of the number of seconds between two time stamps

GetTimeStamp

```
scrGetInfo("GetTimeStamp", "")
```

Returns the time stamp of the last MDE query in the format `yyyymmddhhnnss`

GetMachineData

```
scrGetInfo("GetMachineData", "MNR=4711")
```

Reads the complete row from the machine list

→ MNR=4711|MGRP=122|MBEZK=..

```
scrGetInfo("GetMachineData", "MNR=4711|AKRO=MST")
```

Reads a specific value from the machine list

GetSelectedGridData

```
scrGetInfo("GetSelectedGridData", "LIST3")
```

Reads the complete data row of the selected row from the local lists.

MNR – selected machine

ANR – selected order

LIST3 – 3. list (material, person, resource)

IniSectionExists

```
scrGetInfo("IniSectionExists", "INIFILE=ctaiplay.ini|SECTION=CE-Scan-Liste")
```

Check if a section in the INI file is included ("0"→Yes / "-1"→No)

GetScriptStack

```
scrGetInfo("GetScriptStack", "")
```

Returns stack of user exits - to analyze recursive calls

1.3.2.103 scrSetData (set data)

```
scrSetData( funktion, params):string
```

These function sets data in the terminal or in the dialogs.

SetFocusToField

```
scrSetData("SetFocusToField", "DLG=@ACTIVE|AKRO=EGR:PRB|RED=1")
```

Set focus in the active dialog to the field "EGR:PRB" and color the field red

PressButton

```
scrSetData("PressButton", "DLG=@ACTIVE|RCODE=0")
```

Press keys of dialog from script ' RCODE=0->OK 1->Cancel

```
scrSetData("PressButton", "DLG=@ACTIVE | AKRO=NEXT")
```

Press keys of dialog from script → Selection of button via ID

SetField

Setting content of an input field

```
scrSetData("SetField", "DLG=@ACTIVE | AKRO=CNR | VALUE=1000000000")
```

Setting colors

```
rc=scrSetData("SetField", "DLG=@ACTIVE | AKRO=CNR | FONT.COLOR=clLime | CAPTION.FONT.COLOR=clLime")
```

FONT.COLOR: Font color of the field content

CAPTION.FONT.COLOR: Color of description (Caption)

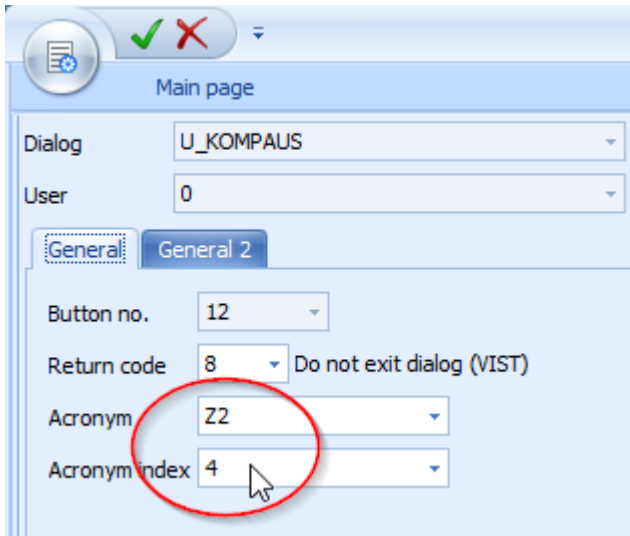
Note: The colors cannot be set with all field types.

Setting labeling of function keys

For buttons of type "ACTIONBUTTON" in the configuration.

```
rc=scrSetData("SetField", "DLG=@ACTIVE | AKRO={ButtonAcronym} [{ButtonAcronymIndex} | BUTTON.CAPTION=XXX")
```

Example:



Dialog: U_KOMPAUS

User: 0

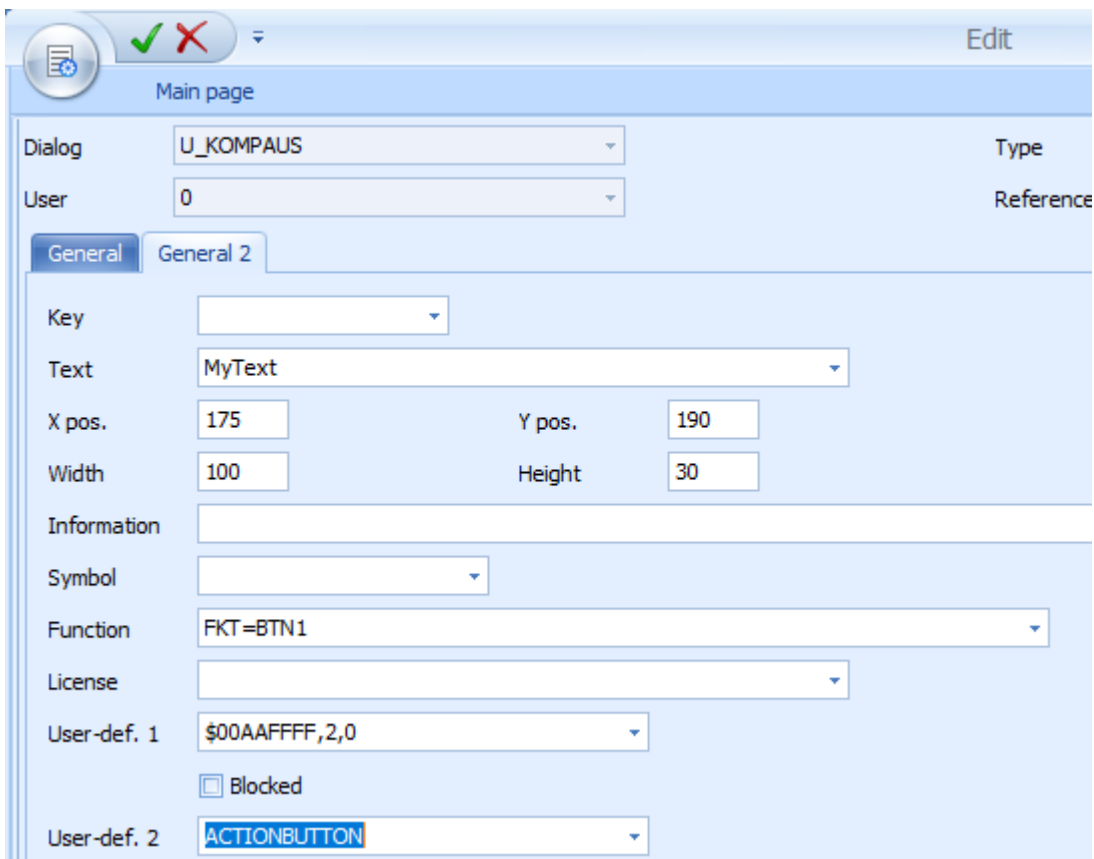
General 2

Button no.: 12

Return code: 8 Do not exit dialog (VIST)

Acronym: Z2

Acronym index: 4



Dialog: U_KOMPAUS

User: 0

General 2

Key:

Text: MyText

X pos.: 175 Y pos.: 190

Width: 100 Height: 30

Information:

Symbol:

Function: FKT=BTN1

License:

User-def. 1: \$00AFFFFF,2,0

☐ Blocked

User-def. 2: ACTIONBUTTON

```
rc=scrSetData ("SetField", "DLG=@ACTIVE | AKRO=Z2 : 4 | BUTTON.CAPTION=MyNewCaption")
```

Using this function, you can also change buttons in the button bar of the dialog (as of AIP version 8.2.0.36).

SetFocusToButton

```
scrSetData ("SetFocusToButton", "DLG=@ACTIVE | RCODE=0 ")
```

Set focus to a key that produces the specified return code.

Standard return codes:

- RCODE=0=>OK
- RCODE=1 => Cancel

SetButtonVisible

```
scrSetData("SetButtonVisible", "DLG=@ACTIVE|FKT=OK|ACTION=HIDE")
```

Show/hide button, ACTION=SHOW/HIDE/READ/TGL

ACTION=DISABLE/ENABLE is also possible.

ButtonClick

```
scrSetData("ButtonClick", "CA_WL")
```

Triggers clicking a button in main program (ctaip) (the acronym transferred is identical to the entry in ctaipbut.ini or to the identifier in the XML GUI).

ProtIntoFile

```
rc=scrSetData("ProtIntoFile", "PROTFILE="+sProtDatei+"|MSG="+sMessage)
```

Logging of messages in any file in the directory spool (time stamp is automatically put in front).

SelectData

```
rc=scrSetData("SelectData", "MNR=4711|ANR=12345")
```

Select machine and/or order on the terminal

SetProductionLock

```
rc=scrSetData("SetProductionLock", "MNR=4711|ACTIVE=1|")
```

Set/release production lock of a machine

DelayedButtonClick

```
rc=scrSetData("DelayedButtonClick","CA_WL")
```

Delayed triggering of a button click. The button is clicked when the timer event is released in the main timer.

Advantage: If the button click opens a dialog, the script processing continues in the background.

```
rc=scrSetData("DelayedButtonClick","CA_WL|FORCEDIALOG=ON")
```

The dialog is repeated until "OK" is pressed.

```
rc=scrSetData("DelayedButtonClick","DLG=@ACTIVE|RCODE=0")
```

Click button with delay in the dialog via RCODE=0

```
rc=scrSetData("DelayedButtonClick","DLG=@ACTIVE|FKT=FKT=SEND")
```

Triggers specified function of the dialog with delay

```
rc=scrSetData("DelayedButtonClick","CA_WL|CLOSE_ALL_DLG=ON|")
```

Before triggering the dialog, all opened dynamic dialogs are closed

```
rc=scrSetData("DelayedButtonClick","DLG=@AINFO|BTN.FKT=AI_CLOSE")
```

Click button in the OP info

(also for DLG=@MINFO)

```
rc=scrSetData ("DelayedButtonClick", "DLG=@ACTIVE | KENN=LOS_MELDEN")
```

The button with KENNUNG=LOS_MELDEN in the dialog is clicked with delay.

UpdateGrid

```
rc=scrSetData ("UpdateGrid", "DLG=@ACTIVE | GRD.ACCESS=TRUE | DLG.GRID=REOPEN")
```

Reread grid of a dynamic dialog

```
rc=scrSetData ("UpdateGrid", "DLG=@ACTIVE | GRD.ACCESS=TRUE | DLG.GRID=RELOAD")
```

Updates list of server and rereads the grid of a dynamic dialog (is only supported if it is a SCRIPT-GRID or WF-GRID and is configured "CMD").

```
rc=scrSetData ("UpdateGrid", "DLG=@ACTIVE | GRD.ACCESS=TRUE | GRD.FILTER=LEVEL=1 | GRD.ORDER=ART | ")
```

Dynamic dialog with grid: reset filter

- GRD.FILTER=<ALL> (no filtering)
- GRD.ORDER=ART (specify sorting)

```
rc=scrSetData ("UpdateGrid", "DLG=@FIL=DLG=CE ASW_RF | GRD.ACCESS=TRUE | GRD.FILTER=ABKZ=N & ATK="+VDlg ("ATK")+" & SLP=0002;00001")
```

SLP=0002;00001 → Semicolon is an OR conjunction

```
rc=scrSetData ("UpdateGrid", "DLG=@ACTIVE | SHOW=0")
```

Here, the grid of a dynamic dialog can be hidden (SHOW=1 to show)

(as of AIP version 8.2.0.35)

AddListFileLine

```
rc=scrSetData ("AddListFileLine", "FILE=mat.lst | CNR=123 | ZLO=....")
```

Entry of a new row in list file

DelayedDialogFunction

```
rc=scrSetData("DelayedDialogFunction", "DLG=@ACTIVE | FLD.COL=CNR, clRed")
```

Color field with delay.

Advantage: If the field is directly colored, it could be overwritten during event processing.

```
rc=scrSetData("DelayedDialogFunction", "DLG=@ACTIVE | FLD.VAL=CNR, 100")
```

Set field value with delay.

```
rc=scrSetData("DelayedDialogFunction", "DLG=@FIL=DLG=CA_WL_MPL | DLG.FOCUS.SED.FLD=ATTR:10")
```

Focus field with delay.

```
rc=scrSetData("DelayedDialogFunction", "DLG=@ACTIVE | SCFKT=DNC_REOPEN_GRID")
```

Delayed call of a script function (scrSetData("ExecFunction",...))

LocalUpdate

```
rc=scrSetData("LocalUpdate", "TYP=ANR, MNR | DLG=A_TR | MNR=4711 | ANR=TK1111110010 | EGR:GUT=5 | EGR:AUS=1")
```

The tables (ANR, MNR) specified under TYPE are locally updated using the event transferred.

TriggerLoopStop

```
rc=scrSetData("TriggerLoopStop", "MODE=ONETIME")
```

Execute UserExitMainInitLoopStop once

DisableReload

```
rc=scrSetData("DisableReload", "PNR, ANR")
```

Deactivate cyclic loading of lists

→MNR, ANR, PNR, MSTAT, HZTYP, AGRD, LPKZ, BPOS, NCOM, LICENSE, ZLO, TPE, PATHS, DNC_FAM, IOP_RQ, AART, MAT, RES, TNRDATA

DialogStartTime

```
rc=scrSetData("DialogStartTime","Elapsed")
```

prevents message 'Dialog is open for more than 5 min'

ActivateSetupFunction

```
rc=scrSetData("ActivateSetupFunction","DisableAllOperationFilters")
```

Deactivate filter of order list in the main view (all OPs of all machines are then displayed that are included in the list anr.lst) (functionality is only available in old AIP GUI).

DeleteLine

```
rc=scrSetData("DeleteLine","File=List.lst|Line=5")
```

Deletes a row in a list. Only the file operation is performed. The GUI is not updated.

ResetBatch

```
rc=scrSetData("ResetBatch","CNR=PR..")
```

Resetting of the last batch number that has been generated automatically using the function SYS_NEW_CNR_FR

SetMaxParallelOrders

```
rc=scrSetData("SetMaxParallelOrders","20")
```

Increasing the maximum number of OPs that are permitted to be run at the same time at a machine with parallel make-to-order production (OPs with different partitioning).

If you use this function, errors might occur because of data strings that are too long!!

AddListFileColumn

```
rc=scrSetData("AddListFileColumn","FILE=vlist.lst|AKRO=SEL|VALUE=")
```

Add column to a list file

ExecFunction

```
rc=scrSetData("ExecFunction","DLG=@ACTIVE|FKT=REFRESH")
```

Call a script function of a dialog (in DynDlgFunctions_.. the function should be implemented)

UpdateTextView

```
rc=scrSetData("UpdateTextView","DLG=@ACTIVE|AKRO=LOC:NOTE|ACTION=CLEAR")
```

Access to a TextView of a dynamic dialog via its ID. Possible calls: ACTION=REOPEN/ RELOAD/ CLEAR

SetFieldVisible

```
rc=scrSetData("SetFieldVisible","DLG=@ACTIVE|AKRO=EGI:GUT|FKT=HIDE")
```

Hide, show, enable, etc. field of a dialog.

Values for FKT:

SHOW → Field is visible

HIDE → Field is not visible

TOGGLE → Toggle visible<->invisible

ENABLE → Field allows input

DISABLE → Field becomes ReadOnly

DeleteListFileLine

```
rc=scrSetData("DeleteListFileLine","FILE=mat.lst|FILTER=ATK=1234")
```

Deletes all rows in a LST file that match the filter criterion.

Return values: 0:OK / -1:file not found /

-2:no data rows available / -3:no data matching the filter

DelayedDlgSelectLine

```
rc=scrSetData("DelayedDlgSelectLine", "DLG=@ACTIVE | AKRO=ATK | VALUE="+sATK)
```

Selects the first row in the dialog grid that matches the filter criterion. The function is performed with the next timer run, also if the current script function has been completed.

```
rc=scrSetData("DelayedDlgSelectLine", "DLG=@ACTIVE | AKRO=EINTNR | VALUE=00044 | SWITCH_ALWAYS=TRUE")
```

SWITCH_ALWAYS=TRUE → if the row that is already active is found during selection, then the active column is at least changed to trigger a CellChange event.

NOTFOUND=SELECTFIRST / NOTFOUND=SELECTLAST

Selects the first or the last row if the filter has no result. If the first or the last row of the grid is generally selected, then the function is faster if AKRO is empty.

DeleteGridLine

```
rc=scrSetData("DeleteGridLine", "DLG=@ACTIVE")
```

delete current row in the dialog grid

ReopenMainGrid

```
rc=scrSetData("ReopenMainGrid", "MNR, ANR, LIST3")
```

Locally reread the specified grids in the main view (no reload from server).

This function is only required in mode XML-GUI=OFF

ProcessMessage

```
rc=scrSetData("ProcessMessage", "INIT=300 | TEXT=Ausgangsloswechsel")
```

Display of SplashScreen that is also shown when the terminal program is booted or when lists are reloaded.

INIT=xxx: opening the window with the specified height

```
rc=scrSetData("ProcessMessage", "TEXT=-----")
```

TEXT=...: adding a text row (the rows are added at the bottom and disappear at the top edge of the window)

```
rc=scrSetData ("ProcessMessage", "INIT=END")
```

INIT=END: closes dialog.

SetGridAutofilter

```
rc=scrSetData ("SetGridAutofilter", "DLG=@ACTIVE|CAPTION=Strukturfilter|  
FONT=ARIAL|SIZE=8|FOCUS=1")
```

Configuration of the auto filter field in the dynamic dialog

CAPTION: Alternative text for "Filter"

FONT/SIZE: setting for label and edit field

FOCUS=1: set focus of dialog on the auto filter field in the grid

```
rc=scrSetData ("SetGridAutofilter", "DLG=@ACTIVE|TEXT=Artikel-Filter")
```

The text of the auto filter field of a grid in the dynamic dialog can be overwritten.

```
rc=scrSetData ("SetGridAutofilter", "DLG=@ACTIVE|ACRO=AUNR")
```

The acronym (filter field) of the auto filter field of a grid in the dynamic dialog is reset

SetMessageDelayTime

```
rc=scrSetData ("SetMessageDelayTime", "TIME=1")
```

On the AIP, the default display time of a message in the status row (top right) is 10 sec. Use this command to change the time. The previously valid default time is returned. It is best to use this time for reset directly after the message.

Example:

```
Function StatusBarTimedMsg (sMsg,sTyp,sTime)
  Dim rc,sDelay
  sDelay=scrSetData ("SetMessageDelayTime", "TIME="+sTime)
  rc=scrStatusBarMsg (sMsg,sTyp,"1")
  sDelay=scrSetData ("SetMessageDelayTime", "TIME="+sDelay)
  StatusBarTimedMsg=sDelay
End Function
```

ActivateMainButton

```
rc=scrSetData("ActivateMainButton", "PANEL=MNR|BTN=VNR_AN,VNR_AB|ACTIVE=-1")
```

Activate/deactivate keys in the main view of the AIP

(only in mode XML-GUI=OFF)

Values for ACTIVE:

Value	Meaning
-1	disable
1	enable
-2	hide
2	show

SetFocusToGrid

```
rc=scrSetData("SetFocusToGrid", "DLG=@ACTIVE|FOCUS=FILTER")
```

Set focus of dialog on the grid

FOCUS=FILTER → Filter field

FOCUS=GRID → Table area of the grid

1.3.2.104 scrExecute(...)

WinExec

```
rc=scrExecute("WinExec", "SW_SHOWNORMAL|""c:\Programme\TextPad 4\TextPad.exe"" "" "+DIR_SPOOL+"druck.000""")
```

Start an external application program

Show parameter: <https://docs.microsoft.com/de-de/windows/win32/api/winuser/nf-winuser-showwindow>

WriteBufferToFile

```
rc=scrExecute("WriteBufferToFile", sPrnFileName+"|"+asDat)
```

Writing an ansi string in a file

CloseDynamicForm

```
rc=scrExecute("CloseDynamicForm", "DLG=TA_CAB_SOND|MNR=ENTGRAT5|CloseActive=1")
```

Close a dynamic dialog

RequestReload

```
rc=scrExecute("RequestReload", "MNR, ANR, PNR")
```

Request reloading of lists.

The list is only reloaded during the next run of the timer. In the script, the result cannot be waited for.

Possible parameters:

MNR, ANR, PNR, MSTAT, HZTYP, AGRD, LPKZ, BPOS, NCOM, PAINT, DLOSE, PPARAM, LOKVLIST, YSR, LICENSE, ZLO, TPE, CAQ_SEND, CAQ_RECV, QMS_TIMER, PROC_INT, PATHS, DNC_FAM, IOP_RQ, AART, SKAL, MAT, RES

ResetRequestReload

```
rc=scrExecute("ResetRequestReload", "MNR, ANR, PNR")
```

Reset reload request of lists

RunWithAttachedPrg

```
rc=scrExecute("RunWithAttachedPrg", "FILE=c:\mpdv\aip2\spool\Infos.doc")
```

The file transferred is started using the application that is linked to the file extension in Windows.

The complete path of the target file must be specified. Also network paths are allowed. Internet links do not work.

RunWithAttachedPrg2

```
rc=scrExecute("RunWithAttachedPrg2", "FILE=c:\mpdv\aip2\spool\Infos.doc|OPERATION=open")
```

Alternative function that also works on the terminal server. All internet links are here possible.

In the case of printable files, "OPERATION=print" triggers immediate printing on the default printer.

(uses ShellExecute)

ShowVirtKeys

```
rc=scrExecute("ShowVirtKeys", "DLG=@ACTIVE|VISIBLE=0")
```

Hide/show virtual keyboard

CheckQueue

```
rc=scrExecute("CheckQueue", "")
```

This function tries to empty the queue. In the offline case, the offline timeout is not waited for. The terminal tries to send all records one after the other. If the function is successful, the function returns the value "0". If records remain in the queue, the return value matches the number of records with a minus sign put in front.

ChangeExtension

```
rc=scrExecute("ChangeExtension", "FILE=C:\data\file.ctw|NEW=.dat|ChangeFile=1|DeleteExisting=1")
```

Change file extension (e.g. from **file.ctw** to **file.dat**)

ChangeFile=1: file is changed (otherwise only the changed name is returned)

DeleteExisting=1: if the target file already exists, this file is replaced.

DeleteIniKey

```
rc=scrExecute("DeleteIniKey", "FILE="+DIR_APP+"hcc_data.ini|SECTION=xxx  
xxx|IDENT=xxxxx")
```

Deleting a key in an INI file

GetQuotedDDItem

```
xV=scrExecute("GetQuotedDDItem", "<ID>+" | "+"<VALUES>")
```

Identifies an item from a dialog string with masked DD items.

Example: VALUES = „<MNR=m1|PARAM=MNR=pM1\|ANR=pA1|ANR=a1>“

Call <ANR|...> == „a1“

Call <PARAM|..> == „MNR=pM1|ANR=pA1|“

MakeQuotedDDItem

```
xV=scrExecute ("MakeQuotedDDItem", <xID>+" | "+<xVALUE>)
```

Creates a masked dialog string item

Example:

```
scrExecute ("MakeQuotedDDItem", "DATA|DLG=A_AN|MNR=M100|ANR=123450010|KNR=9999")
```

➔ DATA=DLG=A_AN\|MNR=M100\|ANR=123450010\|KNR=9999\|

1.3.2.105 scrDeleteItems(asData,asAkros:string):string

(UE) + (DLG) scrDeleteItems (asData,asAkros:string):string

Deleting acronyms from a dialog string

Example: (UserExitDynDlgBeforeSend)

```
Dim sDlg
sDlg=VDlg (" #GET#ALL#VALUES#")
DLGSND="#DELETE#ALL#VALUES#"
sDlg=scrDeleteItems (sDlg,"EGT:GUT|EGT:AUS|EGT:GES")
DLGSND=sDlg
```

1.3.2.106 vbsTranslateDataValues(Items , Values)

(UE) + (DLG) vbsTranslateDataValues(Items , Values)

Implementing in aip_mpdv-system.scr

Translates the values of the passed <items> into a dialog data string <values>

Examples:

```
s=vbsTranslateDataValues ("MSTTXT", "..|MSTTXT=#MST1|..")
-> "..|MSTTXT=PRODUKTION|.."
```

```
s=vbsTranslateDataValues ("S1|S2", "..|S1=#S1|S2=#S2|..")
-> "..|S1=Spalte1|S2=Spalte2|.."
```

1.3.2.107 scrComportDataWrite(string):string

(UE) + (DLG) scrComportDataWrite(string):string

With <HYREADER.DLL>, connection of external reader to write data to external reader with the ID <DATA2WRITE>. Relevant IDs to identify external readers are <COM> and <TYP>

- <COM> is preferred -> Data is only written using the specified COMPORT

- if only <TYP> is specified, the data is transferred to all instances of the <TYP> to write.

Example:

```
rc=scrComportDataWrite("TYP=DRV_CX_CVERIFY|COM=4|VERIFY=ON|CQ.MINPASS=3|")
```

1.3.2.108 scrComportEventResult(string):string

(UE) + (DLG) scrComportEventResult(string):string

With <HYREADER.DLL>, connection of external reader to write an event result to an external reader with the ID <RET> and <RET.TXT> including a description in text form. Relevant IDs to identify external readers are <COM> and <TYP>

- <COM> is preferred -> Data is only written using the specified COMPORT

- if only <TYP> is specified, the data is transferred to all instances of the <TYP> to write.

1.3.2.109 scrGWCUpdateResult(string):string

(UE) + (DLG) scrGWCUpdateResult(string):string

Only available for **UserExitOnGatewayData**

If the DD value <FT_ERROR> is set, the respective result is set for the calling external program.

See section (Using < scrGWCUpdateResult > / < UserExitOnGatewayData >)

1.3.2.110 scrForceDirectories(DIR_SPOOL+"prnlay\")

(UE) + (DLG) scrForceDirectories(DIR_SPOOL+"prnlay\")

Create a directory structure

1.3.2.111 scrLizenz(lizenz:String):boolean

(UE) + (DLG) scrLizenz("AIP-MF")

Function to test a license

Return values: true / if license is active
 false / if license is not active

Example:

```
If scrLizenz("MPL-SNR") Then
    ...
Else
    ...
End If
```

1.3.2.112 Notes on the script functions

Using < scrGWCUpdateResult(..) / UserExitOnGatewayData

You may only execute this application callback function in the user exit **UserExitOnGatewayData**. You use this function with customer-specific gateway events. The function passes the result of events with the ID **FT_ERROR** and sometimes also additional information on the error with the ID **FT_ERROR_TXT** to the calling external program.

The standard processing of a gateway event is that the event is sent to the server with the addition <..|EVCOM=J|..>.

Possible error codes **FT_ERROR** with the default error description **FT_ERROR_TXT**:

FT_ERROR String identifier	Integer value	Default description of FT_ERROR_TXT
fteOK	0	OK
fteTnrTmOt_NO_PLAUSI	1	NO_PLAUSI (TimeOut: TNR <-> DB)
fteDB_PLAUSERERROR	2	PLAUSERERROR (DB)
fteTNR_OFFLINE	3	OFFLINE (TNR-QUEUE)
fteTNR_Busy	4	TNR_BUSY (Process runs)
fteTIMEOUT_GW_TNR	5	TIMEOUT (GateWay <=> TNR)
fteGW_TNR_notINIT	7	CFG: GateWay -> TNR not init
fteTNRnotREADY	8	TNR not Ready for Process
fteTNR_GW_notCONFIG	9	CFG: TNR -> GateWay not init
fteDEFAULT_FT_ERROR	50	DEFAULT FT ERROR (is set if value of <FT_ERROR> cannot be identified)
fteDLG_UNDEF	90	unknown dialog
fteNO_DATA_TO_SND	99	No data sent available
fteNO_VALID_DATA_FORMAT	100	Invalid data format
fteCLIENTSOCKET_GETDATA_EXCEPTION	101	FCT_EXCEPT (exception in processing function)
fteCLIENTSOCKET_GETDATA_UNDEF	102	FCT_UNDEF (undefined processing function)
fteTNR_MNR_NOT_CFG	900	MDE->MNR not config
fteWNR_NOT_CFG	901	ANR->WNR not runs
fteMNR_MST_NOT_CFG	902	MNR->MST not config

fteANR_MNR_NOT_RUNS	903	MNR->ANR not runs
fteAUSGRD_NOT_CFG	904	EGG:AUS not config or empty
fteBARCODE_LEN_ERROR	905	Undefined Barcode length [valid: 0,13,16]
fteMNR_MST_NOT_POSSIBLE_PSP_ACTIVE	906	MNR->MST not possible - PSPerre active
ftePLOCK_COUNT_OFF	907	P-Lock active without counting
fteMNR_KEINE_SCHICHT	950	Machine "no shift"
#DONE#	---	After calling the callback function scrGWCUpdateResult(..) in user exit UserExitOnGatewayData , you must use the instruction UE_RET = Item("FT_ERROR", "#DONE#") to set FT_ERROR to the value #DONE# so that the standard processing is not run. The notify events are still sent to the modules –DLL's (caq72.dll, pzezks72.dll).

If **FT_ERROR** has been set using the string identifier or the respective integer value in the user exit **UserExitOnGatewayData**, the standard processing is not run any more (exception: the notification events of the modules).

(1) Example → setting negative <FT_ERROR> via application (not with application callback scrGWCUpdateResult())

>>> Send string:

```
<DLG=EV_MST|MELDZEI=43200|MELDDAT=03/05/2018|BEARB=KFS|MNR=M000002|MST=1|CLI.SN
D.T=10:51:35.746|>
```

<<<< Receive string:

```
<DT:TNR=2,7970000170|FT_ERROR=906|FT_ERROR_TXT=MNR->MST not possible - PSPerre active (
.An                                     MDE->MNR                                     >M000002<
production lock is active. Maschine status change is not permitted / UserExitOnGatewayData )
[21]|COM.ID=4@|DLG=EV_MST|
MELDZEI=43200|MELDDAT=03/05/2018|BEARB=KFS|MNR=M000002|MST=1|CLI.SND.T=10:5
1:35.746|TNR=17|DT:CLI=3,0000000726|>
```

(2) Example → Set positiver FT_ERROR=0 via application callback scrGWCUpdateResult())

>>> Send string:

```
<DLG=EV_MST|MELDZEI=43200|MELDDAT=03/05/2018|BEARB=EVCOM|MNR=M000002|MS
T=1|CLI.SND.T=12:07:45.011|>
```

<<< Receive string:

```
<DT:TNR=0,2499999013|FT_ERROR=0|FT_ERROR_TXT=OK ( .EV_MST verarbeitet / CallBack
/ scrGWCUpdateResult ) [0]|COM.ID=5@|
DLG=EV_MST|MELDZEI=43200|MELDDAT=03/05/2018|BEARB=EVCOM|MNR=M000002|MST
=1|CLI.SND.T=12:07:45.011|FT_MODE=WAIT;2; 150|TNR=17|DT:CLI=0,3590002656|>
```

Other control identifier and variable that is relevant for the gateway processing: **FT_MODE:**

Empty „“	is equal to <NORMAL> → after setting of result, there is not break in the processing. → The ClientServerThread sends the result, but the GateWay event processing in the terminal is not completed until the Windows queue of the application has been run through. → For example, if after the call < scrGWCUpdateResult > a server communication is performed, the terminal cannot receive a new GateWay command until this action is completed.
SLEEP;2;150	→ after setting the result, the application is stopped 2 times for 150 msec. (with 2 MessageBeep) → Default for the number <2> is 1. The default for the duration <150> is 200 MSec. → For notes on the processing, see < Empty „“ >
WAIT;2;150	→ after setting the result, the application is stopped 2 times for 150 msec. (with 2 MessageBeep and processing of the Windows queue using <i>ProcessMessages</i>) → The ClientServerThread sends the result and the GateWay processing in the terminal is completed. The terminal is therefore available for a new GateWay event.

General notes:

- Only one gateway event can be processed. (display in application)

Verarbeitungsfehler [10] : Eine Anfrage läuft bereits !

<<< Result in calling program **FT_ERROR=4|FT_ERROR_TXT=TNR_BUSY (Process runs)**
[Client Event just runs] [4]....

1.3.3 Working with numbers

The following must be observed when working with numbers in a terminal script:

There are different VB script engines of the Internet Explorer (that we use). When comparing <, >, ... , some script engines identify whether a variable is integer/floats, which variables are to be compared, and then convert them automatically.

Some versions do not identify this, and may compare the variable contents as strings. For this reason, it is always recommended to make an explicit type cast in case of comparisons.

Using conversion functions:

from / to	Int	String	Real
Int	-	CStr(i)	-
String	vbsIntDef(s,"0")	-	scrStr2Real(s)
Real	Rounded: CInt(r) Truncated: Fix(r)	scrReal2Str(r)	-

1.3.4 Using "IF" queries

The terminal scripts execute all comparisons with "IF" queries including "AND" conditions.

For example, this leads to a runtime error in the following instruction if the variable sInt is an empty string or is not convertible:

Example:

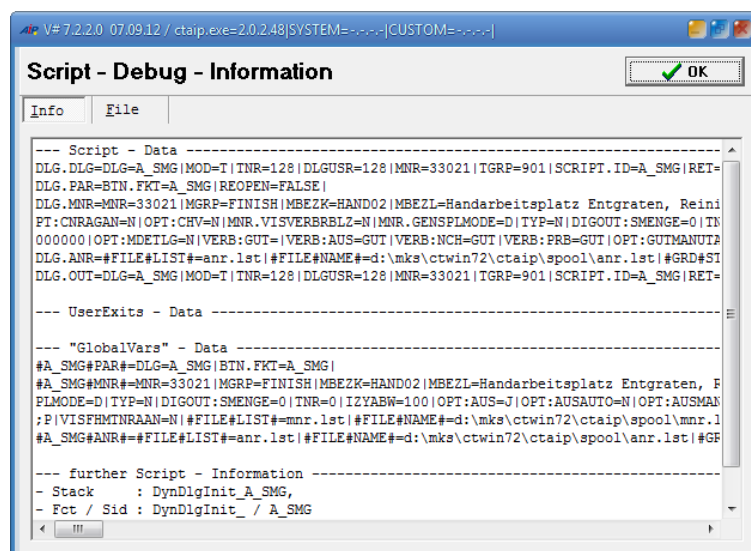
```
Sub ...
  Dim sInt
  ...
  sInt = VDlg("FU:32")      ' *** identify string
  If IsNumeric( sInt ) And Int( sInt ) = 4 Then
    ...
  Else
    ...
  End If

' *** possible procedure without run time error
If IsNumeric(sInt) Then
  If Int(sInt) = 4 Then
    ...
  End If
Else
  ...
End If
...
```

1.3.5 Debugging

1.3.5.1 Script - Debug – Dialog

Using the function **SYS_SCRIPT_DEBUG**, the script debug information is displayed as follows at runtime in an additional dialog.



Tab **"Info"** includes the following information.

- „**Script - Data**“: Data/variables of the current/last DIALOG
- „**UserExits - Data**“: Data/variables of the current/last user exit
- „**GlobalVars - Data**“: Global variables of the application
- „**further Script - Information**“: Information on the current script status.
 - Script call stack, i.e. name of the function that is executed
 - Script function + Dialog ID
 - Counter for active dialog and user exit functions

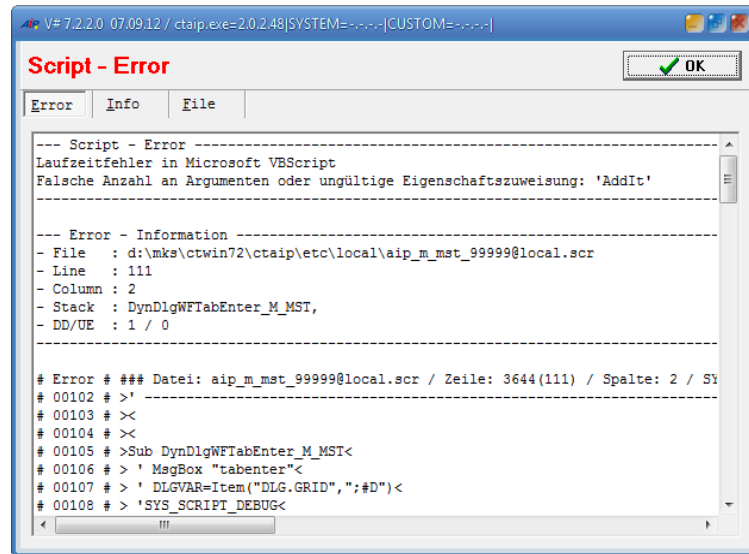
Tab **"File"** includes the following information:

- „**Script - File - Infos**“: Information in tabular form on the scripts currently loaded
- „**Script - File - Overview**“: Short information on the scripts loaded + zip files.
- „**Script - Methods - Overview**“: Information on the functions available in the system + dialog – script files of the scripts currently loaded.

1.3.5.2 Exception - Script – Dialog

In addition to the information described in section **"Script - Debug – Dialog"**, this dialog also includes the tab **"Error"** that displays the following data:

- **Script-Error**: Description of the error occurred
- Error information: extended information on the error:
 - Script file (with error)
 - Row in script file (with error)
 - Column in script file (with error)
 - Script call stack, i.e. name of the function that is executed
 - Counter for active dialog and user exit functions
- A script excerpt where the error is marked.
- The script file that includes the error.



1.4 USEREXIT in the system script

Die Implementierung von kundenspezifischen Userexits wird in einer kundenspezifischen „System“-Script-Datei (aip_system_<KD>.scr) durchgeführt. Im kundenspezifischen Script können Script-Funktionen aus dem Standard System-Script (aip-mpdv-system.scr) verwendet werden.

For information on the storage and naming of the system script, refer to the sections "Storage structure of scripts" and "Naming conventions".

1.4.1 UserExitInitLosnummer

Functionality:

You use this user exit to change the length of the standard batch number locally on the AIP2.

Implementation notes:

If the value **LEN:CNR** is not set, the batch number length specified in the basic settings is used and not changed. Using the user exit, the default batch number length is changed for production batches (e.g. PRxxxxxxx), goods receipt batches (e.g. WExxxxxxx) and handling units (HUxxxxxxx).

Note: The batch number length in the basic settings should always be set to a value greater than the length used in the user exit so that the input fields on the client are displayed with the appropriate length.

Example: the batch number length is fixed and set to 10 digits.

```
Sub UserExitInitLosnummer
  UE_RET = Item("LEN:CNR", "10")
End Sub
```

1.4.2 UserExitLosnummer

Functionality:

This user exit implements a customer-specific generation of batch numbers.

Implementation notes:

Observe the handling of dynamic dialogs and the abort functionality (e.g. in case of an output batch change).

If a dialog function is canceled, an UNDO is usually performed for the number range. The currently assigned batch number is reset to its original value. This can only work if the dialog is closed and then sent, and if send is not performed in the dialog script itself (e.g. using OK). To avoid the problem, the mode UNDO can be used in this user exit (see example below).

<u>Available functions</u>	<u>Description</u>
VTnr ("XYZ")	Info from list TKENN.LST
VVar ("UE:PAR", "XYZ")	Transfer parameters
VVar ("UE:MNR", "XYZ")	Info from list MNR.LST for the current machine
VVar ("UE:ANR", "XYZ")	Info from list ANR.LST for the current order
rsIni (ini, Sektion, Key, Default)	Read INI file (with automatic writing of <default> if entry is not available)
wsIni (ini, Sektion, Key, Value)	Write INI file

Input parameters:

Parameter	Value	Description
MODE	GENERATE	Generate batch number
CNR.FIX.TYP	cmWarenEingang	Create goods receipt batch
CNR.FIX.TYP	cmFertigung	Create production batch
MODE	UNDO	Perform Undo. With RET=0, the standard undo function is skipped.

Return parameters:

Parameter	Value	Description
UE_RET	CNR	Generated batch number (e.g. CNR=NNNNNNNNNN)

Example: generation of a customer-specific batch number for production batches. The elements of the number are read from an INI file.

```

Sub UserExitLosnummer
  Select Case VVar("UE:PAR", "MODE")
    Case "GENERATE"
      '-----
      '-- Modi = "sCnr.FIX.TYP" = "cmWarenEingang", "cmFertigung"
      '-- if no "sCNR=NNNNNNNNNN" in <UE_RET> is defined
      '-- the standard batch number generation is used
      '-----
      OnGenerate
    Case "UNDO"
      '-----
      '      '-- if a "RET=0" in <UE_RET> is set
      '-- the standard batch number undo function is skipped
      '-----
      UE_RET = Item("RET", "0")
  End Select
End Sub

Sub OnGenerate
  Dim sLfd, sCnr
  If VVar("UE:PAR", "sCnr.FIX.TYP") = "cmWarenEingang" Then
    UE_RET = Item("RET", "DEFAULT")
  Else
    sLfd = IncStrDec( sLfd )
    sLfd = rsIni( "u_losnr.ini", "Losnummer", VTnr("TNR")+"->sLfd", "0" )
    sLfd = wsIni( "u_losnr.ini", "Losnummer", VTnr("TNR")+"->sLfd", sLfd )
    sLfd = wsIni( "u_losnr.ini", "Losnummer", "sCnr->UNDO->TNR->sLfd", sLfd )
    sCnr = "2"+VTnr("TNR") + StrFmtRight( sLfd, 5, "0" )
    sCnr = wsIni( "u_losnr.ini", "Losnummer", "sCnr->UNDO->TNR->sCnr", sCnr )
    UE_RET = Item("sCnr", sCnr )
    UE_RET = Item("RET", "0" )
  End If
End Sub

```

1.4.3 UserExitMainInitLoopStop

Functionality:

This user exit is run when the main application is started after initialization (before the terminal switches to the Run - Mode Mainloop), and if necessary in the MainLoop (see example for explanation) and when the terminal is closed.

Input parameters:

Parameter	Value	Description
UE:PAR	Input parameters	Input parameter (VVar ("UE:PAR", "XXX"))
MODE	INIT	Request after terminal restart
MODE	LOOP	Cyclic request if < LOOPTIME=X > if X > 0 has been set.
MODE	STOP	Is called when the terminal program is closed (manually or remote via terminal status)
LOOPTIME=0		Current LOOPTIME (Default = 0)
MINSTEP=5		Minimum cycle (default=5)
ONETIME=FALSE		Unique <LOOP> call if in debug screen "Reload-Status" (Ctrl+Alt+T) UserExitMainInitLoopStop has been activated.

Return parameters:

Parameter	Value	Description
UE_RET	MODE=INIT, LOOP	LOOPTIME=<> Call LOOP after 10 seconds

Example: set loop timer to x seconds

```

Sub UserExitMainInitLoopStop
  Select Case VVar("UE:PAR", "MODE")
    Case "INIT"
      scrLog(" UserExitMainInitLoopStop = PAR ( "+VVar("UE:PAR", "#GET#ALL#VALUES#")+ " )")
      ' --- after program start -> call LOOP after 5 seconds
      UE_RET = Item("LOOPTIME", "5")
      ' --- NOTE: if <LOOPTIME> is not set in "INIT"
      scrLog(" UserExitMainInitLoopStop = RET ( "+VVar("UE:RET", "#GET#ALL#VALUES#")+ " )")
    Case "LOOP"
      ' scrLog(" UserExitMainInitLoopStop = RET ( "+VVar("UE:RET", "#GET#ALL#VALUES#")+ " )")
      ' --- then -> call LOOP after 10 seconds
      UE_RET = Item("LOOPTIME", "10")
    Case "STOP"
      scrLog(" UserExitMainInitLoopStop = PAR ( "+VVar("UE:PAR", "#GET#ALL#VALUES#")+ " )")
  End Select
End Sub

```

Note:

The main timer of the terminal program waits in "LOOP" mode until the UserExitMainInitLoopStop is processed. If a longer action is started from here or a message is displayed, the clock stops at the bottom right of the terminal. This also means that the terminal background processes are not processed. User actions should therefore be processed via "DelayedButtonClick", as the processing of this function does not stop the main timer.

1.4.4 UserExitButtonClick

Functionality:

This user exit is used to check the plausibility of a button from the button bar (lower part of the screen, possibly configured via ctaipbut.ini) or the identifier of an OnClick event in the XML interface.

The current machine and order rows are transferred to the user exit.

Implementation notes:

VVar("UE:MNR", "<ID>")	Access to all data in the machine list mnr.lst
VVar("UE:ANR", "<ID>")	Access to all data in the order list anr.lst
UE_RET=Item("BTN.FKT", "#FKT#->#EXIT#")	Terminates functions (no default processing in the terminal programm)
UE_RET=Item("BTN.FKT", "A_UN")	Overwrites the original function.
UE_RET=Item("ANR+MNR", "RELOADED")	Before starting a script dialog, order and machine data are re-read.
UE_RET=Item("BTN.FKT", "SCRIPT->A_UN")	Instead of the standard implementation in the terminal program a script (here aip_mpdv-A_UN.scr) is used.
UE_RET=Item("BTN.RET", "1")	Error message "Function not implemented".
@<function>	A prefixed @ means that a dynamic dialog is not sought. Instead implementation is performed in the terminal script.
@@<function>	@@ at the beginning of the function does not check whether the function was processed in the UserExitButtonClick. Therefore, a return value must not be set → Item("BTN.FKT", "#FKT#->#EXIT#")

Example:

```

Sub UserExitButtonClick
  Select Case VVar("UE:PAR", "BTN.FKT")
    Case "A_UN"
      If VVar("UE:ANR", "AGNR") = "0051" Then
        scrMsgBox( " (A_UN) bei AGNR = 0051 über Script !\n Script: [ A_UN ] ausführen." )
        UE_RET = Item("BTN.FKT", "SCRIPT->A_UN")
        UE_RET = Item("ANR+MNR", "RELOADED")
      End If
    Case "A_AB"
      If VVar("UE:ANR", "AGNR") = "0052" Then
        scrMsgBox( " (A_AB) with AGNR = 0052 not possible !\n Function: [ A_UN ] execute." )
        UE_RET = Item("BTN.FKT", "A_UN")
      End If
    Case "A_TR"
      If VVar("UE:ANR", "AGNR") = "0053" Then
        scrMsgBox( " (A_TR) with AGNR = 0053 not possible !\n Function is canceled." )
        UE_RET = Item("BTN.FKT", "#FKT#->#EXIT#")
      End If
    Case "@@TEST"
      ' do something / testing during development
  End Select
End Sub

```

1.4.5 UserExitDynDlgBeforeInitialize

Functionality:

This user exit is called before a dynamic dialog is initialized.

This user exit makes it possible to assign a terminal script for a programmed dialog to map a customer-specific extension.

Example: customizations "Machine-related preassignment of badge number" or "Recording of material consumption".

The user exit also allows you to carry out initializations for all dynamic dialogs, since it is always run through. That means there is no need to create a script for each individual dialog.

Input parameters:

Parameter	Value	Description
DLG.DLG	Dialog data	Complete dialog data for the calling dialog

Return parameters:

Parameter	Value	Description
DLG.OUT	Return data	If you set the following return value, you can use this user exit to prevent that the dialog is opened: DLGVAR=Item("RET", "#CANCEL#")

Beispiel 1: Dialogsteuerung der Standarddialoge <A_TR> , <A_UN> , <A_AB> mit dem Script <A_VERB_WZB>.

```
Sub UserExitDynDlgBeforeInitialize
  Select Case VOut("DLG")
    ' ----- Recording of material consumption in toolmaking -----
    Case "A_UN", "A_AB", "A_TR"
      DLGVAR = Item("SCRIPT.ID", "A_MENGE_WZB")
  End Select
End Sub
```

Example 2: customer-specific preassignment of the badge number.

```
Sub UserExitDynDlgBeforeInitialize
  DLGVAR = Item("KNR", GVars("SYSTEM", "KNR"))
End Sub
```

1.4.6 UserExitDynDlgBeforeSend

Functionality:

This user exit is used to complete or suppress all PDM postings that are not processed in a DIALOG script **DynDlgBeforeSend_XYZ**.

Implementation notes:

You can prevent a PDM posting from being sent by using the following line in the script.

```
DLGSND=Item("EVENT", "EVENT_DIALOG_OHNE_SENDEN")
```

Example:

```

Sub UserExitDynDlgBeforeSend
  Select Case VDlg("DLG")
    Case "U MTZ"
      DLGSND = Item("EVENT","EVENT_ONLINE_OHNE_AUTO_MENGEN")
      DLGSND = Item("MNRTNR.TNR",VDlg("TNR"))
      DLGSND = Item("MNRTNR.MNR",VDlg("MNR"))
      DLGSND = Item("MNRTNR.OPT:TMP","J")
      If VDlg("MODUS") = "Z" Then
        ' -- create new dynamic terminal machine assignment -----
        ' -- DLG=MNRTNR.INSERT|MNR=xxxx|TNR=xxx|KNR=xxxx|OPT:TMP=J -----
        DLGSND = Item("DLG", "MNRTNR.INSERT" )
      Else
        ' -- delete dynamic terminal machine assignment -----
        ' -- DLG=MNRTNR.DELETE|MNR=xxxx|TNR=xxx|KNR=xxxx -----
        DLGSND = Item("DLG", "MNRTNR.DELETE" )
      End If
    Case "U XYZ"
      ' *** Implementation of further customer-specific actions
  End Select
End Sub

```

1.4.7 UserExitDynDlgAfterSend

Functionality:

You use this user exit to execute customer-specific requirements after a successful PDM posting (see also section "DynDlgAfterSend_XYZ").

Example: list update (in main view) after a successful PDM posting.

Input parameters:

Parameter	Value	Description
UE:SND		Complete send string in PDM format
UE:RCV		Return value of the PDM command sent

Return parameters:

Parameter	Value	Description
DD_RCV		Extend return value e.g. reload lists

```

Sub UserExitDynDlgAfterSend
  Select Case VSnd("DLG")
    Case "U_MTZ"
      ' ----- Ex. "Dynamic machine terminal assignment -----
      DD_RCV = Item( "LOAD", "MNR,MST,ANR,PNR,"+ VRcv("LOAD") )
    Case "U_XYZ"
      ' *** Implementation of customer-specific actions
  End Select
End Sub

```

1.4.8 UserExitAfterSendError

Functionality:

This user exit is called if the server refuses a posting. The call is made before the error message is displayed on the terminal.

Input parameters:

Parameter	Value	Description
UE:PAR	VVar("UE:PAR", "XYZ")	Send string that the terminal has sent to the server
UE:RET	VVar("UE:RET", "<ID>")	Return string of the server (error number, error text...) The error number is requested with VVar("UE:RET", "RCV.RET")

Return parameters:

Parameter	Value	Description
UE_RET	VIEWERROR=FALSE	Can be used to stop display of error message
	#REPEAT_SND#=J	Resend dialog message

Implementation notes:

Here, you can store data in global variables (→GLOBALVARS) that are used when the dialog is reopened.

Example: catch error code from server and open dialog

```

Sub UserExitAfterSendError
  scrLog("UserExitAfterSendError|" + VVar("UE:PAR", "#GET#ALL#VALUES#"))
  Select Case VVar("UE:PAR", "DLG")
    Case "CA_WL_PA"
      AfterSendError_CA_WL_PA
  End Select
End Sub

Sub AfterSendError_CA_WL_PA
  Dim sRET, sVal
  'sVal=VVar("UE:PAR", "*JA_NEIN_CHECK")
  'sRET=VVar("UE:RET", "RCV.RET")
  If VVar("UE:RET", "U_RET")="7013" Then
    ' Can be used to stop error message
    UE_RET=Item("VIEWERROR", "FALSE")
  End If
End Sub

```

Example 2: using the data when a dialog is reopened:

```

Sub DynDlgInit U_CA_WL
  If VOut("REOPEN")="J" Then
    s=GVars("#RCV#AFTER#SEND#ERROR#", "RCV.RET")
    ' ...
  End If
End Sub

```

Example 3: remove a field and immediately re-send data

```

Sub UserExitAfterSendError
  Select Case VVar("UE:PAR", "DLG")
    Case "A_P_AN", "A_AN"
      AfterSendError_A_AN
  End Select
End Sub

Sub AfterSendError_A_AN
  Dim sRet, sData, sMATCHCHECK, sDialogText
  sData = VVar("UE:PAR", "#GET#ALL#VALUES#")
  sMATCHCHECK = VVar("UE:RET", "MATCHCHECK")
  If VVar("UE:RET", "RCV.RET")="424" and (sMATCHCHECK = "FALSE") Then
    sDialogText=rsCfg("DIALOG->TEXT", "MATCHCHECK_TEXT", "Nummer speichern?")
    sRet=DlgJaNein(scrTranslate("Auswahl", ""), scrTranslate(sDialogText, ""))
  End If
End Sub

```

```

If sRet = "#JA#" Then ' JA: Daten erneut senden
    ' here, the items that are to be deleted (U_MATCHCHECK,HALLO,CHECK)
    UE_RCV=Item("#DELETE_ITEM#", "U_MATCHCHECK,HALLO,CHECK")
    UE_RET=Item("VIEWERROR", "FALSE") + Item("#REPEAT_SND#", "J")
Else
    UE_RET = Item("VIEWERROR", "FALSE")
End If
End If
End Sub

```

1.4.9 UserExitLocalMnrAnrUpdate

Functionality:

You use this user exit to update the local MNR.LST + ANR.LST after a successfully performed posting (= event).

Implementation notes:

This user exit is only executed if the ID MNR or ANR exists in the send string and if these are included in the MNR/ANR list.

Available functions

VVar("UE:SND", "<ID>")

Description

You use this function to access the values of the PDM send string sent.

Example:

PDM send string: DLG=M_MST|MST=2|..|

VVar("UE:SND", "MST") returns the value "2"

scrStoreUpdate(sMode,sID,sValue)

Function to read, write, add up values in DD lists. See also section "Script functions".

Input parameters:

If required, this user exit is requested several times. This depends on the data of the dialog string sent. The different requests have different parameters UE: PAR=MODE and are performed in the order specified in the table:

Sequence	Parameter	Value	Description
1	UE: PAR=MODE	MNR->UPDATE	This user exit is only called with mode MNR->UPDATE if the requested dialog includes the ID "MNR".
2	UE: PAR=MODE	ANR->UPDATE->LAUFEND	This user exit is only called with mode ANR->UPDATE->RUNNING if the dialog requested includes the IDs "MNR" and "ANR" and if the operation has the status "running" at the machine. (AST_OPT_PKENN=L).
3	UE: PAR=MODE	ANR->UPDATE	This user exit is only called with mode ANR->UPDATE if the requested dialog includes the ID "ANR".

Example: When an order-related PDM posting has been performed, the ANP list moves to the next item.

```
Sub UserExitLocalMnrAnrUpdate
```

```

Dim sDlgID,sMode,rc
sMode = VVar("UE:PAR","MODE")
sDlgID = VVar("UE:SND","DLG.DLGCFG")
If sDlgID = "" Then sDlgID = VVar("UE:SND","DLG")
Select Case sDlgID
Case "U_CA_WL_RF"
  Select Case sMode
  Case "MNR->UPDATE"
    scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )" )
  Case "ANR->UPDATE->LAUFEND"
    scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )" )
    If VVar("UE:SND","CNR") <> "" Then
      rc = scrStoreUpdate( "UPDATE","CNR",VVar("UE:SND","CNR") )
      scrLog( " UserExitLocalMnrAnrUpdate ( CNR = "+rc+" )" )
    End If
  Case "ANR->UPDATE"
    scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )" )
    If VVar("UE:SND","KDCNR") <> "" Then
      rc = scrStoreUpdate( "ADD","AGR_FU_11","1" )
      scrLog( " UserExitLocalMnrAnrUpdate ( AGR_FU_11 "+rc+" (+1) )" )
    End If
  End Select
Case "U_A_UN_RF"
  Select Case sMode
  Case "MNR->UPDATE"
    scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )" )
  Case "ANR->UPDATE->LAUFEND"
    scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )" )
    rc = scrStoreUpdate( "UPDATE","AST_OPT_PKENN","U" )
  Case "ANR->UPDATE"
    scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )" )
    If VVar("UE:SND","KDCNR") <> "" Then
      rc = scrStoreUpdate( "ADD","AGR_FU_11","1" )
    End If
  End Select
Case "U_A_AB_RF"
  Select Case sMode
  Case "MNR->UPDATE"
    scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )" )
  Case "ANR->UPDATE->LAUFEND"
    scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )" )
    rc = scrStoreUpdate( "UPDATE","AST_OPT_PKENN","E" )
  Case "ANR->UPDATE"
    scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )" )
    If VVar("UE:SND","KDCNR") <> "" Then
      rc = scrStoreUpdate( "ADD","AGR_FU_11","1" )
    End If
  End Select
End Select
End Sub

```

1.4.10 UserExitEventFinished

Functionality:

You use this user exit to execute customer-specific requirements after a successful PDM posting (= event).

Input parameters:

Parameter	Value	Description
DD.SND	Transmit data	PDM event DLG=A_UN MNR=<..> ANR=<..> ...
DD.RCV	Receive data	PDM result RET=0 KT=<..> LT=<..> ...

Return parameters:

Parameter	Value	Description
UE_RET	---	without processing

Implementation notes:

The script processing of a DB event is structured as follows.

(e.g. for dialog „DLG=A_XYZ|MNR=M100|ANR=1A007..|DLG.DLGCFG=XYZ|...“)

To identify the dialog, you mainly use the item **DLG.DLGCFG**. If **DLG.DLGCFG** is not included, the item **DLG** is used.

The DM event identified may only include the following characters " _ " , "A" .. "Z" , "0" .. "9" !

Other characters are replaced with the character "_". (e.g. DLG=ADEPRO.ADD → „ADEPRO_ADD“)

1. *DynDlgBeforeSend_XYZ *1*
UserExitDynDlgBeforeSend → Case „XYZ“
 (if *1 does not exist or with background event)
2. *DynDlgAfterSend_XYZ *2*
UserExitDynDlgAfterSend → Case „XYZ“
 (if *2 does not exist or with background event)
3. Here, the label printing is performed
4. *UserExitLocalMnrAnrUpdate* → Case „XYZ“
 (if available and <MNR>/<ANR> included in DB event)
5. *UserExitEventFinished_XYZ* → Customer-specific implementation /
 (if available) / *aip_system_<project>.scr*
6. *UserExitEventFinished__XYZ__* → Standard processing
 (if available) / *aip_mpdv-system.scr*

This user exit has been realized for the standard MPL processing of coil cutting processes, for example. Here, you can delete the cutting plans once the coil cutting OPs have been logged off or interrupted.

Example:

```
Sub UserExitEventFinished_U_CAWL_RS
  Dim rc,sSnd,sRcv,sCALT20,rc
  scrLog("UserExitEventFinished_U_CAWL_RS")
  sSnd = VSnd("#GET#ALL#VALUES#")
  sSnd = scrReplaceDDItem("DLG","U_CA_WL",sSnd)
  ' set CALT20 (KFB) from received result in send string
  sRcv = VRcv("#GET#ALL#VALUES#")
  sCALT20 = scrDDItem("RET.CALT20",sRcv)
  sSnd = scrReplaceDDItem("CALT20",sCALT20,sSnd)
  rc = vbsUpdateMnrALosListe(sSnd,VRcv("#GET#ALL#VALUES#"))
End Sub
```

1.4.11 UserExitPccDllToTerminal

Functionality:

You use this user exit to process PCCDLL events (e.g. counter, V variables,...)

Input parameters:

Parameter	Value	Description
UE:DAT	PCCDLL-Event-Daten	Complete event data in PDM format. Events of the MDE blade are transferred with the prefix "PCC".

Return parameters:

Parameter	Value	Description
UE_RET	PCCDLL event return data	If you set the acronym #PCCDATA-MODE# with the value #NEW# , then you can change the return string.

Implementation notes:

Using the function `VVar("UE:DAT","<ID>")` you can access any field of the events.

Using the function `VVar("UE:DAT","", "#GET#ALL#VALUES#")`, the PCCDLL event is read.

The data received from the driver can be changed in this user exit before the terminal program processes the data. The following must be set for this:

```
UE_RET = Item("#PCCDATA-MODE#", "#NEW#")
```

Additional functions:

```
UE_RET = Item("#PCCDATA-MODE#", "#CLEAR#")
```

→ deletes the data

```
UE_RET = Item("#PCCDATA-MODE#", "#EXIT#")
```

→ exits the distribution function (no processing of the data in the terminal program)

Events of the MDE blade are transferred to this user exit with the prefix "PCC". The following events are transferred from the MDE blade:

Shift change:

```
PCC.TID=<>|DLG=PCC.A_ASW|MNR=<>|DAT=<>|ZEI=<>|MST=<>|AGR:HUB=..|
```

Beginning of shift:

```
PCC.TID=<>|DLG=PCC.A_AAN|MNR=<>|DAT=<>|ZEI=<>|MST=<>|AGR:HUB=..|
```

End of shift:

```
PCC.TID=<>|DLG=PCC.A_AUN|MNR=<>|DAT=<>|ZEI=<>|MST=<>|AGR:HUB=..|
```

Cyclic quantities/status update:

```
PCC.TID=<>|DLG=PCC.M_AST|MNR=<>|DAT=<>|ZEI=<>|MST=<>|AGR:HUB=..|
```

Update of counter and display:

```
PCC.TID=<>|DLG=PCC.COUNTER.UPDATE|MNR=<>|AGR.C:5=<>|AGG.C:5=<>|AGB.C:5=<>|..|
```

```
..|AGR:HUB=<>|IZY=<>|PSPERRE=<>|#3
```

```
DLG=LIST.UPDATE| #9 FILE@MNR.LST #9 FILTER@MNR=<> #9 ADD@AGR:GUTP=<>|
```

```
AGR:GUT=<>|AGR:HUB=<>|SET@IZY=<>| #9 FILE@ANR.LST #9 FILTER@MNR=<>&ANR=<>
```

```
ADD@EGS:GUT=<> #9 FILTER@ANR=<> #9 ADD@EGR:GUTP=<>| #9 |TICKCOUNT=<>|
```

Automatic machine status change

```
PCC.TID=<>|DLG=PCC.M_MST|MNR=<>|MST=<>|PSPERRE=<>|DAT=<>|ZEI=<>|DT=<>|TICKCOUNT=<>|
```

Important note: you must always change these automatic events in the MDE blade. You can guarantee a correct data collection with a correct GUI update by doing it this way.

Example 1: catch digital input I:I301 and perform customer-specific action.

```
Sub UserExitPccDllToTerminal
  scrLog("UserExitPccDllToTerminal: "+VVar("UE:DAT", "#GET#ALL#VALUES#"))
  If scrPosStr("|I:I301=1|", VVar("UE:DAT", "#GET#ALL#VALUES#")) <> "" Then
    '...
  End If
End Sub
```

Example 2: customer-specific integration of 2 balances ("Waage") using number 1/2

```
Sub UserExitPccDllToTerminal
  If VVar("UE:DAT", "V:WAAGENR") <> "" Then HandleScales
End Sub

Sub HandleScales
  Dim sWaageNr, sWaageValue
  '-----
  ' From the scales, it is always V:BRUTTO=xxxx
  ' only the scales number changes if the balance changes V:WAAGENR=1 or V:WAAGENR=2
  ' Value of scales 1 (scale 1) writes
  ' Convert value of scale 2 to the correct input field in the dialog
  '-----
  sWaageNr = VVar("UE:DAT", "V:WAAGENR")
  sWaageValue = VVar("UE:DAT", "V:BRUTTO")
  If sWaageNr="2" Then
    UE_RET = ""
    UE_RET = Item("DLG", "GETVAL")
    ' To take over changed data from the script
    UE_RET = Item("#PCCDATA-MODE#", "#NEW#")
    ' Scale value, if balance number=2 is set to dialog field of balance 2
    UE_RET = Item("V:WAAGENR", sWaageNr)
    UE_RET = Item("V:EGR:GUT", sWaageValue)
    ' so that the field of scale 1 is not also filled
  End If
End Sub
```

1.4.12 UserExitAutomaticQuantities

This user exit only exist to guarantee downward compatibility.

Note:

You cannot use this user exit to change automatic quantities because the MDE has been moved to a blade and the GUI is updated using the calculated quantities of the blade. Always change automatic quantities in the blade. Otherwise, you cannot ensure that the blade has the current/valid counter readings to monitor functions like "target quantity reached", etc.

1.4.13 UserExitExternalReaderEvent

Functionality:

You use this user exit to process the external ID/bar code readers integrated via < HYREADER.DLL >.

Implementation notes:

The following two callback functions are supported:

scrComportDataWrite(string):string

- to write data to external reader

scrComportEventResult(string):string

- to write processing result to external reader (e.g. "..|RET=1|RET.TXT=Error->Firmennr|..)



For further information on the structure of bar codes and on bar codes with prefixes, refer to the documentation of the "AIP functions shop floor/machine data".

Available functions

VVar("UE:PAR", "<ID>")

Description

Call parameter with MODE and system/company number

VVar("UE:BAR", "<ID>")

Bar code data from bar code dispatcher

VVar("UE:RET", "<ID>")

Return: Processed bar code

Input parameters:

Parameter	Value	Description
UE:PAR->MODE	CALLBACKEVENT	Call mode with ID/bar code events e.g. VVar("UE:PAR","MODE")
UE:PAR->MODE	CALLBACKSTATE	Call mode with INFO/WARNING/ERROR messages
UE:BAR	<>	Complete bar code data string e.g. request with VVar("UE:BAR", "#GET#ALL#VALUES#")
UE:BAR	RAWDATA	Original data string of reader

Return parameters:

Parameter	Value	Description
UE_RET = Item("RESULT", "-1")	RESULT=-1	Bar code is processed, do not pass to application
UE_RET = Item("RESULT", "1")	RESULT=1	(DEFAULT with special case) Bar code is processed, do not pass to application Using <HYREADER.DLL> function <ComportEventResult()>, data is written on external reader → value of <RESULT> is copied to <RET> Special case: If the ID <KNR> is included in ID/bar code event and no <IDCODE> is included. If the ID <FIR> does not match the configured <company number> (<SYSNR> from TKENN.LST), then <IDCODE> is internally set using ID/bar code event IDs <FIR>+<KNR> +<PZ>.
UE_RET = Item("RESULT", "0")	RESULT=0	(DEFAULT) Bar code is passed to application Special case:

		If the ID <KNR> is included in ID/bar code event and no <IDCODE> is included. If the ID <FIR> matches the configured <company number> (<SYSNR> from TKENN.LST), then <IDCODE> is internally set using ID/bar code event IDs <FIR>+<KNR>+<PZ>.
UE_RET=Item("SEND-AS-BARCODE","FALSE")	False	(DEFAULT) Standard transfer of the data ➔ value is transferred in a dialog into an active field, for example.
UE_RET=Item("SEND-AS-BARCODE","TRUE")	TRUE	Transfer of data as STD-BARCODE (identification of length, acronym,...)

Example:

```

Sub UserExitExternalReaderEvent
  Select Case VVar("UE:PAR","MODE")
    Case "CALLBACKEVENT"
      OnReaderCallbackEvent
    Case "CALLBACKSTATE"
      '--- Here, the messages INFO/WARNING/ERROR are processed
      ' !!! Implementation !!!
    Case Else
      UE_RET = Item("ACTION", "### "+VVar("UE:PAR","MODE")+ " ###")
  End Select
End Sub

Sub OnReaderCallbackEvent
  Dim sEvent,sData
  '--- Here, the ID/bar code events are processed
  sEvent = VVar("UE:BAR","#GET#ALL#VALUES#")
  sData = VVar("UE:BAR","RAWDATA")
  ' !!! Implementation !!! (siehe HINWEISE)
  ' *** !!! bar code processed
  UE_RET = Item("RESULT",-1) ' Barcode verarbeitet->keine weitere Verarbeitung
End Sub

```

1.4.14 UserExitBarcodeToMain

Functionality:

If a barcode is scanned while the terminal is in the basic mask, this user exit is called.

If the terminal program receives a barcode when it is in the basic mask, the barcode is interpreted as machine status. The Change Status dialog opens and the status is preset if the barcode has the appropriate format.

You can use this user exit to call another dialog instead of the Change Status dialog. This could be the dialog *Log Person*.



For further information on the structure of bar codes and on bar codes with prefixes, refer to the documentation of the "AIP functions shop floor/machine data".

Input parameters:

Parameter	Value	Description
UE:BAR	Bar code data	Raw data (bar code as scanned) For example: UE:BAR=BAR=PR3X58G112 LESERTYP=BAR COM=1 DLG=MAIN->FORM BAR.DLGID=CNR BAR.VALUE=PR3X58G112 MNR=RW10 ANR=080006830290
BAR.DLGID	Field ID	Field ID identified via bar code length (e.g. KNR)
BAR.VALUE	Value bar code	Bar code (perhaps without check digit with KNR) – Value passed to the field

Return parameters:

Parameter	Value	Description
UE_RET	Return data	UE:RET=RET=#FKT#->#EXIT# ..“ ➔ prevents the standard processing in the terminal program (processing of the barcode only by the script) ➔ using the function scrAddAction(), you can open a dynamic dialog, for example. No return value: The standard processing in the terminal program opens the dialog "Change status" (M_MST)

Example: Open a customer-specific dialog when a batch number has been scanned.

```
Sub UserExitBarcodeToMain
    Dim rc
    ' MsgBox "UserExitBarcodeToMain = "+VVar("UE:BAR", "#GET#ALL#VALUES#")
    If VVar("UE:BAR", "BAR.DLGID")="CNR" Or Len(VVar("UE:BAR", "BAR"))>= 10 Then
        rc = scrAddAction("mtaDIALOG", "DLG=U_PACK|", Item("CNR", VVar("UE:BAR", "BAR")))
        UE_RET = Item("RET", "#FKT#->#EXIT#")
    End If
End Sub
```

1.4.15 UserExitDynDlgBarcode

Functionality:

You can use this user exit to implement or manipulate a customer-specific bar code processing. The call occurs when a barcode is scanned while the dialog is open.

Das Terminalprogramm entscheidet anhand der Länge des Barcodes und anhand der im geöffneten Dialog vorhandenen Felder, zu welchem Feld der Barcode passen könnte. This only works for fields defined in the standard system. If customer-specific fields are to be scanned, a corresponding implementation is required here.

Implementation notes:

Can only be used if the scanner is connected via COM port. A scanner that is looped into the keyboard does not trigger this user exit!

If a bar code must be processed in the main application (without dynamic dialog), you must use the USEREXIT BarcodeToMain.



You only use this user exit to manipulate bar codes in dynamic dialogs.

The bar code passed to the dialog can be manipulated.



For further information on the structure of bar codes and on bar codes with prefixes, refer to the documentation of the "AIP functions shop floor/machine data".

Available functions

VVar ("UE:BAR")

Description

Transfer parameter from bar code dispatcher (interrupt handler)

VVar ("UE:RET")

Return: Processed bar code

Available parameters

VVar ("UE:BAR", "DLG")

Dialog ID

VVar ("UE:BAR", "LESERTYP")

Reader type (LESERTYP=BAR,.....)

VVar ("UE:BAR", "COM")

Comport of reader

VVar ("UE:BAR", "BAR.DLGID")

Input field ID

??= if no input field can be assigned

Otherwise the ID of the field is transferred (e.g. KNR)

VVar ("UE:BAR", "BAR.VALUE")

Bar code read without prefix (e.g. a badge number)

VVar ("UE:BAR", "DLG.ALL.FLD")

All dialog input field IDs separated by semicolon

VVar ("UE:BAR", "DLG.FLD")

ID of the target field identified in the dialog using the standard bar code identification. If no standard bar code is identified (BAR.DLGID=?), then the ID includes the currently focused dialog field.

VVar ("UE:BAR", "BAR")

Default value of the interpreted bar code

Ex. CNR=xxxxxxxx or ??=xxxxxxxx

VVar ("UE:BAR", "BAR.RAWDATA")

Original bar code string that has been read

VVar ("UE.DLG", "...")

Complete dialog data of the dynamic dialog before bar code processing (e.g. DLG.FOCUSED.FLD is the focused dialog field)

Return parameters:

Parameter	Value	Description
UE_RET		Return data from user exit Example: <pre> UE_RET = "" UE_RET = Item("CNR" , barval) </pre>

Example:

```

Function UserExitDynDlgBarcode
  Select Case VVar ("UE:BAR", "DLG")
    Case "A_P_AN_MPL", "A_AN_MPL", "CE_WL_MPL"
      OnBarcode_MPL
    End Select
  End Select
End Function

Sub OnBarcode_MPL
  Select Case VVar ("UE:BAR", "BAR.DLGID") ' fokusiertes Feld
    Case "DLL", "CNR", "??"
      ' gescannten Wert in das Dialogfeld CNR eintragen
  End Select
End Sub

```

```

    UE_RET=Item("CNR",VVar("UE:BAR","BAR.VALUE"))
End Select
End Sub

```

1.4.16 UserExitOnExternOrderListChange

Functionality:

This user exit is called if entries are added or removed when the order list is reloaded from the server. This is the case if the orders are logged on or off from the server by another terminal, MOC or by an automatism.

Input parameters:

Parameter	Value	Description
UE:DAT	<MNR1>=<ANR1> <MNR2>=<ANR2> ...	Added operations
UE:PAR	<MNR1>=<ANR1> <MNR2>=<ANR2> ...	Removed operations

Return parameters:

Parameter	Value	Description
UE_RET	---	without processing

Implementation notes:

For these orders, the system can perform customer-specific actions that are also performed if the order postings are directly made on the terminal. (Examples: setting an output signal, sending order data to a machine connection,...).

Example:

```

Function UserExitOnExternOrderListChange
    Dim asLostAGData,res
    If IsCustom_ErfassungOFF Then Exit Function
    ' UE:DAT: OPs added
    ' UE:PAR: OPs removed
    ' Format: <MNR1>=<ANR1>|<MNR2>=<ANR2>|<MNR3>=<ANR3>|<MNR4>=<ANR4>|
    asLostAGData=VVar("UE:PAR","#GET#ALL#VALUES#")
    If asLostAGDat<>"" Then
        res=DeleteOpFiles(asLostAGDat)
    End If
End Function 'UserExitOnExternOrderListChange

```

Reading data can be implemented as follows:

```

Dim iPos,sEntry,sMnr,sAnr
iPos=1
Do
    sEntry=scrGetPart(asLostAGDat,"|",iPos)
    If sEntry="" Then Exit Do
    sMnr=scrGetPart(sEntry,"=",1)
    sAnr=scrGetPart(sEntry,"=",2)
    '*** bearbeite sMaschine, sAuftrag
End If

```

```
iPos=iPos+1
Loop
```

1.4.17 UserExitOnGatewayData

Functionality:

This user exit is called with each message that is received via gateway port. There are two types of gateway messages (events):

- 1) **Notify GateWay-Events:** these messages are immediately identified as received and processed for the calling program. The processing is performed asynchronously in the main timer of the main program.

Do not write any interface results here because a reception confirmation has already been sent.

- 2) **Standard Gateway-Events:** These messages are processed as soon as the queue of the main program is processed.

Here, a result must be confirmed if the command is not for an active module.

Input parameters:

Parameter	Value	Description
UE:DAT	Gateway Event-Daten	Complete message in PDM format. For example: COM.ID=2@ DLG=KFS_MST MELDZEI=43200 MELDDAT=03/05/2018 BEARB=KFS MNR=M00000 2 MST=1 CLI.SND.T=10:03:59.983
UE_RET	Gateway Event-Return- Daten	Copy of the complete message in PDM format with attached .. RET=* for internal processing. For example: COM.ID=2@ DLG=KFS_MST MELDZEI=43200 MELDDAT=03/05/2018 BEARB=KFS MNR=M00000 2 MST=1 CLI.SND.T=10:03:59.983 RET=*

Return parameters:

Parameter	Value	Description
UE_RET	Gateway Event-Return- Daten	If you set the acronym #DATA#UPDATE# to the value TRUE , then you can change the return string.

Implementation notes:

You can change the return string of the event processing using the following assignment.

```
UE_RET = Item("#DATA#UPDATE#", "TRUE")
```

As part of a PCC_ADG interfacing, this interface supports the following further options:

- With the parameter “..**|NOTIFY.ERROR.TO.PCC=TRUE|**..”, a notification (answer) is sent to the PCC also in case of a server error if configured accordingly.

- With the parameter “..**EVENT=EXECUTE-AS-LIST**|..” the command is processed as list request.
- With the parameter “..**DATAFORMAT=ANSI**|..” the file loaded from AIP2 in UTF8 format is converted into "ANSI" for the PCC. (if <ANSIFILE=..> has not been specified, the file is converted into <FILE=..> or into ".\evcom.lst" without specification)
- With prefix "**DLG=STORED-EXECUTION:<CMD>**|.. ” the command is performed asynchronously in AIP2, i.e. the PCC does not wait for the result of the command.

e.g.

„DLG=ABC.REQUEST|FILE=c:\mpdv\aip2\spool\lu_data.lst|EVENT=EXECUTE-AS-LIST|NOTIFY.ERROR.TO.PCC=TRUE|DATAFORMAT=ANSI|“

Performs a list request with notification also with server error and converts the requested list into ANSI.

With **DLG= STORED-EXECUTION:ABC.REQUEST|..**“ the execution is performed asynchronously in AIP2

For further information, refer to the description of **scrGWCUpdateResult**

Example:

```
Sub UserExitOnGatewayData
    Dim sDlgID,sDATA,sTCMS,rc,sMsg
    sDlgID = VVar("UE:RET","DLG")
    Select Case sDlgID
        Case "U_MST","U_STK","U_HUB"
            sMsg = "# Event-Verarbeitung für Dialog [ <DLG> ] läuft! Bitte warten ... #"
            sMsg = scrTranslate(sMsg,Item("DLG",sDlgID))
            rc = scrStatusBarMsg(sMsg,"EVMsg",-1)
            sTCMS = scrDateTime("TCMS")
            sDATA = VVar("UE:RET","#GET#ALL#VALUES#")
            If vbsExecuteEvent(sDATA) Then
                scrLog("vbsExecuteEvent(TRUE) "+StrFmtRight(CStr(scrDateTime("TCMS")-sTCMS),8,"0") _
                    +" msec<"+sDlgID+">"+sDATA+"<")
                sMsg = "# Event-Verarbeitung für Dialog [ <DLG> ] beendet! #"
                sMsg = scrTranslate(sMsg,Item("DLG",sDlgID))
                rc = scrStatusBarMsg(sMsg,"EVMsg",1)
            Else
                scrLog("vbsExecuteEvent(FALSE) "+StrFmtRight(CStr(scrDateTime("TCMS")-sTCMS),8,"0") _
                    +" msec<"+sDlgID+">"+sDATA+"<")
                sMsg = "# Abbruch der Event-Verarbeitung für Dialog [ <DLG> ] ! #"
                sMsg = scrTranslate(sMsg,Item("DLG",sDlgID))
                rc = scrStatusBarMsg(sMsg,"EVMsg",1)
            End If
    End Select
End Sub
```

1.4.18 UserExitModifyListCmd

Functionality:

This user exit is called when the terminal requests lists from the server. Here, you can change the PDM command to load a list (e.g. DLG=LIST;74).

Input parameters:

Parameter	Value	Description
-----------	-------	-------------

UE:PAR	VVar("UE:PAR","XYZ")	Load command that the terminal has sent to the server
--------	----------------------	---

Return parameters:

Parameter	Value	Description
UE_RET		In UE_RET the changed send data can be returned. If you want to change the send string, the complete string must be transferred!

Example: If the list 74 is requested, the ID TEST=XYZ is added

```
Sub UserExitModifyListCmd
  UE_RET = ""
  Select Case VVar("UE:PAR","DLG")
    Case "LIST;74"
      ' If the list 74 is requested, the ID TEST=XYZ is added
      ' all data and the additional items must be returned
      UE_RET = VVar("UE:PAR","#GET#ALL#VALUES#") + Item("TEST","XYZ") + Item("XXX","XYZ")
    End Select
  End Sub
```

1.4.19 UserExitSysReadFile

Functionality:

This user exit is called, if a file has been loaded using the basic function of the main program (sys_read_file).

Input parameters:

Parameter	Value	Description
UE:PAR UE:DAT	Infostring	Information on the lists loaded including number of files

Return parameters:

Parameter	Value	Description
UE_RET	---	without processing

Implementation notes:

The loaded files are inserted in the list using lower case letters.

The files can be read using VVar("UE:PAR","mnr.lst") or VVar("UE:PAR","mnr.lst").

The data is structured as follows:

```
FILE:COUNT=1|anr.lst=c:\mpdv\aip2\anr.lst;8132;2011-05-27;09:24:45.036;1;|...
<FILE:COUNT>      =      <number of loaded files>
<file name>         =      <path+file name>          ;
                        <file size in bytes>          ;
                        <date> (Format YYYY-MM-DD) ;
                        <time> (Format HH:MM:SS.ZZZ) ;
                        <transfer format> (0=binary,1=text)
```

Example: A customer-specific action is performed when the machine list has been loaded (file "mnr.lst").

```
Sub UserExitSysReadFile
  If VVar("UE:PAR","mnr.lst") <> "" Then
    ' DO CUSTOM ACTION AFTER READ <MNR.LST>
    doCustomActionAfterReadFileMNR
  End If
End Sub
```

1.4.20 UserExitAfterListLoaded

Functionality:

Using this user exit, the developer can perform standard and custom extensions in the user exit after request of lists.

This user exit is therefore run twice to develop the custom and the standard extensions each.

1. *UserExitAfterListLoaded_LIST_13* → Custom implementation /
(if available) / *aip_system_<project>.scr*
2. *UserExitAfterListLoaded__LIST_13__* → Standard processing
(if available) / *aip_mpdv-system.scr*

Input parameters:

Parameter	Value	Description
UE:LST.PAR		Input parameter of list
UE:LST.SND		Send parameter for list command

Return parameters:

Parameter	Value	Description
UE_RET		

Example: Extension of list DLG=LIST;13|MOD=P|..

```
Sub UserExitAfterListLoaded_LIST_10
  Dim sFileName,rc
  sFileName=VVar("UE:LST.PAR","FILE")
  If Right(sFileName,7)="mnr.lst" Then
    rc=scrSetData("AddListFileColumn","FILE="+sFileName+"|AKRO=SEL|VALUE=")
  End If
End Sub
```

1.4.21 UserExitGetCellData

Functionality:

This user exit is used for the free programming of a field content in a grid.

To do so, you must define a field name in the grid configuration in the file ctaiply.ini, which starts with the character "@".

Example for ctaipplay.ini:

```
[order list]
...
@PAL.CNT=N10.0,60,R,A.S.a.P. ; number of parts on pallet
```

Input parameters:

Parameter	Value	Description
UE:RET	@GRD.ITMFLD	Acronym of the field (→@PAL.CNT)
	@GRD.ITMVAL	Previous value
	@GRD.ROWNUM	Row in the grid
	@GRD.COLNUM	Column in the grid
	@GRD.ACTROW	Active (selected) row of the grid
	@GRD.TABLENAME	Section in ctaipplay.ini → 'order list'
	@GRD.FILENAME	List file with path
	@GRD.EXTFILENAME	List file → "anr.lst"
	@GRD.INIFILE	Actual name of layout file
	@GRD.FILTER	Filter of grid (e.g. "MNR=4711")
	@GRD.ORDER	Sorting of grid
UE:GRD		The complete data row of the grid that is to be drawn

Return parameters:

Parameter	Value	Description
UE_RET	@GRD.ITMVAL	The value that is identified is returned in UE_RET using the ID "@GRD.ITMVAL".

Example:

```
Function UserExitGetCellData
Dim sFile,sAuftrag,sAcro,sValue
sFile=VVar("UE:RET","@GRD.EXTFILENAME")
If sFile="anr.lst" Then
sAuftrag=VVar("UE:GRD","ANR")
If IsNumeric(sAuftrag) Then
sAkro=VVar("UE:RET","@GRD.ITMFLD")
If sAkro="@PAL.CNT" Then
sMaschine=VVar("UE:GRD","MNR")
sValue=CStr(iSchlagPalette("Read",sMaschine,0))
UE_RET=Item("@GRD.ITMVAL",sValue)
End If
End If
```



```

End If
End If
End Function 'UserExitGetCellData

```

1.4.22 UserExitPzeCfgLoad

Functionality:

This user exit is called with the cyclic loading of the PZE configuration. You use this user exit to load additional customer-specific files.

Input parameters:

Parameter	Value	Description
UE:PAR	Terminal label (configuration)	Includes the data of the terminal label (configuration) Example: UE:PAR=TNR=826 TYP=830 CFG:1=1 HWADR=10.10.62.163 TZ= .. PORT= '

Return parameters:

Parameter	Value	Description
UE_RET	---	Return value is not evaluated in the terminal program. Example: UE:RET=RET=* .. CNT=< number of files loaded > ..

```

Sub UserExitPzeCfgLoad
    Dim cnt
    cnt = "0"
    If LoadWageType Then cnt=IncStrDec(cnt) '*** Lohnarten laden
    If LoadCostCenter Then cnt=IncStrDec(cnt) '*** Kostenstellen laden
    UE_RET = Item("CNT",cnt)
End Sub

Function LoadWageType
    UE_SND = ""
    UE_SND = Item("DLG", "SYSTEM.CALL" )
    UE_SND = Item("PROG", "custom_list.scr" )
    UE_SND = Item("AKTION", "lohnart" )
    UE_SND = Item("DATEI", ".\spool\lohnart."+SYS_USR )
    UE_SND = Item("FILE", "lohnart.lst" )
    ' ----- UE_SND = Item("CMD:CPY", "BINARY" ) ' load binary if required
    scrUECmd(UE_SND)
    LoadWageType=(VVar("UE:RCV","RET")="0")
End Function

Function LoadCostCenter
    UE_SND = ""
    UE_SND = Item("DLG", "SYSTEM.CALL" )
    UE_SND = Item("PROG", "custom_list.scr" )
    UE_SND = Item("AKTION", "kostenst" )
    UE_SND = Item("DATEI", ".\spool\kostenst."+SYS_USR )
    UE_SND = Item("FILE", "kostenst.lst" )
    ' ----- UE_SND = Item("CMD:CPY", "BINARY" ) ' load binary if required
    scrUECmd(UE_SND)
    LoadCostCenter=(VVar("UE:RCV","RET")="0")
End Function

```

1.4.23 UserExitAGInfoGetCaption

Functionality:

You use this user exit to customize the AIP dialogs MINFO(MMINFO) and AINFO(MAINFO).

Input parameters:

Parameter	Value	Description
UE:PAR	MODE	The dialod used to run the function is specified via the ID MODE="MINFO" or "AINFO"
	MNR.MNR	Machine number

Return parameters:

Parameter	Value	Description
UE_RET	Field ID	As return, the field IDs can be set in the info dialog.

Implementation notes:

The dialogs "MMINFO" and "MAINFO" are only available in the classic main view without XML GUI on AIP2.

It is possible to change the output or make changes for a dialog field.

You can also describe an added field in the dyn. dialog configuration

Example:

```

Sub UserExitAGInfoGetCaption
    Dim sShowMode, s, sl, sHub, sMnr, sSzy, rSzy, r
    On Error Resume Next
    s = VVar("UE:PAR", "#GET#ALL#VALUES#")
    scrLog(s)
    sMnr = VVar("UE:PAR", "MNR.MNR")
    sShowMode=VVar("UE:PAR", "MODE")      ' MINFO / AINFO
    'Ex.: Output CLOCK/MIN in MINFO
    'Query: MINFO=Machine info data
    'Query: AINFO=Order infor data
    If sShowMode = "MINFO" Then '// or respectively AINFO
        sMnr = VVar("UE:PAR", "MNR.MNR")
        mData = scrGetInfo("GetMachineData", "MNR="+sMnr)
        sSzy = scrDDitem("SZY",mData)
        rSzy = scrStr2Real(sSzy) / 1000
        If rSzy <> 0 Then
            r = 60 / rSzy
            s = RealToStrNK(r,2)
            sl = RealToStrNK(rSzy,2)
            UE_RET=Item("HUB",s) ' + Item("MNR.SZY",sl)
        End If
    End If
    If Err.Number <> 0 Then
        scrLog("Error:UserExitAGInfoGetCaption|ERR.Number:"&CStr(Err.Number) _
            & "|Source:"&Err.Source & "|Description:"&Err.Description&|"")
    End If
    On Error Goto 0
End Sub

```

1.4.24 UserExitCAQChangeImageTreeView

Functionality:

This user exit is used to change the standard image in the CAQ tree view. To this end, add images to the file pict_cust.zip.

To change data, call the user exit in a node of the tree.

Input parameters:

Parameter	Value	Description
UE : SND	NODEDATA	Here, the data of the internal tree node is available. The data is separated by the character chr(8) (backspace). The separator must then be converted to a pipe character.
	CHARACTERISTICDATA	Here, the characteristic data of the internal tree node is available. The data is separated by the character chr(8) (backspace). The separator must then be converted to a pipe character.
	IMAGEINDEX	Default ImageIndex of the application
	COLUMN	Column in tree (in the tree, 4 columns are available)
	QUALITYSTATE (numeric)	Current quality status of the node 0 = QS_IO 1 = QS_NIO 2 = QS_BEDINGT_IO 3 = QS_UNKOWN 4 = QS_CALC
	ERFASSUNGSSTATUS (numeric)	Current collection status of the node 0 = ESTA_NOETIG 1 = ESTA_MOEGLICH 2 = ESTA_FERTIG 3 = ESTA_ABGESCHLOSSEN 4 = ESTA_FEHLER 5 = ESTA_UNDEF 6 = ESTA_NULL
	ERFASSUNGSSTATUSTEXT	Current collection status as text

	TOLERANCELIMITREACHED (numeric)	Is tolerance of node respected 0 = TL_UNKNOWN 1 = TL_UPPER 2 = TL_LOWER
	TOLERANCELIMITREACHEDTEXT	Is tolerance of node respected
	INTERVENTIONLIMITREACHED (numeric)	Is action limit of node respected 0 = IL_UNKNOWN 1 = IL_UPPER 2 = IL_LOWER
	INTERVENTIONLIMITREACHEDTEXT	Is action limit of node respected

Return parameters:

Parameter	Value	Description
UE_RET	RET	To accept the new ImageIndex, RET=0 must be returned.
	IMAGEINDEX	The customer-specific images start from 20. By default, 5 images are read from the zip files.

Implementation notes:Standard images with indexCustomer-specific images

The customer-specific images are only loaded if the user exit is defined.

The files with the name "caq_image_[index].png" are then loaded. The images must have the format 24 x 24 pixels. The indexes for customer-specific images are always from 20 onwards.

In the standard configuration, 5 customer-specific images are loaded from the indexes 20 to 24. If you require more than 5 images, you can increase the number of possible customer-specific images in the file caq_dc_t.ini.

Example for caq_dc_t.ini:

```
[OPTIONS]
...
NUMBER_OF_ADDITIONAL_IMAGES=10
...
```

The images must be available in the file pict_cust.zip.

Example:

```
Sub UserExitCAQChangeImageTreeView
  scrLog VVar("UE:SND", "#GET#ALL#VALUES#")
  nodeData = scrStrReplace( VVar("UE:SND", "NODEDATA") , Chr(8) , "|" )
  merkmalData = scrStrReplace(VVar("UE:SND", "CHARACTERISTICDATA"), Chr(8) , "|")
  scrLog "nodeData:" + nodeData
  scrLog "merkmalData:" + merkmalData
  if (VVar("UE:SND", "COLUMN")) = "1" then
    'attributive without cavity (data collection based on characteristics)
    if ((scrDDItem("NODE:PREFIX", nodeData)) = "PPKT_MM") AND _
      ((scrDDItem("BEURTBASIS", nodeData) = "STICHPR_MSTP") OR _
      (scrDDItem("BEURTBASIS", nodeData) = "STICHPR_ESTP")) AND _
      (scrDDItem("KEINNEST", nodeData) = "1") then
      if (VVar("UE:SND", "NEWDATA") = "TRUE") then
        UE_RET = Item("RET", "0") + Item("IMAGEINDEX", "20")
      else
        'scrMsgBox VVar("UE:SND", "QUALITYSTATE")
        if (VVar("UE:SND", "QUALITYSTATE") = "0") then
          UE_RET = Item("RET", "0") + Item("IMAGEINDEX", "0")
        end if
        if (VVar("UE:SND", "QUALITYSTATE") = "1") then
          UE_RET = Item("RET", "0") + Item("IMAGEINDEX", "21")
        end if
      end if
    end if
  end if
  'attributive without cavity (data collection based on characteristics)
  if ((scrDDItem("NODE:PREFIX", merkmalData)) = "PPKT_MM") AND _
    ((scrDDItem("BEURTBASIS", merkmalData) = "STICHPR_MSTP") OR _
    (scrDDItem("BEURTBASIS", merkmalData) = "STICHPR_ESTP")) AND _
    (scrDDItem("KEINNEST", merkmalData) = "0") then
    if (VVar("UE:SND", "NEWDATA") = "TRUE") then
      UE_RET = Item("RET", "0") + Item("IMAGEINDEX", "20")
    else
      'scrMsgBox VVar("UE:SND", "QUALITYSTATE")
      if (VVar("UE:SND", "QUALITYSTATE") = "0") then
        UE_RET = Item("RET", "0") + Item("IMAGEINDEX", "0")
      end if
      if (VVar("UE:SND", "QUALITYSTATE") = "1") then
        UE_RET = Item("RET", "0") + Item("IMAGEINDEX", "21")
      end if
    end if
  end if
end if
End Sub
```

1.5 DIALOG scripts

The script dialog processing has been implemented for the initialization and the dialog control of new dynamic dialogs (that are not implemented in the source code).

This kind of dialog is configured or called via entry in the file "**ctaipbut.ini**" in the main view. In the new GUI, the dialog must be configured in a layout file such as e.g. "**l_anr.xml**", "**l_mnr.xml**", "**l_pnr.xml**" or "**l_res.xml**".

For information on the storage and naming of dialog scripts, refer to section "1.1.1 Storage". For notes on the processing, refer to section "1.1.2 Processing".

Currently, the following dialog user exits are implemented or defined:

Dialog "user exits"	Script description
DynDlgInit_[DIALOG-ID]	Dyn. dialog (initialization)
DynDlgGridInit_[DIALOG-ID]	Dyn. dialog (grid initialization)
DynDlgFieldChange_[DIALOG-ID]	Dyn. dialog (input field - change/bar code)
DynDlgFieldExit_[DIALOG-ID]	Dyn. dialog (input field - exit)
DynDlgFieldListe_[DIALOG-ID]	Dyn. dialog (input field - attached list)
DynDlgFunctions_[DIALOG-ID]	Dyn. dialog (button - function)
DynDlgBeforeSend_[DIALOG-ID]	Dyn. dialog (before DB posting)
DynDlgAfterSend_[DIALOG-ID]	Dyn. dialog (after DB posting with <RET=0 .>)
DynDlgTimer_[DIALOG-ID]	Dyn. dialog (timer for cyclic processings)
DynDlgFormValidationBeforeFunction_[DIALOG-ID]	Dyn. dialog (entry of validation before execution of function)
DynDlgKeyDown_[DIALOG-ID]	Dyn. dialog (keyboard - events)
DynDlgPluginCreate_[DIALOG-ID]	Dyn. dialog (plug-in - initialization)
DynDlgWFTabEnter_[DIALOG-ID]	Dyn. dialog (display before workflow)
DynDlgWFTabExit_[DIALOG-ID]	Dyn. dialog (exit before workflow)

The following examples for the DIALOG "XYZ" explain the functions provided by the dialog user exits.

1.5.1 DynDlgInit_XYZDynDlgInit_XYZ

Functionality:

This user exit is called when a script dialog is initialized.

<u>Available functions</u>	<u>Description</u>
VVar("DLG.DLG", "XYZ") oder VDlg("XYZ")	Basic initialization of the dyn. dialog (e.g. DLG=M_MST ...)
VPar("XYZ")	Parameter of the dialog call (or the dialog data of the calling dialog)
VMnr("XYZ")	Current machine info from MNR.LST for the machine selected in the main view. If the acronym is included in in VPar("XYZ") , this acronym is used.
VAnr("XYZ")	current order info from ANR.LST for the order selected in the main view. If the acronym is included in in VPar("XYZ") , this acronym is used.
VVar("DLG.CGD", "XYZ")	includes (if available) the current row of the third grid of the main view or the selected row if the call has been made using a button in a dynamic dialog with grid.

The functions: VMnr(), VAnr() und VVar("DLG.CGD") include the following additional information:

<#FILE#LIST#> → includes the file name without path
 <#FILE#NAME#> → includes file name with path
 the values of these fields are in lower case letters

For the active grid of the main view, the following information is additionally passed:

→ "..|#GRD#STATE#=FOCUS|.."

From a script dialog with grid, the following value is passed

→ "..|#GRD#STATE#=DIALOG|.."

Implementation notes:

- (1) If a value is entered in the dialog, the information is not updated, i.e. if you change the machine or the order in the dialog, these values are not changed.
- (2) Also if you access the **DynDlg..._** user exit that follows, the variable content might not be available or correct.

The values required for the processing should be included in STATUS or in hidden dialog fields.

You can create a hidden field in the dialog in each **DynDlg...** user exit sing **DLGVAR = Item("MNR" , VMnr("MNR"))**.

You can also save values using **GLOBALVARS = "#XXX #PAR#=WAAGENTERMINAL=1"**. The developer is responsible for editing the contents and deleting after use.

- (3) Mind the note in section "Dynamic dialog/workflow with a WF step"

```
Sub DynDlgInit_XYZ
  If VOut("REOPEN") = "J" Then
' ----- repeated opening of the dialog, e.g. after DB plausibility error <RET=..|KT=..|LT=..|>
  Else
' ----- Plausibility checks if authorization for opening dialog exists
    If "X" <> "X" Then
      scrMsgBox(" Dialog -> Plaus. error [ "+VOut("DLG")+ " / "+VOut("ScriptID")+ " ]")
      DLGVAR = Item("RET", "$">+VOut("DLG")+"<")
      DLGVAR = Item("KT", "(Kurztext)")
    
```

```

    DLGVAR = Item("LT", "(Langtext)")
    ' alternativ: Dialog nicht öffnen - ohne Fehlermeldung:
    'DLGVAR = Item("RET", "#INVISIBLE#MSG#")
Else
' ----- opening the dialog e.g. via <ButtonClick()> or via <Remote-Dialog-Call()>
'   scrMsgBox(" Dialog -> Init [ "+VOut("DLG")+" / "+VOut("ScriptID")+" ]")
    DLGVAR = Item("DT", SYS_DT, "")
    DLGVAR = Item("MNR", VMnr("MNR"))
    DLGVAR = Item("ANR", VAnr("ANR"))
    'DLGVAR = AddIt("CNR", "", cFFDisable)
End If
End If
End Sub

```

1.5.2 DynDlgGridInit_XYZ

Functionality:

If the dialog includes a grid, you can initialize it using this user exit. Condition: The grid must be configured with the "field attribute" SCRIPT_GRID.

Input parameters:

Parameter	Description
GRD.CMD	Command to load the list from the server If value is set, the list is loaded on opening of dialog.
GRD.FILE	The data of this file (in the subdirectory "spool") is displayed.
GRD.INI	Configuration file including the layout configuration (default: ctaiply.ini).
GRD.SECTION	Section in the configuration file that includes the layout.
GRD.FILTER	Filter criterion to show only part of the data records of the list.
GRD.ORDER	Sorting criterion – you can specify several field IDs separated by " ". The first criterion has the highest priority. Example for descending sorting: GRID_ORDER=MSDAUER=-

Implementation notes:

1. With the following entry, you can also use the < GRID_ORDER > entry of a < SCRIPT_GRID > that is included in the configured INI section of the INI file.

```
SCRVARS = "GRD.ORDER=" + "#USE#INI#ITEM#"
```

2. If you want to reload the grid after having changed the file, you can set the following value in a dialog script (e.g. DynDlgFieldListe_xx):

```
DLGVAR = Item("DLG.GRID", "RELOAD", "")
```

Example:

```

Sub DynDlgGridInit XYZ
    SCRVARs = "GRD.CMD=" + "DLG=LIST;u_l_list|MOD=U|MNR=<MNR>|ANR=<ANR>|"
    SCRVARs = "GRD.FILE=" + "u_list.lst"
    SCRVARs = "GRD.INI=" + "hytnrcfg.ini"

```



```

SCRVARS = "GRD.SECTION="+ "Layout->U_LIST"
SCRVARS = "GRD.FILTER=" + ""
SCRVARS = "GRD.ORDER=" + "CNR"
End Sub

```

1.5.3 DynDlgFieldChange_XYZ

Functionality:

After execution of the function **scrFieldChange**, the result configured with "LST.RET" is passed to <[n#]DLG.OUT>.

If no entry is found, the specified fields are deleted and the input field is colored in **magenta**.

Important!

In the user exit itself *-identifiers cannot be set (DLGVAR = Item("**ABC", ... is ignored).

Input parameters:

Parameter	Value	Description
DLG.DLG VDlg ("...")	Dialog data	All dialog data in the dynamic dialog as PDM string. Note: the data changed is not yet set here.
	DLG.FLD	ID of the changed field (e.g. MST)
	DLG.VAL	Value of the changed field (e.g. MST)
	DLG.GRD	Selecting a row in the grid
	DLG.GRD.DBLCLK	Selecting a row in the grid via double-click
VStore ("...")	Data row	Selected row in the grid with all data
	DLG.GRD.ROWCOUNT	Returns number of rows in the grid
	DLG.GRD.REOPEN	Is called when the grid is reopened (reloaded) If you set the value "DLGVAR=DLG.PROCESS.RESULT=TRUE" the assigned values are processed.

Return parameters:

Parameter	Value	Description
DLG.OUT	Field ID	As return, the field IDs can be set in the dialog.

Example 1: If you manually enter a machine status in field "U_MST", the relevant machine status text is identified and entered in the dialog field "U_MSTTXT". If the machine status text is not found, the field "U_MST" is colored in **magenta**.

```
Sub DynDlgFieldChange_XYZ
```

```

Select Case VDlg("DLG.FLD")
Case "U_MST"
    LSTVARS = "LST.FILE=" + "mstat.lst"
    LSTVARS = "LST.FILTER=" + "MNR="+VDlg("MNR")+ " & "+"MST="+VDlg("FLD.VAL")
    LSTVARS = "LST.RET=" + "U_MSTTXT=MSTTXT"
    scrFieldChange
End Select
End Sub

```

Example 2: If you select a row, data of this row should be passed to the dialog fields.

```

Sub DynDlgFieldChange RES AB
Select Case VDlg("DLG.FLD")
Case "DLG.GRD", "DLG.GRD.DBLCLK"
    If VStore("RES") <> "" Then
        DLGVAR=Item("RES",VStore("RES"))
        DLGVAR=Item("RESTYP", VStore("RESTYP"))
    End If
End Select
End Sub

```

Example 3: When the file is opened, the value of the first row of column "POS" is passed to the field "POS" and focused.

Via double-click, the value changes between blank ("") and "X" in the column "SELECT" of the currently selected row. These implementations are often used if a multiple selection is implemented.

```

Sub DynDlgFieldChange XYZ
Select Case VDlg("DLG.FLD")
Case "DLG.GRD.REOPEN"
    ' after reading the grid, the field <POS> is filled with the value
    ' of the first row and focused.
    DLGVAR = Item("DLG.PROCESS.RESULT","TRUE" )+Item("POS", VStore("POS")+";#F" )
Case "DLG.GRD.DBLCLK"
    'select row in grid
    If VStore("SELECT") = "X" Then
        tmp = SStore("SELECT","")
    Else
        tmp = SStore("SELECT","X")
    End If
End Select
End Sub

```

Note when implementing the event "DLG.GRD.DBLCLK" that the AIP is primarily intended for use with a touch screen. A double click is not a practical on a touch screen. It is better to make a selection using an additional button in the dialog.

1.5.4 DynDlgFieldExit_XYZ

Functionality:

This user exit is called if an input field in the dialog is exited or if a field has obtained a bar code. The bar code event can be identified via the request:

If VDlg("FLD.MOD")="BARCODE" Then

Implementation notes:

The dialog data is available in VDlg(„XYZ“). Use the function DLGVAR to pass values to the dialog.

Input parameters:

Parameter	Value	Description
DLG.DLG VDlg ("...")	Dialog data	All dialog data in the dynamic dialog as PDM string.
	DLG.FLD	ID of the changed field (e.g. MST:1) "FLD.MOD" = "FLDEXIT" Field that has been exited "FLD.MOD" = "BARCODE" target field of bar code
	FLD.MOD=MOUSEDOWN	Mouse button has been pressed

Return parameters:

Parameter	Value	Description
DLGVAR	Dialog data	As return, the field IDs can be set in the dialog via DLGVAR.

Example 1: After having exited a field with the ID "MST:1", the status text is read from the list and entered in the dialog in field "MSTTXT:1".

```
Sub DynDlgFieldExit XYZ
  Select Case VDlg("DLG.FLD")
    Case "MST:1"
      LSTVARS = "LST.FILE="      + "mstat.lst"
      LSTVARS = "LST.FILTER="    + "MNR="+VDlg("MNR")+ " & MST="+VDlg("MST:1")
      LSTVARS = "LST.RET="      + "MSTTXT:1=MSTTXT"
  End Select
scrFktList
End Sub
```

Example 2: Using the extension < VDlg("FLD.MOD") = "MOUSEDOWN" >, you can implement a "localization grid" in a configured <IMAGE> using the field attribute <MOUSEDOWN>. For example, in this "grid recording", a grid is placed over an article image in order to be able to record the precise position of a defect.

```
Sub DynDlgFieldExit_XYZ
  Dim x,y
  If VDlg("FLD.MOD") = "MOUSEDOWN" Then
    Select Case VDlg("DLG.FLD")
      Case "ATKIMG"
        x = Int( CInt(VDlg("ATKIMG@XPOS")) / Int(CInt(VDlg("ATKIMG@WIDTH")) / 10))+1
        y = Int( CInt(VDlg("ATKIMG@YPOS")) / Int(CInt(VDlg("ATKIMG@HEIGHT")) / 5))+1
        If x > 10 Then x = 10
        If y > 5 Then y = 5
        DLGVAR = Item("RASTER:X",CStr(x))+ Item("RASTER:Y",CStr(y))
      Case Else
    End Select
  Else
    ...
  End Select
End Sub
```

Example 3: When a field with the ID "ABC" is left, the column "2" in the dialog grid is selected. Using the optional parameter [;LOCK] you can prevent that the processing "DynDlgFieldChange_XYZ" is executed. These implementations ensure that in case of a later selection, also in the current grid row a selection is still possible. Note: If a dialog is opened with a grid, the column "2" is always opened initially. With a later selection, the column "2" should therefore be configured to be hidden (display width of 0 pixels, e.g. "DMY1=C3,0,Z").

```
Sub DynDlgFieldExit XYZ
  Select Case VDlg("DLG.FLD")
    Case "ABC"
      DLGVAR=Item("GRD.SETCOL","2;LOCK")
  End Select
End Sub
```

1.5.5 DynDlgFieldListe_XYZ

Functionality:

You can use this user exit to implement a list selection for any field.

Input parameters:

VDlg („DLG.FLD“) includes the ID of the field whose list button has been pressed.

The field attribute "DIALOGLIST" is set in the dialog configuration to show a list button. The value "SCRIPT_LIST" is also set in "Dialog list function".

Example for a field with a list button: **Ausschussgrund** 

The function LSTVARS is filled with the parameters for the list:

LST.CMD	Command to request the list from the server (optional).
LST.FILE	File name (the local directory "spool" is always put in front).
LST.CAPTION	Window caption of the selection dialog
LST.FILTER	Filter for the list to be displayed (e.g. "MNR=100 & ZUMAN=J N")
LST.SORT	List sorting
LST.INI	INI file where the <section> is read (""=ctaipay.ini)
LST.SECTION	INI section including the layout definition of the list to be displayed
LST.RET	Configuration of the values from the list that are transferred into the calling dialog e.g. < MST:1=MST"+" & "+"MSTTXT:1=MSTTXT" > copies the values of columns <MST> and <MSTTXT> of the selected entry of the list in the dynamic dialog fields <MST:1> und <MSTTXT:1>.
LST.MODE	Additional processing modes (configurations separated by " ") "COLNUMSORT=TRUE" (or in INI section GRID_COLNUMSORT)

	"DYNAMICFILTER=MGRP,MNR,MST" (or in INI section <GRID_DYNAMICFILTER) "PAGESCROLLING=TRUE" (or in INI section GRID_PAGESCROLLING) "WILDCARD=+" "FILTERSENSITIVE=TRUE" (or in INI section GRID_FILTERSENSITIVE)
--	---

Implementation notes:

The result string configured in LST.RET is transferred into the global variable <[n#]DLG.OUT> (is equal to <DLGVAR>). Additionally, the complete row selected is stored in <[n#]LST.VALUES>. If the list has been read by the server (→LST.CMD), the result of the server request is saved in <[n#]LST.CMD:RET>.

In general, only a static list should be specified as **LST.FILE** for the display without previous update (LST.CMD="").

Example: Selection of a local list

Example : Input of an additional MST in a dialog (this selection is not realizable in the standard system, i.e. a realization is only possible with script).

Using the general selection list **scrFieldList**:

```

Sub DynDlgFieldListe XYZ
  Select Case VDlg("DLG.FLD")
    ' ---- Input field with dialog list button
    Case "MST:1"
      ' ---- Initialization of the <LSTVARS>
      LSTVARS = ""
      ' ---- File name (local spool directory is always put in front)
      LSTVARS = "LST.FILE=" + "mstat.lst"
      ' ---- Window caption of the selection dialog
      LSTVARS = "LST.CAPTION=" + "machine status list [ <DLG> ]"
      ' ---- Filter auf die anzuzeigende Liste ( z.B. "MNR=100 & ZUMAN=J|N" )
      LSTVARS = "LST.FILTER=" + "MNR="+VDlg("MNR")
      ' ---- Sortierung der Liste
      LSTVARS = "LST.SORT=" + "MNR|MST"
      ' ---- Ini file where the <section> is read ("=ctaisplay.ini)
      LSTVARS = "LST.INI=" + ""
      ' ---- Ini-Section mit der Layoutdefinition der anzuzeigenden Liste
      LSTVARS = "LST.SECTION=" + "Maschinenstatusliste"
      ' ---- Configuration of the values from the list that are transferred into the calling
      dialog
      ' ---- - z.B. < MST:1=MST"> & "+"MSTTXT:1=MSTTXT" >
      ' ---- - kopiert die Werte der Spalten <MST> und <MSTTXT> des selektierten Eintrag
      ' ---- - der Liste in die dynamischen Dialogfelder <MST:1> und <MSTTXT:1>
      ' ---- - bei < MST "+" & "+" MSTTXT" > erfolgt keine DlgID - Umsetzung
      LSTVARS = "LST.RET=" + "MST:1=MST"+" & "+"MSTTXT:1=MSTTXT"
      ' ---- Additional processing modes (configurations separated by "|") ' ---- zusätzliche
      Verarbeitung-Modi (mit "|" getrennte Konfigurationen)
      ' ---- - "COLNUMSORT=TRUE" (or in Ini-Section <GRID_COLNUMSORT)
      ' ---- - "DYNAMICFILTER=MGRP,MNR,MST" (or in Ini-Section <GRID_DYNAMICFILTER)
      ' ---- - "PAGESCROLLING=TRUE" (or in Ini-Section <GRID_PAGESCROLLING)
      ' ---- - "WILDCARD=+"
      ' ---- - "FILTERSENSITIVE=TRUE" (or in Ini-Section <GRID_FILTERSENSITIVE)
      LSTVARS = "LST.MODE=" + ""
      ' ---- further Ini-Section-configurations / internal note
      ' ---- - GRID_MAXIMIZE_LIST=TRUE (displays the selection list maximized)
      ' ---- - the field contents of the calling dialogs are also transferred
scrFieldList
  End Select
End Sub

```

Example: Selection of an ONLINE/server list

Example: Implementation of a module or a customer-specific sequencing list (user exit to server)

Verwendung der Auswahlliste **scrFieldList**.

INI section [sequencing list] of customer-specific global INI file **hytnrcfg.ini**.

```
Sub DynDlgFieldListe A AN RS
  Select Case VDlg("DLG.FLD")
    case "ANR"
      ' ---- Initialization of the <LSTVARS>
      LSTVARS = ""
      ' ---- File name (local spool directory is always put in front)
      LSTVARS = "LST.FILE=" + "vrslst.lst"
      ' ---- Server command to request file
      LSTVARS = "LST.CMD=" + "DLG=LIST;11|MOD=V|MNR="+VDlg("MNR")+ "|MOD3=M|"
      ' ---- Window caption of the selection dialog
      LSTVARS = "LST.CAPTION=" + "Vorgabeliste [ <MNR> -> <DLG> ]"
      ' ---- Ini file where the <section> is read ("=ctaisplay.ini)
      LSTVARS = "LST.INI=" + "hytnrcfg.ini"
      ' ---- Ini file where the <section> is read ("=ctaisplay.ini)
      LSTVARS = "LST.SECTION=" + "sequencing list"
      ' ---- Configuration of the values from the list that are transferred into the calling
      dialog
      LSTVARS = "LST.RET=" + "ANR" & "+" & "ATK" & "+" & "ABEZ=AGBEZ"
      ' ---- Additional processing modes (configurations separated by "|")
      ' ---- - "COLNUMSORT=TRUE" (or in Ini-Section <GRID_COLNUMSORT)
      LSTVARS = "LST.MODE=" + ""
scrFieldList
  Case Else
    ' scrMsgBox ( "FLD.LISTE = "+VDlg("DLG.FLD") )
  End Select
End Sub
```

1.5.6 DynDlgFormValidationBeforeFunction_XYZ

Functionality:

You use this DIALOG user exit to check the dialog entries before the user exit "DynDlgFunctions_XYZ" is executed.

DLG.RESTYP in DLGVAR wird hier nicht verarbeitet.

DLGVAR=Item("DLG.PROCESS.RESULT", "FALSE") → DLGVAR wird nicht übernommen

DLGVAR=Item("DLG.FORM.VALIDATION", "TRUE") → FormValidation is executed before the configured button function.

DLGVAR=Item("DLG.SET.FORM.VALIDATION.ERROR", "XXX") → Function is not executed; the field with the identifier XXX is selected and red

Example:

```
Sub DynDlgFormValidationBeforeFunction_XYZ
  If VDlg("DLG.FKT") <> "" Then
    ' --- <FormValidationBeforeFunction> activate because of defined <Button> function
    Select Case VDlg("DLG.FKT")
      Case "DLG=V_BLZ"
        ' --- Transfer of <DLGVAR> before <FORMVALIDATION> ---
        DLGVAR = Item("DLG.PROCESS.RESULT", "TRUE")
        DLGVAR = Item("DLG.RESTYP", "9")
```

```

' --- perform / activate <FORMVALIDATION> before <Button> function ---
DLGVAR = Item("DLG.FORM.VALIDATION","TRUE")
If VDlg("XXX") = "" Then
    ' --- Setting a <FORM.VALIDATION.ERROR> independent of DynDlg configuration ---
    DLGVAR = Item("DLG.SET.FORM.VALIDATION.ERROR","XXX")
    ' MsgBoxPopUp scrTranslate("Wert für Feld <XXX> erforderlich","", "3"
End If
Case Else ' DEFAULT [ VTST, .. ]
    DLGVAR = Item("DLG.FORM.VALIDATION","FALSE")
End Select
Else
    ' --- <FormValidationBeforeFunction> aktivieren Aufgrund von <Button>-<RCODE>
    Select Case VDlg("DLG.RESTYP")
        Case "0","7"
            ' OK
            ' DLGVAR = Item("DLG.FORM.VALIDATION","TRUE")
        Case "1"
            ' CANCEL
    End Select
End If
End Sub

```

1.5.7 DynDlgFunctions_XYZ

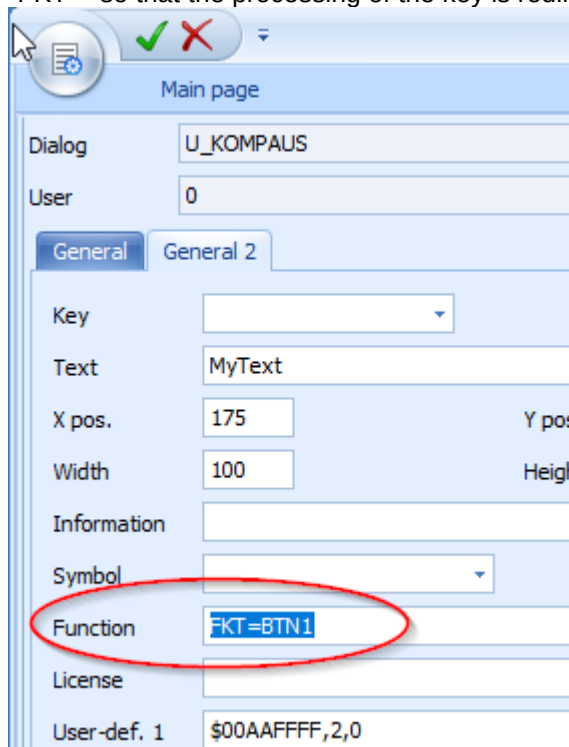
Functionality:

You use this user exit to implement function keys of the dialog. It is also called if the dialog is exited via OK or CANCEL and if the processing returns from a dialog called.

Implementation notes:

Note the following for function keys:

- The assignment of the function to the function key in the dialog configuration must start with "FKT=" so that the processing of the key is redirected to the terminal script.



- For the example above, the request would be "BTN!" in the select-instruction for VDlg („DLG.FKT“).
- A value must be returned to the function. The terminal program then knows that the function key has been processed in the terminal script. Otherwise an error message is displayed.
Example: DLGVAR="RET=0"

The return from a called dialog (e.g. CASE "DLG=U_FILTER") occurs if the call was made via the function key configuration (also with "DLG=U_FILTER").

The data of the dialog called (and now closed) is then available in VVar("DLG.RET","XYZ"). You can access the dialog called as usual via VDLG(„XYZ“).

If you leave the dialog, the case "DLG.CLOSE=TRUE" is run. The query <<VDlg("DLG.RESTYP")="1">> specifies if the request was started by clicking OK or CANCEL.

```
Sub DynDlgFunctions_XYZ
  Select Case VDlg("DLG.FKT")
    Case "MST:BTN:1"
      ' *** Example for a selection list -> unusual see < DynDlgFieldList_... >
      DLGVAR = "RET=0"
      LSTVAR = "LST.FILE=" + "mstat.lst"
      LSTVAR = "LST.CAPTION=" + "machine status list [ <DLG> ]"
      LSTVAR = "LST.FILTER=" + "MNR="+VDlg("MNR")
      LSTVAR = "LST.SORT=" + "MNR|MST"
      LSTVAR = "LST.SECTION=" + "Multi->Maschinenliste"
      LSTVAR = "LST.RET=" + "MST:1=MST" & " "+"MSTTXT:1=MSTTXT"
scrFieldList
    Case "DLG=U_XYZ"
      ' *** Example of a return to the calling dialog/script
      ' *** after execution of a script dialog with the
      ' *** Condition: button with function „DLG=U_XYZ“ must have RCode (7,8,9)
      ' *** Purpose: take over values
      ' *** VDlg("<>") -> data of calling dialog
      ' *** VVar("DLG.RET","<>") -> data of dialog called
      If VVar("DLG.RET", "DLG.RESTYP") = "7" Then
        DLGVAR = Item("FAKTOR", VVar("DLG.RET", "FAKTOR"))
      End If
    Case "DLG=U_ABC"
      ' *** Example of a return to the calling dialog/script
      ' *** after execution of a script dialog with the
      ' *** Condition: button with function „DLG=U_XYZ“ must have RCode (7,8,9)
      ' *** Purpose: process control
      If VVar("DLG.RET", "DLG.RESTYP") = "1" Then
        DLGVAR = Item("DLG.RESTYP", "9") ' dialog remains open
      Else
        DLGVAR = Item("DLG.RESTYP", "0") ' dialog is closed and sent
      End If
    Case "DLG.CLOSE=TRUE"
      ' *** To prevent that dialog can be closed if a condition is met
      ' *** Ex.: If the dialog field has the value <> „0“ the dialog must not be
      ' *** closed via ESC / virtual key "Cancel" or button with RCode (1)
      Select Case VDlg("DLG.RESTYP")
        Case "1" ' CANCEL
          If VDlg("NUM") <> "0" Then
            DLGVAR = Item("DLG.CLOSE", "FALSE")
          End If
        Case "0" ' OK
          '
      End Select
    Case "OUT:1#2"
      ' *** Script function <FKT=OUT:1#2> to set an output with PCCDLL connection
      ' set output 1 (=channel 301) and remove output 3 (=channel 303)
      scrPCCValues("DLG=SETVAL|O:0301=1|O:0303=0|")
  End Select
End Sub
```



```
End Select
End Sub
```

1.5.8 DynDlgBeforeSend_XYZ

Functionality:

Just like the „UserExitDynDlgBeforeSend“, the dialog-specific user exit „DynDlgBeforeSend“ is called before a posting is sent to the server. The call is only performed if the dialog script is loaded - i.e. if the dialog is open or has just been closed with this posting. If a posting with identical dialog ID (DLG=XYZ) is sent in the background when the dialog is closed, the dialog script does not work.

Implementation notes:

If „DynDlgBeforeSend“ is loaded, then „UserExitDynDlgBeforeSend“ is not run!

Especially with complex project, it is therefore recommended to use only "UserExitDynDlgBeforeSend" in the system script. It is also possible to call "UserExitDynDlgBeforeSend" directly from "DynDlgBeforeSend", so that the functions implemented are effective.

You can use the function DLGSND to change the send string as shown in the example below. The processing of the posting is controlled via the ID "EVENT=EVENT_...":

EVENT_DIALOG_OHNE_SENDEN:

If you press OK, the dialog is not sent. Is used, if the dialog is only meant to display data, or if the actual posting is explicitly sent by a script function (e.g. scrDDSndRcv()) when the dialog is open.

EVENT_OHNE_AUTO_MENGEN:

No automatic quantities of the machine are added to the posting. You should use this setting for customer-specific postings because the server does not process automatic quantities by default. This way, quantities can be lost.

EVENT_MIT_AUTO_MENGEN:

This is the default behavior.

EVENT_QUEUE_OHNE_AUTO_MENGEN, EVENT_QUEUE_MIT_AUTO_MENGEN:

This posting is first set in the queue of the terminal and is then issued with delay. The same behavior is used in the standard system for shift changes and for PZE COME/GEHT postings.

EVENT_ONLINE_OHNE_AUTO_MENGEN, EVENT_ONLINE_MIT_AUTO_MENGEN:

The posting may only be sent online. If an immediate posting cannot be sent to the server, the data record is not added to the queue. Instead the message is rejected with an error code. This variant is used if it is important for further processing that the posting has been booked on the server. This way, the server can perform processing steps that are not known to the terminal. After confirmation of the booking, the terminal can load lists that include the result of the processing.

Example:

```
Sub DynDlgBeforeSend_XYZ
  DLGSND = Item("DT", SYS_DT)
  DLGSND = Item("EVENT", "EVENT_ONLINE_OHNE_AUTO_MENGEN")
  ' *** Here, you can change / correct the dialog send ID / field ID if required
```

```
DLGSND = Item("DLG", "A_TR")
DLGSND = Item("MST", VDlg("MST:1"))
End Sub
```

1.5.9 DynDlgAfterSend_XYZ

Functionality:

This user exit is called, once a posting has been transferred successfully to the server. It is an alternative option to the UserExitDynDlgAfterSend in the system script. The same rules apply with respect to the processing at the same time as for DynDlgBeforeSend and UserExitDynDlgBeforeSend.

Implementation notes:

With script dialogs, the main lists (MNR,ANR,PNR,TNRMAT) are loaded by default after a posting. If you set the item <LOAD>, the standard update is not performed. In the example below, the MNR.LST and MSTAT.LST are reloaded in addition to the reloads (VRcv("LOAD")) set by the server.

If you add <RES> for the resource list (as of MDE/WRM >= 7.2.1), the system ignores this, if the product version <WRM> is smaller than <7.2.1> or if no active resource list display is configured for this terminal (<MNR.VISLIST3> does not include „R“).

Example:

```
Sub DynDlgAfterSend_XYZ
' *** for DD-LIST-Reload [ ANR,MNR,PNR,MAT,MST,RES ""=no DD-Lst-Updates] *****
' *** <RES> for resource list (as of WRM/MDE > 7.2)
' *** => this row updates the MNR.LST + MSTAT.LST on the terminal
' *** => with „VRcv("LOAD")" the DD-List-Reloads are added by the "Server"
DD_RCV = Item( "LOAD", "MNR,MST," +VRcv("LOAD") )
End Sub
```

Tip 1: reopen dialog after posting until "Cancel" is pressed:

```
Sub DynDlgAfterSend_RES_AN
rc=scrSetData("DelayedButtonClick","RES_AN")
End Sub
```

Tip 2: reopen dialog after posting until sending is successful (prevent "Cancel"):

```
rc=scrSetData("DelayedButtonClick","CA_WL|FORCEDIALOG=ON")
```

1.5.10 DynDlgWFTabEnter_XYZ

Functionality:

This DynDlg user exit is executed before the display of a dialog or a workflow tab. You can use this user exit to perform an initialization similar to the configured dialog function (1) (for example, "FKT=DLGSHOW") in user exit <DynDlgFunctions_XYZ> without changing the dynamic dialog configuration.

Example:

```
Sub DynDlgWFTabEnter_XYZ
```

```
If VDlg("*XYZ") = "" Then
    DLGVAR = Item("*XYZ", "1")
End If
End Sub
```

1.5.11 DynDlgWFTabExit_XYZ

Functionality:

This DynDlg user exit is executed before a dialog or a workflow tab is exited. You can use this user exit to perform an initialization similar to the configured dialog function (2) (for example, "FKT=DLGEXIT") in user exit <DynDlgFunctions_XYZ> without changing the dynamic dialog configuration.

Implementation notes:

If you set the item <RESULT> to the value <FALSE>, the dialog cannot be closed or the workflow tab cannot be exited.

Example: If the field "ABC" is empty, the workflow tab is not left. The field turns to red.

```
Sub DynDlgWFTabExit XYZ
    Dim rc
    If VDlg("ABC") <> "" Then
        DLGVAR = Item("RESULT", "FALSE")
        rc=scrSetData("SetFocusToField", "DLG=@ACTIVE|AKRO=ABC|RED=1")
    End If
End Sub
```

1.5.12 DynDlgTimer_XYZ

Functionality:

Besides the timer function available in the system script via UserExitMainLoopStop, "DynDlgTimer" can also be used to implement a cyclic call within the dialog.

Implementation notes:

The timer is activated, if you set the interval in ms in the UserExit DynDlgInit_XYZ:

```
DLGVAR=Item("DYNDLG.TIMER", "100")
```

The timer event is only triggered, if the terminal is running in the foreground.

You can change the interval in the timer. You deactivate the timer if you pass "0".

Example:

```
Sub DynDlgTimer XYZ
    ' *** DLG.RESTYP is not processed in the result <DLGVAR>
    ' *** the following row displays the date and the current time in the dialog field <DT>
    DLGVAR = Item("DT", Cstr(now))
End Sub
```

1.5.13 DynDlgKeyDown_XYZ

Functionality:

You use this DynDlg user exit for the dialog-specific processing of keyboard events. In general, you can use this user exit to react in the script to each single key pressed. But because the focus changes each time, this can have the result that you must reprogram basic editing functions in the script. It is therefore recommended to use the function only for the ENTER key (13) as in the example.

```
Sub DynDlgKeyDown_XYZ
  If VDlg("DLG.FLD") = "KNR" Then
    If VVar("DLG.PAR", "KEY") = "13" Then
      ' ... <Action> ...
      DLGVAR = AddIt("KNR", "", cFFFocus)
    End If
  End If
End Sub
```

1.5.14 DynDlgPluginCreate_XYZ

Functionality:

You use this DynDlg user exit to initialize a dialog / workflow plug-in.

The processing is equal to the processing of the DynDlg user exit <DynDlgInit_XYZ>.

This user exit is only used with CAQ dialogs.

1.6 Porting notes from CTWIN/AIP to AIP2

Find below in the following sections some notes if you change from the programs CTWIN or AIP to AIP2 and if you want to port functions.

1.6.1 Dynamic dialog/workflow with one WF step

The difference in the terminal script processing in AIP and AIP2 in case of a workflow with one workflow step is as follows:

Example: Dialog/workflow: [U_TST] DialogTab: [WF_U_TST]

On the AIP, all DynDlg-UEs were executed as "workflow script" ("U_TST").

On AIP2 and independent of the dynamic workflow/dialog configuration:

- the UE "DynDlgInit_" is always executed as "workflow script" ("U_TST").
- all other DynDlg-UEs are called in dialog TabScript ("WF_U_TST").

As of AIP2 version 8.2.1.10, the processing is performed with the WorkFlow configuration

"Step 1"	"WF_U_TST"	(STEP:1= WF_U_TST)
"Script"	"W"	(WFSCR:1=W)

analogous to the processing on AIP in the "workflow script" ("U_TST").

1.6.2 Porting of customer-specific terminal scripts

VB script does not support the required UTF-8 format when normal files are written.

To write data, you should use callback functions. The following code examples illustrate the use of the VB script function on AIP and its implementation on AIP2.

```
'-----
'- AIP: Function <ViewSelectedInfo> aus <aip_mpdv-AINFO_NOTES.scr>
'-----
Sub ViewSelectedInfo
    ' write the note selected in the grid into the memo
    '     raw data from server: notes.lst
    ' processed for grid: notes_gr.lst
    ' text for TextViewer: notes.txt (-> is created here)
    Const ForReading=1, ForWriting=2, ForAppending = 8
    Dim asGrid,sKey1,rc,fso,sDatFileName,sTxtFileName,f,ts
    asGrid=scrGetInfo("GetGridData","DLG=@FIL=DLG=AINFO_NOTES|LINE=-1")
    sKey1=scrDDItem("SUBKEY:1",asGrid)
    sDatFileName=DIR_SPOOL+"notes.lst"
    sTxtFileName=DIR_SPOOL+"notes.txt"
    Set fso=CreateObject("Scripting.FileSystemObject")
    If scrFileExists(sTxtFileName)<>"0" Then
        scrFileDelete(sTxtFileName)
    End If
    fso.CreateTextFile sTxtFileName
```

```

Set f=fso.GetFile(sTxtFileName)
Set ts=f.OpenAsTextStream(ForWriting,TristateUseDefault)
rc=GSrce("LOAD","FILE="+sDatFileName)
rc=GSrce("FIRST","")
While rc<>"#EOF#STORE#"
    ' show info if ANR|SUBKEY:1|INFO.OPT:INFO=J
    If VSrce("SUBKEY:1")=sKey1 And VSrce("INFO.OPT:INFO")="J" Then
        For i=1 To 10
            ts.WriteLine VSrce("INFO.INFO:"+CStr(i))
        Next
    End If
    rc=GSrce("NEXT","")
Wend
rc=GSrce("CLOSE","")
ts.Close
' invite file notes.txt in TextView!
' --> ctaisplay.ini->[TV@NOTES]->TEXTFILE=notes.txt
DLGVAR="LOC:NOTE=#REOPEN#"
End Sub 'ViewSelectedInfo

'-----
'- AIP2: Function <ViewSelectedInfo> from <aip_mpdv-AINFO_NOTES.scr>
'-----

Sub ViewSelectedInfo
    '-----
    ' write the note selected in the grid into the memo
    '     raw data from server: notes.lst
    ' processed for grid: notes_gr.lst
    ' text for TextViewer: notes.txt (-> is created here)
    '-----

    Dim asGrid,sKey1,rc,ss,sDatFileName,sTxtFileName
    asGrid=scrGetInfo("GetGridData","DLG=@FIL=DLG=AINFO_NOTES|LINE=-1")
    sKey1=scrDDItem("SUBKEY:1",asGrid)
    sDatFileName=DIR_SPOOL+"notes.lst"
    sTxtFileName=DIR_SPOOL+"notes.txt"
    If scrFileExists(sTxtFileName)="0" Then
        rc=scrFileDelete(sTxtFileName)
    End If
    rc=GSrce("LOAD","FILE="+sDatFileName)
    rc=GSrce("FIRST","")
    While rc<>"#EOF#STORE#"
        ' show info if ANR|SUBKEY:1|INFO.OPT:INFO=J
        If VSrce("SUBKEY:1")=sKey1 And VSrce("INFO.OPT:INFO")="J" Then
            For i=1 To 10
                ss = VSrce("INFO.INFO:"+CStr(i))
                rc = scrWriteDataIntoFile(ss,sTxtFileName)
            Next
        End If
        rc=GSrce("NEXT","")
    Wend
    rc=GSrce("CLOSE","")
    ' invite file notes.txt in TextView!
    ' --> ctaisplay.ini->[TV@NOTES]->TEXTFILE=notes.txt
    DLGVAR="LOC:NOTE=#REOPEN#"
End Sub 'ViewSelectedInfo

```

1.7 Special Fields of Application

1.7.1 Tips and tricks with the dialog control

This section describes tips and tricks to control dialogs.

- Starting a dialog timer for monitoring
(Example: Cycle 500 msec
→ Processing in <DynDlgTimer_XYZ >)
→ `DLGVAR = Item("DYNDLG.TIMER","500")`
- Starting a dialog autoclose timer
(Example: run time <7> seconds + return code <1> = CANCEL
→ Processing in < DynDlgFunctions__XYZ >)
→ `DLGVAR = Item("DLG.TIMER","7^1")`
(Example: run time <10> seconds + return code <0> = OK
→ Processing in < DynDlgFunctions__XYZ >)
→ `DLGVAR = Item("DLG.TIMER","10^0")`
(Example: Timer remains active also without dialog focus
Default is „`„.^.^1“` / Timer stops if dialog is not active/focused
Run time <10> seconds + return code <1> = CANCEL
→ Processing in < DynDlgFunctions__XYZ >)
→ `DLGVAR = Item("DLG.TIMER","10^1^0")`
- Creating temporary dynamic dialog variables
(Example: Variable <*XXX> with value <1>
→ Processing in all < DynDlg.._XYZ > functions)
→ `DLGVAR = Item(" *XXX","1")`
- Controlling of dialog buttons with ID <> ""
(Example: Button (BTN.<CANCEL>) with text <ESC> and font color <clRed>
→ Processing in all < DynDlgFunctions_XYZ >)
→ `DLGVAR = AddIt("BTN.CANCEL","ESC,clRed",cFFEnable)`
- Color text field in the dialog red → UserExitDynDlgBeforeInitialize
→ does not work if the field has the attribute "STATUS"
→ `If VDIg("DLG")="..." Then`
`DLGVAR = AddIt("INFO","",cFFVisible+"#COL-clRed")`

- Prevent opening the dialog → UserExitDynDlgBeforeInitialize
 → If VDlg("DLG")="..." Then
 DLGVAR=Item("RET","#CANCEL#")
- In the dialog script DynDlgFunctions_XYZ, one can react to escaping the selection list (e.g. scrap reason) via "ESC" :
 → Case "@@LIST_CANCEL"
 OnListCancel

1.7.2 Assignment of a script function to a key without DDLG

In **ctaipbut.ini**, the ID must start with '@':

F8=@WKP_CNR_VA_DEL,Voranmld. Delete

Verarbeitung im Skript **aip_system_<Projekt>.scr**:

```
Sub UserExitButtonClick
  Select Case VVar("UE:PAR","BTN.FKT")
    Case "@WKP_CNR_VA_DEL"
      OnButton_WKP
      UE_RET=Item("BTN.FKT","#FKT#->#EXIT#")
  End Select
End Sub 'UserExitButtonClick

Sub OnButton_WKP
  Dim sMnr,sCnr,sRes
  sMnr=VVar("UE:MNR","MNR")
  sCnr=GetVLos(sMnr)
  If sCnr="" Then
    scrMsgBox("kein Los vorangemeldet")
  Else
    sRes=DlgJaNein("delete advance logon","really delete batch logged on in advance?")
    If sRes="#JA#" Then
      DeleteVLos(sMnr)
    End If
  End If
End Sub
```

It is important to set the return value „BTN.FKT=#FKT#->#EXIT#“. Otherwise the error message "unknown button ID..." is displayed.

If the identifier starts with "@@" instead of "@", you do not need to set a return value.

1.7.3 How to use the functions GSrce, VSrce

You can use **GSrce()** to access a list file.

The following parameters are possible:

- GSrce ("LOAD", "FILE=XXX") 'XXX is the file that is loaded including directory
- GSrce ("FIRST", "XXX") 'XXX is an optional filter like e.g. MNR=102030
- GSrce ("NEXT", "XXX") 'XXX is an optional filter like e.g. MNR=102030

FIRST and NEXT provide a return code. If the return code is <> "#EOF#STORE#", then a further row has been found

VSrce() is then used to access the current row, e.g. VSrce("MNR") is used to read the machine number of the current row.

- GSrce ("CLOSE", "SAVE=TRUE") ' SAVE=TRUE is only set if the file is to be saved.

- sLine=GSrce("GETLINE", "") - read row number
- rc=GSrce("SELECTLINE", sLine) - select row („0“ – first row)
- rc=GSrce("DELETELINE", "") - delete current row
- rc=GSrce("DELETELINE", sLine) - delete specific row

An example is included in the description of the GSrce() function (chapter „1.3.2.55 GSrce(sFct,sParam)“)

1.7.4 Update grid at the push of a button

A grid can be updated by calling DLGVAR=Item("DLG.GRID", "RELOAD"). Requirement: A command has already been assigned to the GRD.CMD to get the list. This can be done, e.g. in the user exit DynDlgGridInit

```
...
SCRVARS = "GRD.CMD=" + "DLG=LIST;104|MOD=U|MNR=<MNR>|ANR=<ANR>|"
...
```

1.7.5 Read first row from list file

The script function scrQuickSearch can process the parameter "FIRST" instead of the filter:

```
asAuftrag=scrQuickSearch(DIR_SPOOL+"anr.lst", "FIRST")
```

It is not necessary to set an explicit filter after loading a single-row info list (e.g. nanr.lst, lnr.lst).

1.7.6 Script event when changing cell in the machine list

In the old GUI (AIP8.1, CTWIN), you can use the event "@@MNR.CELLCHANGE" to enable/disable buttons with reference to the selected machine.

```
Sub UserExitButtonClick
  Select Case VVar("UE:PAR", "BTN.FKT")
    Case "@@MNR.CELLCHANGE"
      CheckBlockButtonsActivation
  End Select
End Sub

Sub CheckBlockButtonsActivation
```

```

Dim rc
If GetFu29="J" Then
    rc=scrSetData("ActivateMainButton","PANEL=MNR|BTN=VNR_AN,VNR_AB|ACTIVE=-1")
Else
    rc=scrSetData("ActivateMainButton","PANEL=MNR|BTN=VNR_AN,VNR_AB|ACTIVE=1")
End If
End Sub

```

1.7.7 Script event when loading additional info

An event is triggered after the operation additional information has been loaded. Here, the file can be manipulated from the script before it is read by the terminal program. The type of the additional info and the file name are passed in the global variable "#AINFO#".

```

Sub UserExitButtonClick
    Select Case VVar("UE:PAR","BTN.FKT")
        Case "@@AINFO.LOADED"
            If GVars("#AINFO#","TYPE")="AI" Then
                If scrFileExists(GVars("#AINFO#","FILE"))="0" Then
                    ' ... change the file ...
                End If
            End If
        End Select
    End Sub

```

1.7.8 Extended customizing with label printing

The parameter "**PRN->PARAM**" is used for an extended customizing of a configured label with a posting event/dialog. Using this parameter, you can control if the "print order" is completely stopped or if only printing is stopped.

Parameter	Description
„PRN->PARAM=SKIP PRINTJOB“	Print order is completely stopped.
„PRN->PARAM=SKIP PRINTING“	Label printing is stopped. A server script configured in the label and a configured logging are performed. (Function available as of AIP V# 8.2.0.40)

Example: Label printing is stopped or cancelled for the customer-specific posting event/dialog "Entry of quantities (U_MENGE)".

```

Sub DynDlgBeforeSend_U_MENGE
    Select Case VDlg("PRNMODE")
        Case "L"
            DLGSND=Item("PRN->PARAM", "SKIP PRINTING")
        Case "N"
            DLGSND=Item("PRN->PARAM", "SKIP PRINTJOB")
        Case Else
            End Select
    End Sub

```

1.7.9 Notes on the centralized MDE

When using the central MDE, the first step in accessing the controller is to specify which PCC is responsible for MDE processing on the machine. To do so and to access the control (machine control), the following functions are available in the terminal scripts:

- `vbsGetCentralPccID(sFilter)`
- `vbsCentralPCCValues(sCMD, sPCCID)`
- `scrPCCValues(sValue)`

For a detailed description of the functions, refer to section *"Script functions"*.

You can use the functions to access the control (machine control) via the functions **GETVAL** and **SETVAL** for e.g.:

- setting outputs
- customer-specific connection of balances
- transfer of setting data to a machine with operation logon

Requirements:

- Installed SP 13 and included hotfixes
- MQTT – must be installed and activated
- The following licenses are required --> Licenses (authorization keys)
 - AIP-EBM#8.2, SCS-PCB (MDE-NOTIFICATION)
 - PDV-RPM#8.2, PDV-RPM#8.3 (PDV-RPM)
- The following program versions are required at least:
 - `ctaip.exe` - 8.2.2.6
 - `pcc.exe` - 7.2.4.6
 - `hymwmd72.dll/.so` - 8.1.1.144

1.7.10 Function to identify an order info

You can use the function below to read the order information of an order transferred.

The data can be identified and used via the following command:

```
asAnr = sys_GetAGDataAnywhere(sAnr,sMnr)

Function sys_GetAGDataAnywhere(sAnr,sMnr)
  Dim asAnr
  sys_GetAGDataAnywhere=""
  If sAnr="" Then Exit Function
  asAnr=scrQuickSearch(DIR_SPOOL+"anr.lst", "ANR="+sAnr)
  If asAnr="" Then asAnr=scrQuickSearch(DIR_SPOOL+"vlist."+sMnr+".lst", "ANR="+sAnr)
  If asAnr="" Then asAnr=scrQuickSearch(DIR_SPOOL+"nanr.lst", "ANR="+sAnr)
  If asAnr="" Then
    '*****
    asAnr=sys_GetAGDataFromDB(sAnr)
    '*****
  End If
  sys_GetAGDataAnywhere=asAnr
End Function
```

```
Function sys_GetAGDataFromDB(sAnr)
    sys_GetAGDataFromDB=""
    LSTVARS=""
    LSTVARS="LST.FILE="+nanr.lst"
    LSTVARS="LST.CMD="+DLG=LIST;11|MOD=A|ANR="+sAnr+"| "
    '*****
scrFktList
    '*****
    If VVar("LST.CMD:RET","RET")="0" Then
        sys_GetAGDataFromDB=scrQuickSearch(DIR_SPOOL+nanr.lst,"ANR="+sAnr)
    End If
End Function
```

The function is fast because it first searches the local lists for the operation. Only if the operation is not found locally, the data will be requested from HYDRA Server.

1.7.11 Correct use of the component list with/without resources in the function "Log operation on"

Depending on the machine configuration, either the mat.lst or the combined resource/material list (fhm.lst) is active when logging on an operation. You can use the following function to read the fields of the correct list.

Example to read articles:

```
Function GetVISFHMNTNRAAN(sMnr)
    Dim asMnr, sAtk, sFilter, rc
    asMnr=scrQuickSearch(DIR_SPOOL+"mnr.lst","MNR="+sMnr)
    If scrDDItem("VISFHMNTNRAAN",asMnr) = "J" Then
        FileName="fhm.lst"
    Else
        FileName="mat.lst"
    End If
    sFilter=""
    rc=GSrce("LOAD","FILE="+DIR_SPOOL+FileName)
    rc=GSrce("FIRST",sFilter)
    While rc<>"#EOF#STORE#"
        If VSrce("DLL")="" and VSrce("ART")="M" Then
            sAtk=sAtk+VSrce("ATK")+ "| "
            rc=GSrce("NEXT",sFilter)
        End if
    Wend
    rc=GSrce("CLOSE","SAVE=FALSE")
End Function
```

1.7.12 Staff badge number with leading zeros

You can change the badge number transferred and extend it to the badge number length defined in the basic settings. If required, the badge number is filled with leading zeros.

```
Function sys_fillKnr(sKnr)
    sys_fillKnr = StrFmtRight(sKnr,vbsIntDef(VTnr("LEN:KNR"),0),"0")
End Function
```

1.7.13 Change XML layout of script

After completing a function it may be necessary to switch to a specific XML page.

Request the current XML page: `VVar ("UE:DAT", "XML-GUI")`

All active XML pages are displayed separated by "comma".

Result:

`L_VIEW_MNR` -> icon view

`L_VIEW_MNR,L_MAIN` -> main view /overview

`L_VIEW_MNR,L_MAIN,L_ANR` -> detail view ANR

You can change to an XML page via `rc=scrSetData ("XML.ShowLayout", "LAYOUT=L_VIEW_MNR")`.

Example:

```
Function UserExitDynDlgAfterSend
Dim sXml, iDlgRuns
Select Case VSnd("DLG")
Case "A TR"
sXml = VVar("UE:DAT", "XML-GUI")
iDlgRuns = CInt(VVar("UE:DAT", "DLG-RUNS"))
If scrGetPart(sXml, ",", 2) <> "" Then
If iDlgRuns=0 Then
' if no dialog is open, change layout to main screen
rc = scrSetData("XML.ShowLayout", "LAYOUT="+scrGetPart(sXml, ",", 1))
End If
End If
End Select
End Function
```

Note: Changing the XML layout triggers the function "@@XML.LayoutChanged" in UserExitButtonClick.

In order to react to the button "Register PLC" from within the script, the global variable `GVars("$XMLGUI$PAR", "CAPTION")` can be read.

Example:

```
If GVars("$XMLGUI$PAR", "CAPTION") = "+" Then
DLGVAR=Item("MNR", VMnr("MNR"))
End If
```